SCAMP TUTORIAL WIP

Introduction

The SCAMP Pixel Processor Array

The concept of Pixel Processor Array (PPA) is a camera sensor where each pixel capturing light is also a processor. The SCAMP sensor is a prototype of such a PPA with an image resolution of 256x256. This means SCAMP has 65,536 of these pixel-processors positioned in 2D grid layout. Processors are not isolated but connected to their neighbours within this grid. This forms a powerful computational mesh referred to as the “Processing Array”, across which data can be transferred between individual processors, allowing them to operate collaboratively and solve a wide range of visual tasks.

This PPA architecture places a huge amount of parallel compute power directly at the start of the visual pipeline, allowing light to be captured and processed upon the sensor itself. Crucially, the sensor's output can consist of any form of information/data extracted by this on-sensor computation.

This is in contrast to traditional camera sensors with no on-sensor compute, where the entire images are output from the sensor and then processed externally. Such images consist of a massive amount of data, and the process of transferring them from sensor to external processing comes at huge costs in time and power consumption.

A major benefit of on-sensor compute is that it allows us to avoid this image transfer process. Instead the sensor itself can extract the information required for a specific task (such as features/bounding boxes etc) and then **only** output this high level information.

Such sparse information can be several orders of magnitude smaller in size than a whole image. Naturally the time & power costs involved in transfer are tiny by comparison, and so on-sensor compute can remove the data transfer bottleneck in the visual pipeline.This allows PPAs to excel at performing both extreme high-speed real-time applications and very low power applications, neither of which are possible with traditional sensors.

Single Instruction Multiple Data

The Processor Array of SCAMP is a 'Single Instruction Multiple Data' (SIMD) architecture. This means that instead of each of the 65,536 pixel-processors running separate programs, a single global program is executed simultaneously across all processors.

The execution of this global program is synchronised across all processors, ensuring that they receive and execute the same instructions simultaneously in parallel. However, each processor operates on its own local data. This is the essence of the SIMD architecture, as even though identical instructions are executed by all processors, the results can vary because each processor operates upon its own local set of data.

With SIMD each processor does not need to store its own program instructions, instead instructions are simply transmitted to the array and executed by all processors. This allows processor design to be greatly simplified, allowing for such a large number of processors to be placed into the sensor.

Local Memory & “Register Planes”

The individual pixel-processors within the array are identical, each equipped with the same functionality and set of local memory registers. SCAMP has a 256x256 processor array, giving 256x256 instances of each local memory register. For example, every processor has a local memory register "A" leading to a total of 256x256 "A" registers across the entire array.

Now, instead of focusing on a single instance of register "A" inside a specific processor, imagine zooming out to consider all the "A" registers across the 256x256 grid of pixel-processors. This collective view can be visualised as a 256x256 image (or a 2D array of data), where each processor stores one pixel of this image inside its own “A” register. The same concept applies to each of the other local memory registers. Thus, across the entire array, each local memory register can be considered to represent a separate 256x256 image.

We refer to these collections of local registers as "Register Planes". It is natural to think of operations performed by the processor array as pixel-wise operations between the images of different register planes. This is because each instruction to the array encodes both the operation to be performed and the specific memory registers to be used. Thus, all pixel-processors operate on the same memory registers simultaneously, and it is not possible for individual pixel-processors to operate on different memory registers from one another. For example, the instruction "add(B, C, D)" can be considered as adding together the 256x256 images in register planes C and D, storing the resulting image in register plane B.

TODO FIGURES

Instruction Kernels

Working with the SCAMP system involves writing C++ code that runs upon an embedded microcontroller connected to the processor array. This microcontroller is responsible for both sending instructions to the processor array for execution, and also retrieving data from the array and forwarding it to a host system (e.g. a PC over a USB interface).

Programs for this microcontroller are written in standard C++ code, with the addition of SCAMP "instruction kernels." These kernels contain sequences of SCAMP-specific instructions enclosed between the lines "scamp_kernel_begin" and "scamp_kernel_end", which declare the start and end of the kernel, respectively. An example of such a kernel is shown below.

```
scamp7_kernel_begin();
      mov(B,A);
      sub(A,F);
scamp7_kernel_end();
```

Whenever a kernel is reached during execution of the C++ program, the instructions of that kernel are sent to the Processor Array and executed. This is referred to as "calling" a kernel, and is the mechanism by which we control the Processor Array to perform useful computation.

A typical SCAMP program consists of multiple kernels, combined with standard C++ flow control, to command the Processor Array to perform computations for a specific task. When designing an algorithm for the SCAMP system, it's important to leverage the Processor Array's vast parallel computing power by offloading as much computation as possible to the array through these kernels.

SCAMP algorithm design can be a challenging task. The SIMD Processor Array's architecture is quite unique, and existing algorithms cannot typically be "ported" over without significantly redesigning them. The following chapters will cover all the functionality of SCAMP with example programs and explanations.

SCAMP Functions

The SCAMP library also includes various functions such as "scamp5_scan_events()", "scamp5_flood()". These functions are provided for convenience to perform various operations involving the Processor Array, and typically will include multiple kernels within them.

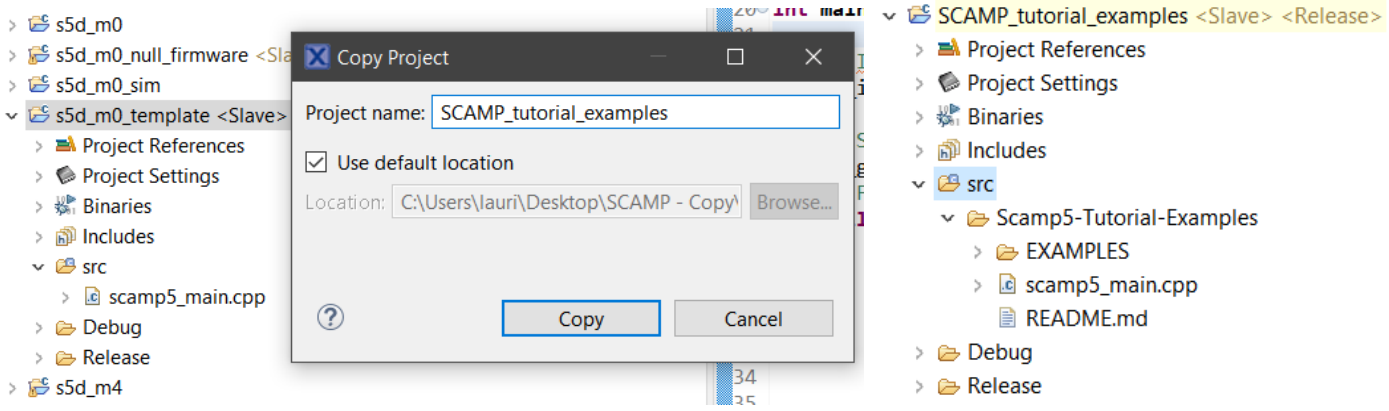
C++ Floating Point Operations. Don't Use Them!

Also note that the ARM microcontroller used in SCAMP5 & SCAMP7 systems, which will run your C++ code, does not have floating point acceleration. This means floating point operations are incredibly slow. Adding just 100 floating point numbers takes ~100 Microseconds! While adding 100 integer numbers is less than 0.1 Microseconds (ie. >1000X faster). I recommended you replace any floating point arithmetic with integer based approximations. Even a handful of floating point operations can impact your program's performance.

Running The Examples

There are many examples included in this document, each of which has been implemented on both SCAMP7 and SCAMP5. Their code is stored on github, HERE for SCAMP7 and HERE for SCAMP5, but links to specific examples will be also provided along the descriptions across this document.

To easily run these examples first follow the steps laid out in the Scamp Documentation HERE if you have not already done so, for installing the IDE, setting up a work space etc. Then make a copy of the template project "s5d_m0_template" (or "s5d_m0_hello_world"), renaming it to something appropriate (e.g. "SCAMP_tutorial_examples"). You can then simply download the examples repository from github and unceremoniously copy its contents into the project's src folder as shown.



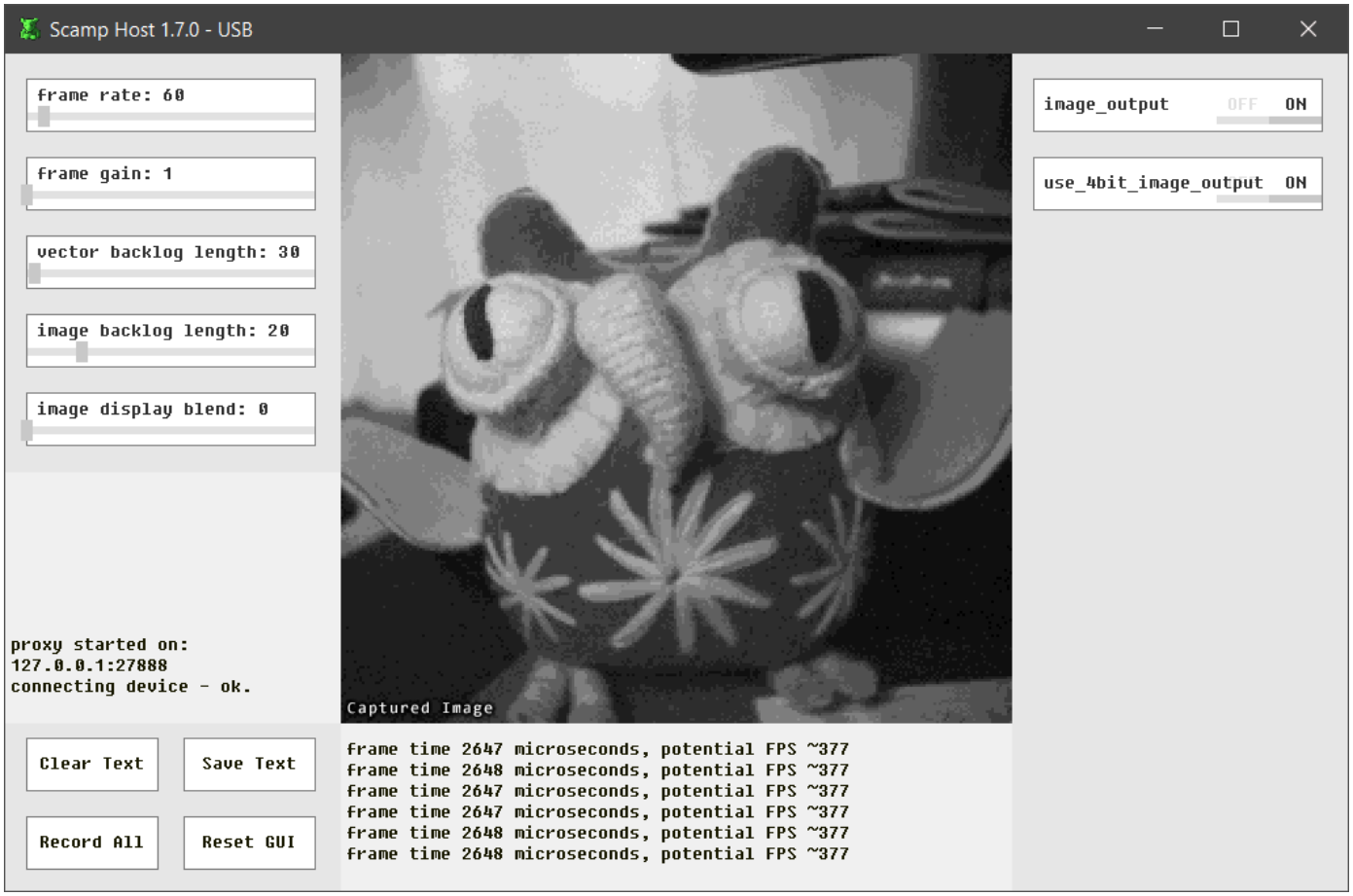
Each of the examples is contained in a different header file, each of which is included in "scamp_main.cpp". To select which example to use, simply comment out all of them except the one you are interested in, and then build the project. Remember you can also Ctrl+Click on an example's "#include" to go straight to its code file.

```
12 //#include "EXAMPLES/EX_EVENT_READOUT.hpp"
13 //#include "EXAMPLES/EX_GLOBAL_OR.hpp"
14 //#include "EXAMPLES/EX_INTERLEAVED_DATA.hpp"
15 //#include "EXAMPLES/EX_KERNEL_COST.hpp"
16 //#include "EXAMPLES/EX_OPTIMIZING_AREG_SHIFT.hpp"
17 //#include "EXAMPLES/EX_RF_AND_RM.hpp"
18 //#include "EXAMPLES/EX_SHAPE_EXTRACTION.hpp"
19 #include "EXAMPLES/EX00_IMAGE_CAPTURE.hpp"
20 //#include "EXAMPLES/EX01_IMAGE_CAPTURE_AND_AREG.hpp"
21 //#include "EXAMPLES/EX02_DREG_BASICS.hpp"
22 //#include "EXAMPLES/EX03_FLAG.hpp"
```

0.My First Program

We start with a simple program to capture and display images. This program consists of the “main()” function where execution begins, inside of which is the “main frame loop”. This loop runs continuously, each iteration capturing a new image within the processor array, with each PE holding a single pixel. This image is then output from the array and transferred to PC via USB, where it is then displayed upon the SCAMP Host’s GUI as shown below. Let us go through the features introduced in this example.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP7**



Host GUI Displays

To show images output from the sensor on PC we must add “displays” to the Host’s GUI using the function

```
vs_handle vs_gui_add_display("Display Title", int x_position, y_position, int size);
```

This adds a new display to the Host GUI with a specified title, position and size. It also returns a “vs_handle”, which is a unique identifier for this newly added display. We can add multiple displays, and output different images to each one separately using these identifiers.

Displays are added inside main(), before entering the frame loop. In this example we add a single display and store its handle inside “display_00” to use later for image output.

```
//SETUP IMAGE DISPLAYS
int disp_size = 2;
vs_handle display_00 = vs_gui_add_display("Captured Image",0,0,disp_size);
```

Host GUI Control Elements



The SCAMP Host’s GUI also provides various ways to interact with the program running upon the SCAMP system’s microcontroller. This is done by adding interactable control elements to the Host GUI, and linking these controls to various integer variables in the program. The types of controls are listed in the table below.

Type	Function used to add to GUI
Button	vs_handle vs_gui_add_button(char* name)
Switch	vs_handle vs_gui_add_switch(char* name, bool init, int* linked_variable)
Slider	vs_handle vs_gui_add_slider(char* name, int min, int max, int initial_value, int* linked_variable)

Note when adding a new control we do not provide a position or size, instead all controls appear down the right hand side of the Host GUI in the order in which they were added.

Switches and Sliders must be linked to a specific variable in the program when added. This is done by providing the variable’s address in memory using the “address-of” operator in C++ “&”, as shown below.

```
int box_x = 0;
vs_gui_add_switch("Box_Pos_X", 0 , 255, 50 , &box_x);
```

When a user interacts with Switches and Sliders the variables linked to those controls will be updated accordingly. For example clicking a switch toggles between its ON and OFF states, and updates its linked variable’s value to 0/1(ON/OFF). Specifically this update occurs inside of “vs_frame_loop_control();” which is called at the start of the frame loop.

Button controls are not linked to variables. Instead we can create a function that is triggered whenever the button is pressed using “vs_on_gui_update()” which will be demonstrated in a later chapter.

In this example two switches are added, each linked to a different variable, allowing the user to alter if and how the program outputs an image from the sensor.

```
int image_output = 1;
vs_gui_add_switch("image_output",1,&image_output);
int use_4bit_image_output = 1;
vs_gui_add_switch("use_4bit_image_output",1,&use_4bit_image_output);
```

Image Capture

Each PE contains a special purpose “PIX” register, in which light entering the PE is accumulated. The instruction “get_image(X,Y)” commands each PE to copy this accumulated signal into a certain register. This effectively captures an image frame, with each PE now storing 1 pixel of the image. This function also resets PIX, effectively triggering the start of capturing the next frame.

This example contains a single kernel inside the main frame loop.

```
scamp7_kernel_begin();
      get_image(A,F);//A = pixel data of latest frame, F = intermediate result
scamp7_kernel_end();
```

This will capture a new image into register A every iteration. Note “get_image” is actually a macro that contains a number of instructions. More advanced imaging such as HDR is possible but this is left to a later chapter.

Image Output



Scamp7_output_image



output_4bit_image_via_DNEWS

There are two functions we can use to output a register plane image and display it on the Host GUI.

Function	Time Cost	Precision
void scamp7_output_image(areg_t reg,vs_handle display)	~26,654µs	16 Bit
void output_4bit_image_via_DNEWS(areg_t reg,vs_handle display)	~2,646µs	4 Bit

Both of these functions take two parameters, the register plane to output, and the handle of the display it should be displayed on. The function “scamp7_output_image” is part of the scamp library, but “output_4bit_image_via_DNEWS” is a function provided by this tutorial which outputs a lower precision image, but takes significantly less time. This is illustrated in the images above.

Timing

It is often important to time various parts of a program to identify bottlenecks. The class “vs_stopwatch” provides a simple way to perform such timings. This class has two methods, “void reset()” and “int get_usec()” which returns the microseconds elapsed since the last call of reset().

The example uses a single stopwatch, defined at the top of the file, to time each iteration of the frame loop.

```
vs_stopwatch frame_timer;
```

This is done by resetting the stopwatch to zero at the start of the loop.

```
 frame_timer.reset();//reset frame_timer
```

And then recording the time that has passed at the end of the loop.

```
int frame_time_microseconds = frame_timer.get_usec(); //get the time taken this frame
```

Text Output

In addition to images, we can also output text to display upon the Host GUI with the function

```
void vs_post_text(char*format,...)
```

This uses the same formatting as the standard C++ printf, and is typically used for debugging and showing various information about the program while it is running.

Inside the frame loop of the example we first perform vs_post_text(“!clear”). The argument “!clear” is a special case that will trigger the Host GUI to clear any text currently being displayed.

We then output text containing the elapsed frame time and the approximate FPS the program is running at, calculated using the frame_timer stopwatch described previously.

```
//clear the text display on the Host
vs_post_text("!clear");
//calculate the possible max FPS
int max_possible_frame_rate = 1000000/frame_time_microseconds;
//display this values on host
vs_post_text("frame time %d microseconds, potential FPS ~%d
\n",frame_time_microseconds,max_possible_frame_rate);
```

```
frame time 2647 microseconds, potential FPS ~377
```

Sensor Output Vs FPS

The example allows us to switch between the two methods of image output, or output no image at all. This is done using the variables “image_output” & “use_4bit_image_output” that were linked to switches on the GUI.

```
if(image_output)
{
	//output register plane as image
	if(use_4bit_image_output)
	{
		//output AREG quickly using 4bit approximation
		output_areg_via_bitstack_DNEWS(A,display_00);
	}
	else
	{
		//output AREG at higher accuracy but taking significantly longer
		scamp7_output_image(A,display_00);
	}
}
```

Switching between these options you should observe significant changes in frame-rate.

Image Output Function	Frame Time	Frame-Rate
SCAMP 7_output_image	~26,654µs	~37
Output_areg_via_bitstack	~2,646µs	~377
None	~4µs	~250,000?

From this it is easy to see how the cost of image output is by far the biggest performance bottleneck. This demonstrates again that any algorithm designed for SCAMP should ideally not rely on the output of images, as the transfer of such a large quantity of data immediately torpedoes performance.

Instead Image output should be used only for debugging & visualisation, and the algorithm itself should output sparse data encoding higher level information than simple pixel values.

vs_ functions

Finally the example’s code also contains a few other important functions listed below.

vs_ function	Description
vs_init()	Standing for “vision system initialization”, this performs various calibrations and system setups, it should always be called at the start of main().
vs_frame_loop_control();	Performs various tasks including updating the values of variables linked to GUI controls, and limiting the rate at which the frame loop runs. Should always be called at the beginning of the frame loop.
vs_disable_frame_trigger();	Disables the frame trigger, allowing the frame loop to run as fast as possible.

1.Analogue Register Basics

Each PE of the Processor array has a set of analogue memory registers (AREG). These registers can store continuous values, but have a positive upper and a negative lower bound. Whenever the content of an analogue register is read out of the processor array, it is converted to a signed 8-bit value (-128 to 127). Data can be transferred between these registers (even across PEs), and various operations can be performed between the contents of different registers.

A major benefit of using analogue registers is that many operations, such as addition and subtraction of values, can be performed far faster and more efficiently than their digital equivalent. For example, assuming analogue registers to store values equivalent to 8-bit signed numbers, addition of two registers can be performed in just two native analogue instructions. By comparison, adding two 8-bit numbers in digital memory involves iterating each of the 8 bits one at a time, at each step computing the bit for the result and keeping track of any carried bit. This is a far more complicated process requiring many digital instructions per step.

The AREG inside each PE are shown below. Many operations are restricted to the 6 general purpose registers.

AREG	Special Property
A,B,C,D,E,F	None / General Purpose
NEWS	Can be accessed by neighbourings PEs. Used for data transfer.
PIX	Light Capture

AREG Operations

The table below lists the basic operations that can be performed between AREG, including addition, subtraction and division by two. Note, there is no operation to perform multiplication of two registers. The "Δ" indicates that the operation results in an offset/error being added to a register's content, and "-" indicates a register is unchanged by the operation.

Instruction	Description	X=	Y=	Z=	Restrictions
mov(X,Y)	Copy one register into another	Y	-	-	X≠Y
add(X,Y, Z)	Add two registers and store the result	Y+Z	-	-	Y≠Z
sub(X,Y, Z)	Subtract one register from another	Y-Z	-	-	X≠Z
neg(X,Y)	Copy the negative of one register into another	-Y	-	-	X≠Y
abs(X,Y)	Copy the absolute value of one register into another	|Y|	-	-	X≠Y
divq(X,Y)	Divide a register by 2 and store the result	Y/2+Δ	-	-	X≠Y
div(X,Y,Z)	Divide a register by 2 and store the result. Uses an intermediate register to improve accuracy.	Z/2	-Z/2+Δ	Z+Δ	X≠Y≠Z
diva(X,Y,Z)	This is the most accurate of divide-by-two instruction	X/2	-X/2+Δ	-X/2+ Δ	X≠Y≠Z

bus(X,Y)	Moves an inverted copy of one register into another. Native operation performed in a single instruction.	-Y+Δ	-	-	X≠Y

Analogue Issues

Note there are a number of considerations that must be made when using analogue registers.

- **Analogue Decay**
  - The content of analogue registers decays with time, this makes them unfit for the long term storage of data. Each PE has a set of digital registers (covered in the next chapter) which can be used for this purpose.
- **Analogue Saturation**
  - There is a limited range of values that analog registers can represent, and any computation involving values outside of this range will produce incorrect results. For example, consider repeatedly adding positive values to an analogue register. The register's value will increase with each addition until the range's upper bound is reached, at which point the register's value can increase no further and addition produces an incorrect result.
- **Analogue Noise**
  - Every analogue operation will impart some small amount of noise into its result. When performing long computation with many analogue operations, each operating upon results from previous operation, this noise can accumulate and begin adversely affecting results. Essentially the signal to noise ratio of results will go down as more and more analogue operations are performed. Techniques such as discretization can be used at stages of the computation to remove this noise accumulation.

These factors are discussed in greater detail in a later chapter.

Example Program : AREG Basics

Let us now demonstrate some basic analogue operations, building upon the introductory example from the previous chapter. In this new example we again continuously capture new image frames, but now also store the previous captured frame. We then subtract the previous frame from the current frame, effectively creating an image of the differences between their pixels. Optionally we also perform "abs()" upon this difference image. As shown below this program now outputs three images to three different displays on the Host GUI, the current frame, the previous frame, and the difference between the two.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP7**



This program contains two kernels inside the frame loop. The first shown below captures a new image frame, computes the difference image between this new frame and the stored previous frame, then replaces the

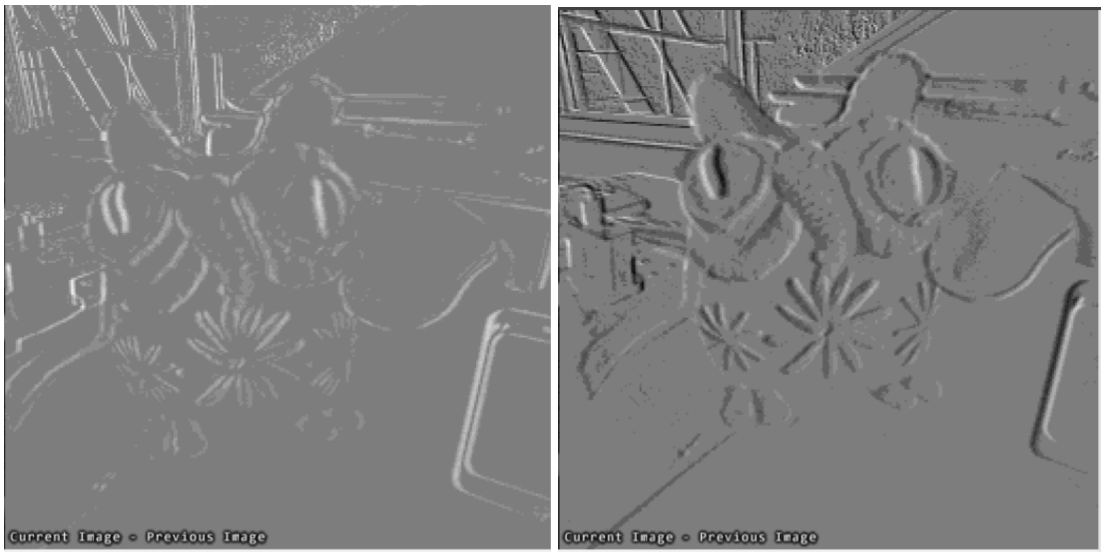
stored previous frame. The AREG A,B,C are now all in use, each storing different data.

```
//Analogue Register (AREG) usage
//A - used to store the latest frame's pixels
//B - used to store the previous frame's pixels
//C - used to store the difference between frames A-B
scamp7_kernel_begin();
      //A = pixels of latest image frame, F = intermediate result
      get_image(A,F);
      //C = (A - C) == (latest frame pixel - previous frame pixel)
      sub(C,A,B);
      //B = A, update the previous frame by replacing it with the current frame
      mov(B,A);
scamp7_kernel_end();
```

The second kernel is used to perform the absolute value operation on the difference image, but is only run if the user has selected to do so using a switch on the Host GUI. Note that as “abs(C,C)” is not valid, this kernel requires an extra instruction to move the result from the absolute value operation back into C.

```
if(perform_abs)
{
   scamp7_kernel_begin();
      abs(F,C);//F = |C|
      mov(C,F);//copy the result in F back into C
   scamp7_kernel_end();
}
```

Below shows a comparison of difference images produced with and without performing this absolute value operation.



2.Digital Registers Basics

In addition to AREG each PE in the array has a number of digital registers (DREG). These DREG store discreet 1-bit values, in contrast to AREG which store continuous values. Below we see an 1-bit image stored across a DREG plane, alongside an image stored in an AREG plane for comparison.





DREG instructions can be used to perform standard boolean logic operations between the content of such registers (AND, OR, NOT etc). There are a total of 14 DREG in each PE of SCAMP -5 as shown in the table below. Some of these registers have special properties which will be covered in later chapters.

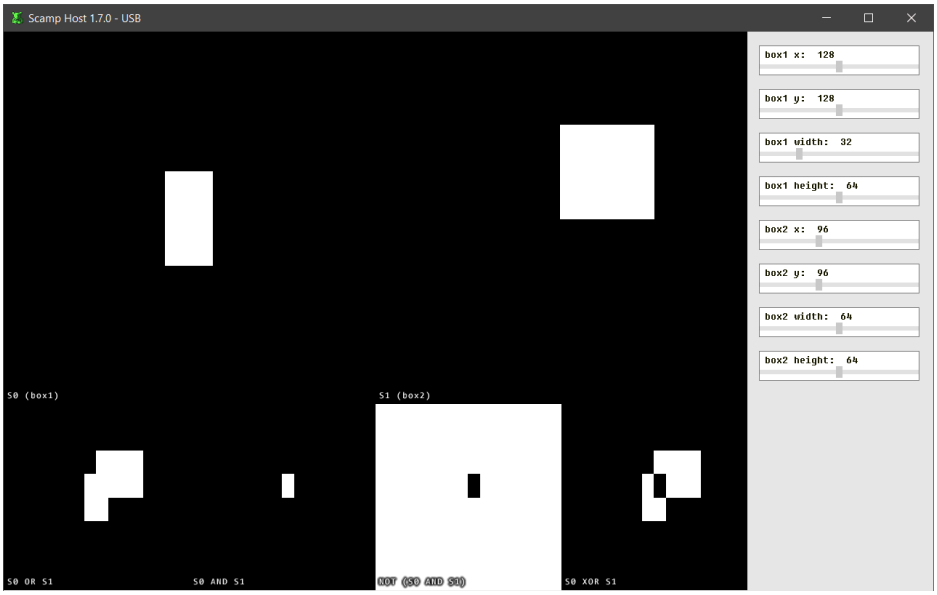
DREG	Special Property
S0, S1, S2, S3, S4, S5, S6	None
FLAG	Enables/Disables execution of Analogue instructions in PE
RN, RS, RE, RW	Controls direction of digital data transfer
RF	Enables/Disables execution of Digital instructions on **RM** in PE
RM	Digital instruction execution dependant upon **RF**

DREG Operations

The table below lists the basic operations that can be performed between DREG.

Instruction	Description	X=	Notes
NOT(X,Y)	Logical NOT.	!Y	Restriction X≠Y
OR(X,Y,Z)	Logical OR between two registers.	Y||Z	
NOR(X,Y,Z)	Logical NOR between two registers.	!(Y||Z)	
AND(X,Y,Z)	Logical AND between two registers.	Y&Z	RM & RF Uses as temp registers.
NAND(X,Y,Z)	Logical NAND between two registers.	!(Y&Z)	RM & RF Uses as temp registers.

Example Code : DREG Basics



[EXAMPLE CODE SCAMP5] [EXAMPLE CODE SCAMP 7]

This example serves as an introduction to DREG. We first load two rectangular regions, one into S0, and another into S1 as shown in the top two displays. The position and dimensions of these rectangular regions can be changed via GUI sliders on the right while the program is running. DREG instructions are then used to perform various boolean logic between S0 & S1 as shown across the bottom displays. Let's walk through the new concepts introduced in this example's code.

GUI Sliders



The first new functionality is the addition of GUI sliders. Like GUI switches there provide an extremely valuable tool for visualisation and debugging.

The syntax to add a slider to the GUI is as follows.

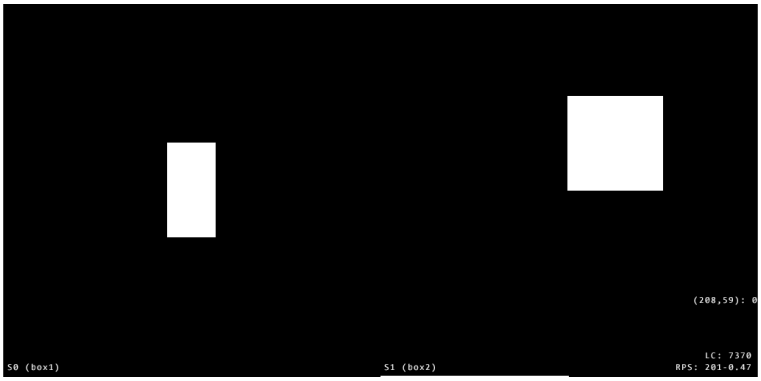
```
vs_gui_add_slider("Title", min_val, max_val, start_val, &linked_varible);
```

Typically the code adding such sliders is within main(), before the frame loop begins. This code is then run only once, upon the program starting.

In the example we use four variables to define each rectangular region, and link each variable to a specific slider as shown below, allowing the user to manipulate them.

```
int box1_x, box1_y, box1_width, box1_height;
vs_gui_add_slider("box1 x: ", 0, 255, 128, &box1_x);
vs_gui_add_slider("box1 y: ", 0, 255, 128, &box1_y);
vs_gui_add_slider("box1 width: ", 0, 128, 32, &box1_width);
vs_gui_add_slider("box1 height: ", 0, 128, 64, &box1_height);
```

DREG Load Region



In the main loop, these rectangular regions of “1s” are loaded into the S0 and S1 DREG, using the function

```
scamp7_load_region(dreg_t DREG,int top_row,int right_coloum,int bottom_row, int left_coloum);
```

In this function a rectangular region is defined by giving the index of its top and bottom rows, along with its leftmost and rightmost columns. The given DREG will then be set to 1 in all PEs within this region, and 0 in all PEs lying outside of it.

This region formulation however can feel quite clunky, as in most visual applications we think of regions in terms of location and size within the image.

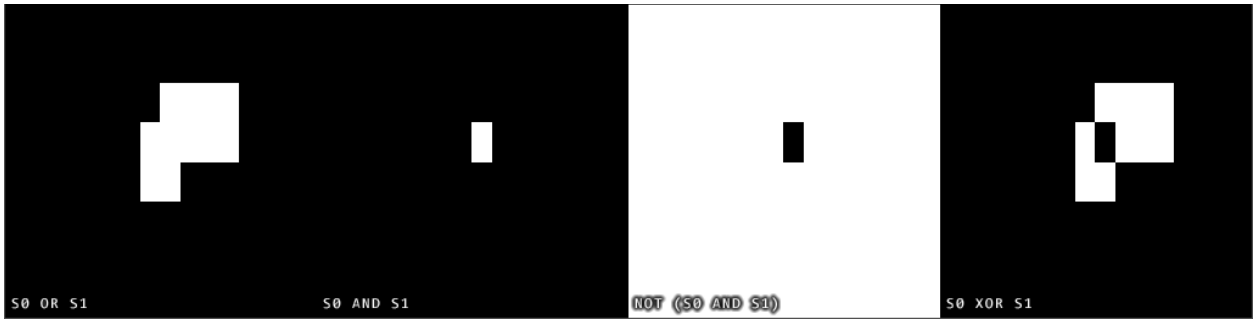
As such we make use of a wrapper function taking a different set of parameters, which internally calls “scamp7_load_region”.

```
void DREG_load_centered_rect(dreg_t reg,int x,int y,int width,int height)
{
    int top_row = y-height/2;
    if(top_row < 0)
    {
       height += top_row;
       top_row = 0;
    }
    int right_column = x-width/2;
    if(right_column < 0)
    {
       width += right_column;
       right_column = 0;
    }
    int bottom_row = top_row+height;
    int left_column = right_column+width;
    scamp7_load_region(reg, top_row, right_column, bottom_row, left_column);
}
```

And use this function each frame to load the rectangular regions into both S0 and S1

```
//load box1 & box2 into DREGs
DREG_load_centered_rect(S0,box1_x,box1_y,box1_width,box1_height);
DREG_load_centered_rect(S1,box2_x,box2_y,box2_width,box2_height);
```

DREG Logic Operations



After loading these regions a number of DREG instructions are performed in a single kernel, generating different results in the S6,S5,S4 & S3 registers, which are later displayed to the user.

```
	//COMPUTE VARIOUS LOGIC OPERATIONS
	scamp7_kernel_begin();
		OR(S6,S0,S1);//S6 = box1 OR box2
		AND(S5,S0,S1);//S5 = box1 AND box2
		NOT(S4,S5);//S4 == NOT(box1 AND box2)
		AND(S3,S6,S4);//S3 == box1 XOR box2
	scamp7_kernel_end();
```

As with AREG instructions, DREG instructions are executed by all PEs simultaneously in parallel. The first argument to each instruction indicates which DREG results are stored into.

```
OR(S6, S0, S1)//S6 = box1 OR box2
```

Instructs every PE to perform an OR operation between the bits of its S0 and S1 registers, and then stores the result in S6.

```
AND(S5, S0, S1)//S5 = box1 AND box2
```

Performs AND between the bits of S0 and S1, storing the result in S5.

```
NOT(S4, S5)//S4 == NOT(box1 AND box2)
```

Performs NOT upon S5 and stores the result in S4.

Finally, by combining the results from previous operations, we can construct an XOR operation performed between S0 & S1. This is performed by the instruction

```
AND(S3, S6, S4)//S3 == box1 XOR box2
```

To see that this is performing an XOR note that

- S6 = (S0 OR S1)
- S4 = NOT S5

So… AND(S3,S6,S4) == “AND(S3, (S0 OR S1), NOT S5)”

And also that

- S5 = S0 AND S1

So… AND(S3,S6,S4) == “AND(S3, (S0 OR S1), NOT(S0 AND S1))”

Which we may now recognize as XOR(S3,S0,S1), as S3 = 1 when only 1 of the bits (S1 or S0) is 1.
So… AND(S3,S6,S4) == “XOR(S3,S0,S1)“

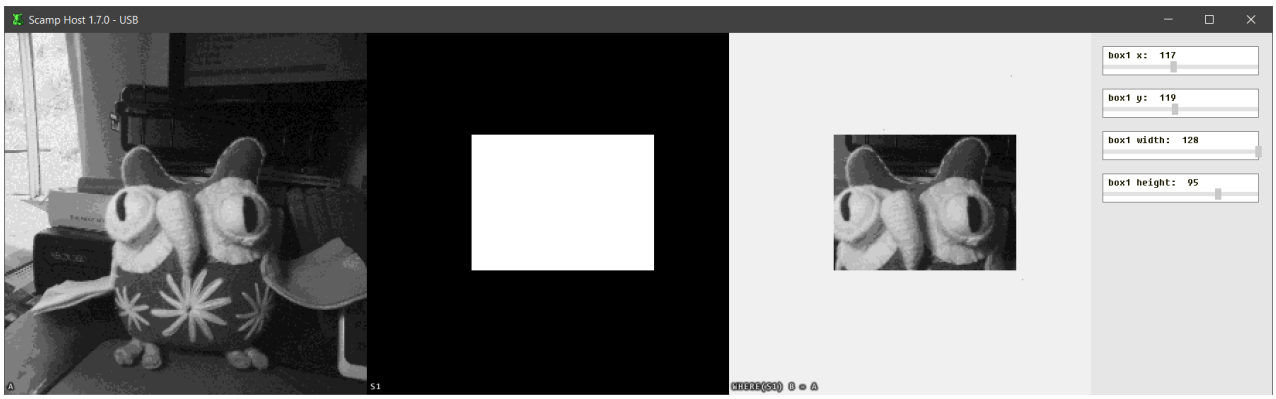
3.FLAG & AREG

Inside every PE there is a particular digital register referred to as the FLAG register. This register functions in an identical manner to other DREG but serves an additional special function. The FLAG register can be used to enable/disable execution of analogue instructions.

As each PE has its own separate FLAG register, this allows conditional execution of instructions on a per PE basis, a very powerful tool. Specifically, when a PE receives an analogue instruction, it will execute that instruction if its FLAG = 1, and discard the instruction if FLAG =0.

Example Code : FLAG

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



This example gives a simple introduction to FLAG and its use in conditional execution of analog instructions. We first capture an image into AREG A, and load a region of 1s into DREG S1. We then copy S1 into FLAG, and copy the captured image of A into B.

However, this analogue instruction to copy A into B is only executed in PEs where FLAG = 1 (those PEs within the loaded region), resulting in the result shown in the rightmost display.

Following a familiar setup, in the frame loop we first acquire an image into AREG A

```
            scamp7_kernel_begin();
                  get_image(A,E);
             scamp7_kernel_end();
```

And then a rectangular region of 1s into DREG S1

```
DREG_load_centered_rect(S1,box1_x,box1_y,box1_width,box1_height);
```

Next we demonstrate the use of the FLAG register

“WHERE” and “ALL”

The FLAG register can be used in digital instructions the same as any other DREG, however there are a number of extra instructions associated with it.

- The “WHERE(X)” instruction moves X into FLAG and is equivalent to “MOV(FLAG,X)”.
- The “ALL()” instruction is equivalent to “SET(FLAG)”

These extra instructions are primarily used to increase code readability, making it clear where conditional execution of analogue instructions is being performed.

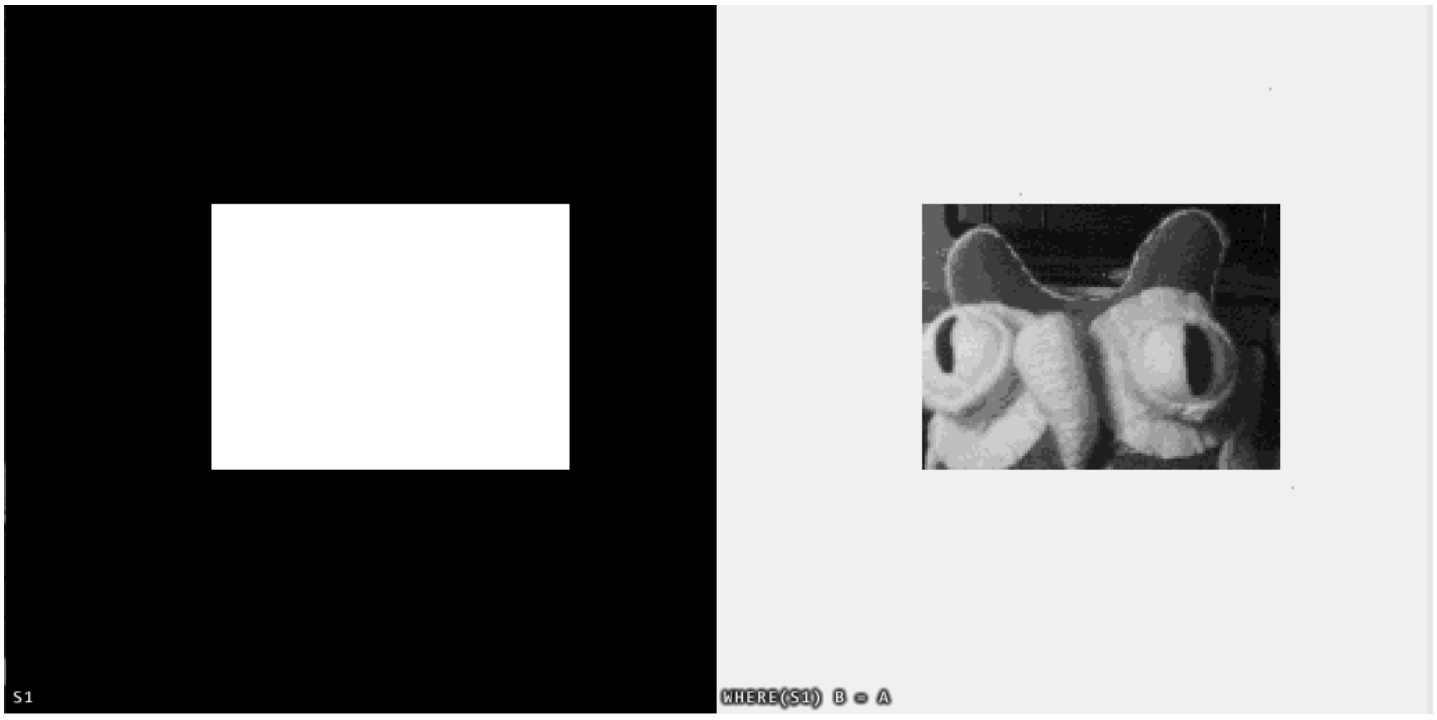
In the example code the main frame loop runs the following kernel

```
scamp7_kernel_begin();
    //copies S1 into FLAG, Identical to "MOV(FLAG,S1)"
    WHERE(S1);

       //AREG instructions are only performed in PEs where FLAG == 1
       //Thus "mov(B,A)" will now copy A, into B, ONLY where S1 == 1
       mov(B,A);

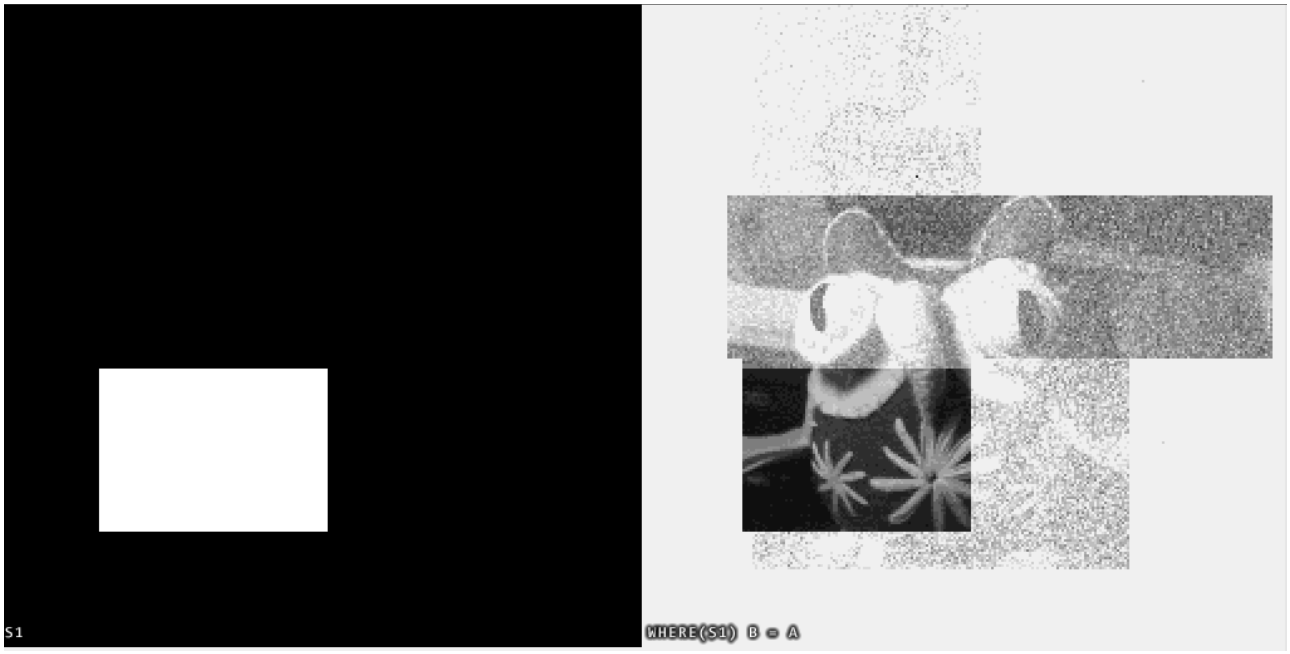
    //sets FLAG = 1 in all PEs, Identical to "SET(FLAG)"
    ALL();
scamp7_kernel_end();
```

The first instruction “WHERE(S1)” copies the content of each PE’s S1 register into its FLAG register. The next instruction “mov(B,A)” is analogue and will only be executed in PEs where FLAG = 1. As a region of 1s was loaded into S1, and S1 was copied to FLAG, only PEs within this region will execute “mov(B,A)”. This is shown below in the displays for S1 and B. Finally the “ALL()” instruction is used to reset FLAG=1 across all PEs.



AREG Decay

As you move the rectangular region about in this example, the PEs inside which A is being copied into B will also change. Those PEs where A was being copied into B previously will then keep hold of this old data. Such old data does not remain fixed, it will drift up in value at a different rate per PE as shown below.



This is our first insight into the “decay” of analogue data which occurs over time, and is one of the issues that must be considered when designing algorithms for SCAMP .

Example Code : Thresholding

Previously we introduced the FLAG DREG and how it is used for conditional execution of analogue instructions on a per-PE basis. We also saw how the content of FLAG can be altered, using “WHERE(X)” to copy another DREG X into FLAG, and using “ALL()” to reset FLAG to 1 in all PEs.

However this only demonstrated how to change FLAG’s content using digital instructions and the content of other DREG. This chapter introduces “where(Y)”, which is used to set FLAG based upon the content of an AREG.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**

This example demonstrates using “where” to threshold a captured image in AREG, to a binary image in DREG.



Outside of the frame loop a slider is linked to the threshold variable to allow the user to alter it.

```
int threshold_value = 64;
vs_gui_add_slider("threshold_value",-127,127,threshold_value,&threshold_value);
```

Then the process of image capture and thresholding is performed by the following code inside the frame loop. Let's walk through this piece of code now.

```
//load threshold value into C across all PEs
scamp7_in(C,threshold_value);

scamp7_kernel_begin();
    //A = pixel data of latest frame, F = intermediate result
    get_image(A,F);
    //F = (A - C) == (latest frame pixel - threshold)
    sub(F,A,C);
    //sets FLAG = 1 in PEs where F > 0 (i.e. where A > C), else FLAG = 0
    where(F);
       //copy FLAG into S0
       MOV(S0,FLAG);
    all();
scamp7_kernel_end();
```

“scamp_in”

The function “scamp7_in” is first used to load a value into a specific AREG, across all PEs. The range of accepted values ranges from -127 to 127, and

```
scamp7_in(C,threshold_value)
```

The code used here will load the threshold value, controlled by a GUI slider, into the AREG C

allowing FLAG to be set based upon AREG content

“where”

The instruction “where(Y)” is the analogue equivalent to “WHERE(X)”, and allows the content of the FLAG register to be set based on the content of a given AREG Y. Specifically, “where(Y)” will set FLAG=1 in all PEs where the value of AREG Y is positive, and FLAG=0 in all other PEs. This provides a very useful mechanism for extracting specific information & converting content from AREG to DREG. Note, unlike all other analogue instructions, the execution of “where(Y)” in a PE does not depend on FLAG, it is always executed.

In the example code an image is captured into AREG A

```
get_image(A,F);
```

Next we subtract the C from A, storing the result in F.

```
sub(F,A,C);
```

“scamp_in” was used to load the threshold value into AREG C of every PE.
It follows that F = (A - threshold).

```
where(F);
```

Now we use “where(F)” to set the content of FLAG. This instruction will set FLAG=1 in PEs where F>0, and FLAG=0 in all other PEs. The content of F = (A - threshold), so any PE where F > 0 must be a PE in which A > threshold.

As the image captured into A, FLAG=1 in those PEs whose pixel of the captured image are higher in value (i.e. “brighter”) than the threshold. Across the PE array, this generates a binary thresholded image within FLAG.

This result in FLAG is copied into S0, and then later displayed to the user as shown in the right display below.

```
MOV(S0,FLAG);
```



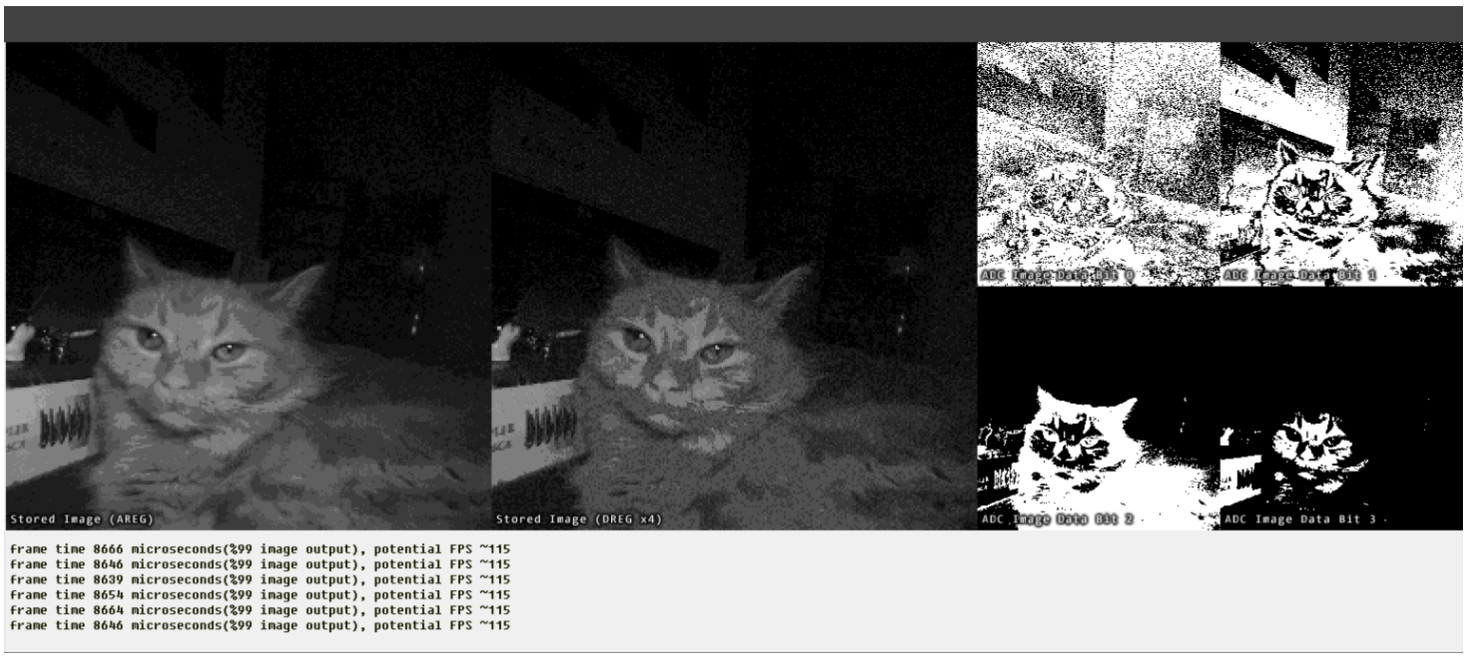
The final instruction in this kernel simply resets the content of FLAG, setting FLAG=1 in all PEs.

```
all();
```

Example Code : Image ADC & DAC

EXAMPLE CODE SCAMP5 EXAMPLE_CODE SCAMP 7

This example demonstrates how “where()” can be used to perform ADC (Analogue to Digital Conversion). Analogue image data is converted to a 4-bit digital approximation, using a separate DREG plane for each bit of the image, as shown in the 4 smaller displays on the right. An analog image can also be reconstructed from this digital data as shown in the middle display.



Captured analogue image data is converted into a digital representation across four DREG planes, S0, S1, S2, and S3, each representing a single bit associated with the values 128, 64, 32, and 16, respectively. This conversion is done through a series of comparisons between data and values, utilising the “where()” instruction to mark the locations of PEs whose bits should be set with 1s in the FLAG register plane.

```
scamp7_in(F,127);
//Compute Bit 3
scamp7_kernel_begin();
      mov(D,A);//Create working copy of captured image
      where(D);//PEs whose working image data is > 0
          MOV(S0,FLAG);//These PEs must have the 128 (Bit 3) DREG set
          NOT(RF,FLAG);
      WHERE(RF);//PEs whose working image pixel is < 0
          //add 127 to make data positive and allow later bits to be determined
          add(D,D,F);
      all();
scamp7_kernel_end();
```

First, we determine the most significant bit, S0, which represents the value 128. This is done by applying the “where()” function directly to the unaltered image data, setting FLAG = 1 in PEs with positive image data. Since the analogue data is assumed to range from -127 to 127, the bit representing 128 is set whenever the data is positive. The FLAG register now contains the values for this bit and is directly copied into S0 for storage. For PEs with negative data, the bit’s value (~127) is added to shift the data to a positive range, enabling the “where()” function to be used to determine the remaining bits for these PEs.

```
//Compute Bit 2
scamp7_kernel_begin();
      divq(E,F);
      mov(F,E);//F ~= 64
      sub(E,D,F);
      where(E);//Where working image > 64
          mov(D,E);//Subtract 64
          MOV(S1,FLAG);//Update Bit 2
      all();
scamp7_kernel_end();
```

The remaining bits, S1, S2, and S3, are then computed sequentially. For each bit, the process involves subtracting the bit's value from the working image data and using the "where()" function to identify PEs where the data exceeds the current bit value. These locations are marked with 1s in the FLAG register plane. For the flagged PEs, the value is subtracted from their analogue data, and the corresponding DREG bit is set. This process is repeated for each bit, converting the analogue image data into a complete digital representation.

```
scamp7_in(C,-127);//Retrieved image will be placed into C
scamp7_in(E,127);
scamp7_kernel_begin();
      WHERE(S0);//Retrieve Bit 3 (128)
           add(C,C,E);
      ALL();

      divq(D,E);//D ~= 64
      WHERE(S1);//Retrieve Bit 2 (64)
          add(C,C,D);
      ALL();

      divq(E,D);//E ~= 32
      WHERE(S2);//Retrieve Bit 1 (32)
          add(C,C,E);
      ALL();

      divq(D,E);//D ~= 16
      WHERE(S3);//Retrieve Bit 0 (16)
          add(C,C,D);
      ALL();
scamp7_kernel_end();
```

The code above shows the far simpler process of retrieving an analogue image from this 4bit digital storage. This involves first loading the smallest value of -127 into an analogue register plane. Next we iterate through the DREG for each bit, identifying PEs in which the bit is set using "WHERE()", and then adding the bits associated value to these PEs.

4. RM & RF

Each PE has a FLAG register (DREG) that “masks” the execution of analogue instructions. When the processor array receives an analogue instruction, only the PEs where FLAG = 1 will execute the instruction. By setting the FLAG register, we can control the execution of analog instructions on a per-PE basis as described previously in a Previous Chapter.

Similarly, digital instructions can be conditionally executed using the DREGs RF (Flag) and RM (Masked). The RF register masks the execution of digital instructions targeting the RM register, just as the FLAG register controls the execution of analog instructions. That is, a digital instruction targeting RM will only be executed by PEs where RF = 1.

This allows for fine-grained control over which PEs execute specific instructions.

Example Code : RM & RF

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



This example demonstrating conditional execution of digital instructions is analogous to the Previous Example for analogue instructions. An image is captured and then thresholded to generate a 1-bit image in one DREG, while a rectangular region is loaded into another. We then first copy this rectangular region into RF, and then copy the thresholded image into RM. However, as digital instructions targeting RM are masked by RF, this produces a result such as shown in the right hand display above.

The code inside the main loop which performs this conditional execution is shown below.

```
//Load a rectangular region into S6, this will be copied to RF later
DREG_load_centered_rect(S6,box_x,box_y,box_width,box_height);

scamp7_kernel_begin();
      MOV(RF,S6);//Copy rectangular region in S6 into FLAG
          MOV(RM,S0);//Copy thresholded image of S0 into RM, only in PEs where RF = 1.
scamp7_kernel_end();
```

First a rectangular region is loaded into S6, rather than RF. This is because the function “scamp7_load_region”, uses RF as a temporary register (as do many other scamp7 functions).

```
//Load a rectangular region into S6, this will be copied to RF later
DREG_load_centered_rect(S6,box_x,box_y,box_width,box_height);
```

Now we perform a single kernel, whose first instruction copies this rectangular region into RF

```
MOV(RF,S6);//Copy rectangular region in S6 into FLAG
```

Followed by an instruction targeting RM, which will be executed only in PEs where RF = 1

```
MOV(RM,S0);//Copy thresholded image of S0 into RM, only in PEs where RF = 1.
```

DREG Decay & Refresh

Altering the rectangular region will change the PEs in which RF = 1, and hence the PEs executing the digital instruction “MOV(RM,S0)” to copy the thresholded image. PEs where this instruction is no longer being executed will still contain old copied data.

Despite being digital data, the content of a DREG can still “decay”. This occurs when a significant amount of time has passed, without any operations being performed involving the register. Such decay typically results in 1s flipping to 0s, and in much rarer instances, 0s flipping to 1s. This is shown below in the right hand image.



Fortunately we can prevent DREG decay by periodically performing operations upon the DREG in question. The digital instruction “REFRESH(DREG X)” was specifically created for this purpose.

The example contains the code below, inside the frame loop, to prevent the decay of RM.

```
if(refresh_RM)
{
      scamp7_kernel_begin();
          SET(RF);
               REFRESH(RM);
      scamp7_kernel_end();
}
```

The kernel here contains the instruction to refresh RM, after performing.

```
SET(RF);
```

This is required as we wish to refresh RM in all PEs, and so we must first set RF = 1 in all PEs. Again, all digital instructions targeting RM (including “REFRESH”) are only executed in PEs where RF = 1.

5.Analogue NEWS

This Chapter describes how to transfer Analogue data between PEs. In the SCAMP architecture each PE is connected with four immediate neighbouring PEs via orthogonal connections, typically referred to as “North”, “South”, ”East” & “West”.

Data in one PE can only be directly transferred to one of these four immediate neighbours. However data can be transferred between any two PEs in the array, simply by performing multiple direct transfers shuffling the data across neighbouring PE,

PE data transfer is a key element in visual processing, allowing each PE to access & process visual data captured in its surrounding area. Without data transfer each PE would be limited to simply analysing the single pixel of data it alone captured, a pretty useless architecture.

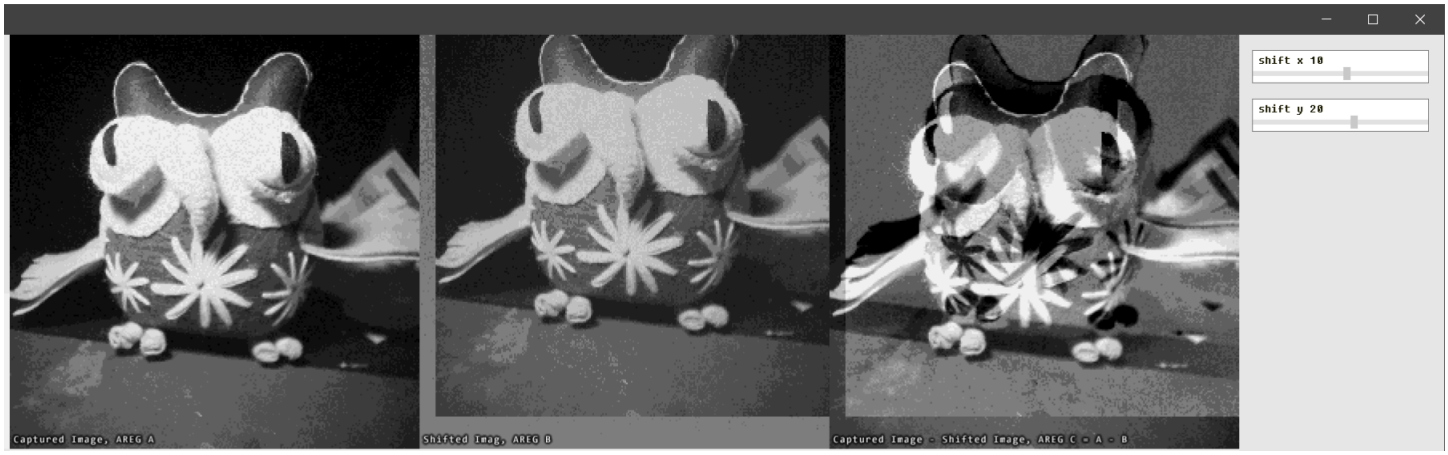
TODO DIAGRAM OF PE CONNECTIONS

It should be noted that while it is possible to transfer data between distant PEs in the array, the number of data transfer instructions required (and hence computational cost) is proportional to the Manhattan distance between the two PEs. To operate efficiently an algorithm should largely restrict data transfer between PEs to a local area.

Example Code : AREG NEWS

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**

This example demonstrates Analogue data transfer by allowing the user to shift a captured image both horizontally and vertically. The shifted image is then subtracted from the original image visualised as shown on the right. This data transfer is made possible by the NEWS AREG and the “bus(X,Y)” instruction.



“bus”

The instruction “bus(X,Y)” copies and inverts the content of an AREG Y, into another AREG X, resulting in X=-Y+offset. This is a native instruction, meaning it is executed in a single clock cycle.

Many instructions/Macros are constructed by various combinations of “bus” instructions. For example the instruction “mov(X,Y)” actually consists of the following two “bus” instructions, using the NEWS AREG as a temporary register. For this reason NEWS is not a valid argument to “mov(X,Y)”.

```
bus(NEWS,X);//NEWS = -x + offset
bus(Y,NEWS);//Y = -NEWS + offset == x - offset + offset == x
```

NEWS AREG

The NEWS AREG has a unique feature, its content can be accessed by the neighbours of a PE. This is why all transfers of analogue data between PEs must go through NEWS.

When using the “bus” instruction, the NEWS registers of neighbouring PEs can be accessed via the syntax “XN”, ”XS”, ”XE” & ”XW”. For example, consider the following code, taking FLAG = 1 in all PEs.

```
bus(NEWS,B);//NEWS = -B
bus(B,XW);//B = -NEWS OF WEST NEIGHBOR
```

The first instruction “bus(NEW,B)” will instruct all PEs to copy -B into NEWS. The second instruction bus(B,XW) will then instruct all PEs to copy -XW into B. This XW refers to the NEWS register of a PEs immediate “west”/”left” neighbour, and so these two bus instructions transfer the data of B between PEs. This process can also be viewed as “shifting” the analogue data stored in register plane B, one PE to the right (as data is copied from each PEs “left” neighbour).

Shifting Analogue Data

To transfer analogue data further than simply between neighbouring PEs, we can just execute the instructions to transfer between immediate neighbours multiple times. Consider this code taken from the example.

```
for(int x = 0; x < shift_x; x++)
{
    //KERNEL SHIFTS B ONE "PIXEL" RIGHT
    scamp7_kernel_begin();
       bus(NEWS,B);//NEWS = -B
       bus(B,XW);//B = -NEWS OF WEST NEIGHBOUR
    scamp7_kernel_end();
}
```

Here a kernel for shifting the data of B one PE to the right lies inside a loop. This kernel is executed “shift_x” times, and hence will shift the content of B “shift_x” PEs to the right. The code of the example contains four similar loops, each for shifting B in a particular direction, allowing the user to use GUI sliders to arbitrarily shift the content of B.

Analogue Degradation

TODO VXC SLIDER!!

When playing around with the example you may notice a “fading” in the shifted image when shifting by a large distance. This is due to the degradation issues faced by Analogue registers. The exact behaviour of these issues is complex to formulate and will vary between each SCAMP system. However, the basic premise is that a small amount of noise will be added into the result from every Analogue operation. When performing a large number of analogue operations upon the same data (such as repeatedly transferring a piece of data between PEs) this noise will accumulate and result in a loss of signal such as seen here. This is by no means a fatal issue however, it instead must simply be considered when designing algorithms for SCAMP architectures and will be covered again later in this tutorial.



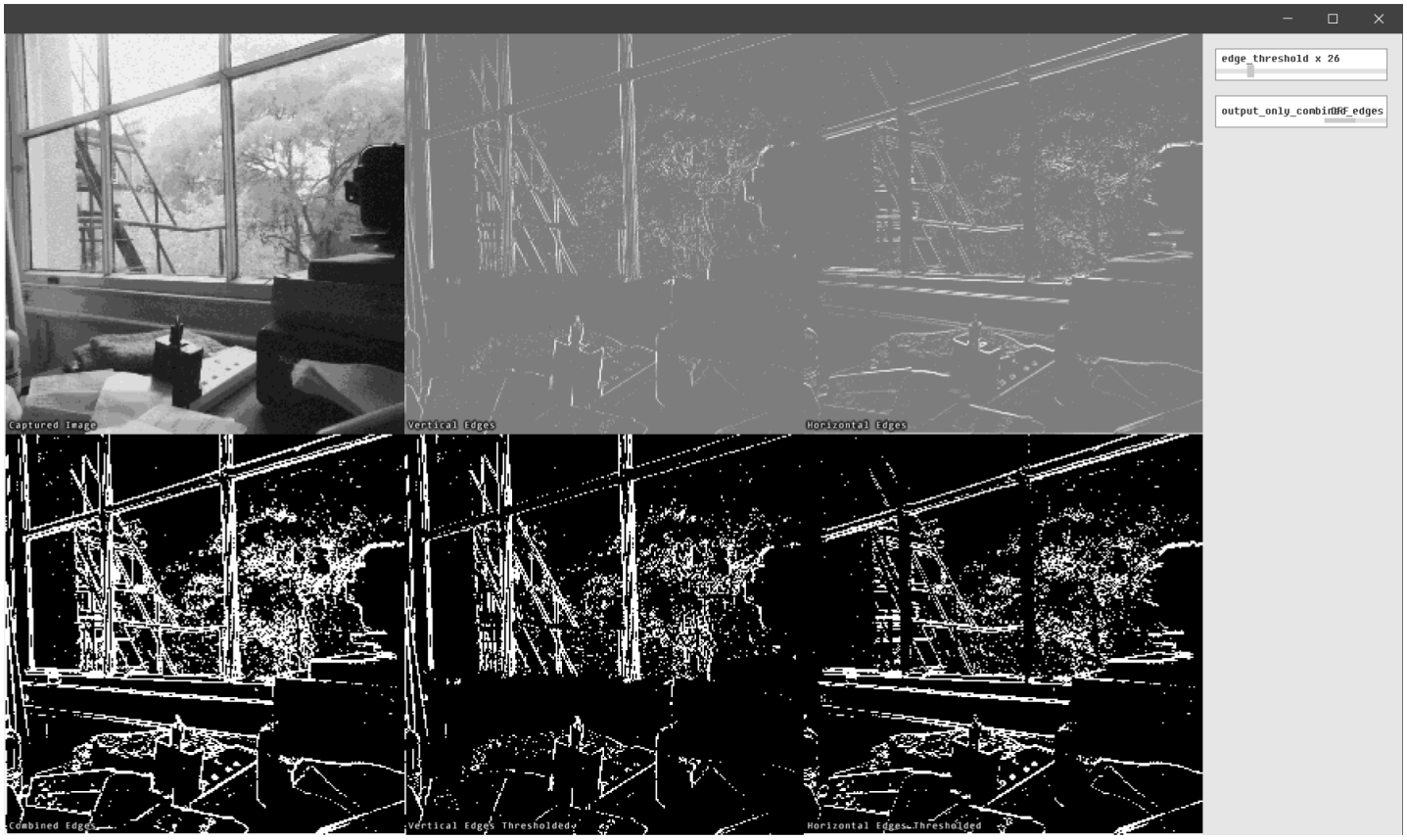
Example : Simple Edge Detection

This chapter brings together what has been covered so far to create a simple edge detection program as shown below. Using analogue instructions and data transfer via NEWS, this example generates vertical and horizontal edge detection data in two separate AREG. Thresholding is then used to convert both of these analogue edge detection maps into DREG equivalent. Finally the content of both of these DREG is combined together to form a single edge detection map as shown in the bottom left display.

By default this example outputs and displays many images. We can toggle off all such output except the final edge detection map. In doing so we see the FPS jump from ~90 to well over ~1000, once again illustrating how the costs of outputting image data are the dominating factor. Let's now go though this code.

Example Code : Simple Edge Detection

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



First note at top of the file we define a set of constant AREG and DREG variables. This effectively is used to rename certain registers to increase code readability, with each variable's name describing its use.

```
const areg_t AREG_image = A;
const areg_t AREG_vertical_edges = B;
const areg_t AREG_horizontal_edges = C;

const dreg_t DREG_combined_edges = S0;
const dreg_t DREG_vertical_edges = S1;
const dreg_t DREG_horizontal_edges = S2;
```

Such variables make code easier to understand, however we must be careful to keep track of what registers are being used in each part of the program. In longer programs a single register might be used in many different computations, as is often necessary due to the limited number of registers. There is less of a case for using named variables for such registers.

Analogue Edge Detection Maps

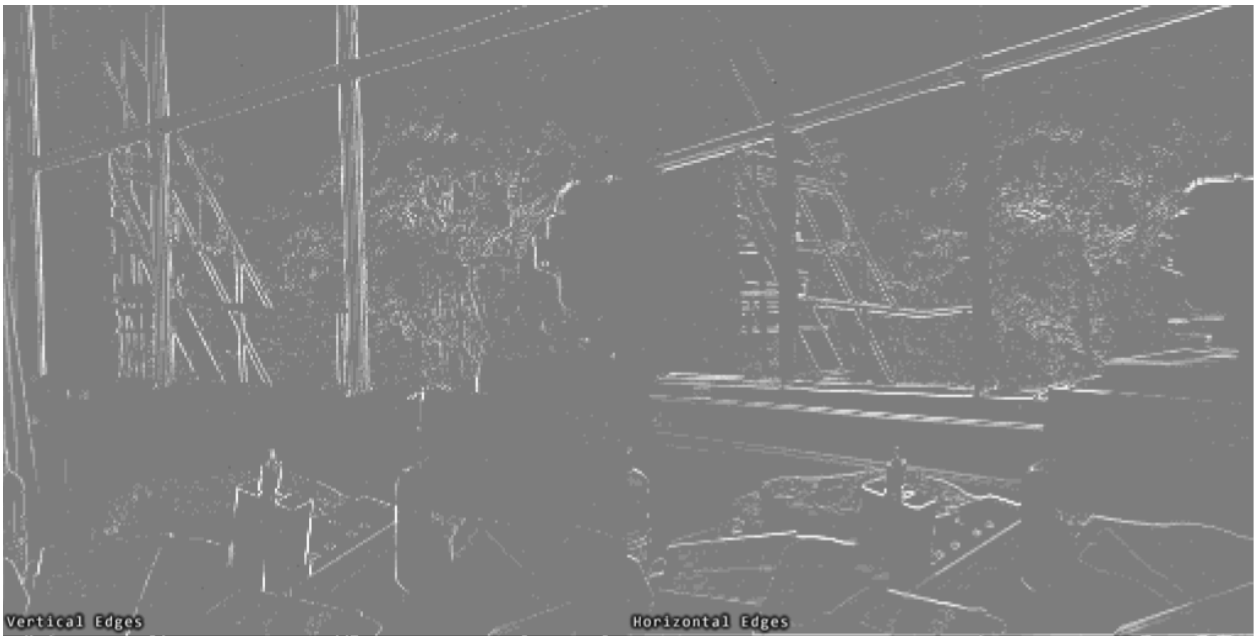
Later in the main frame loop, after capturing an image, both vertical and horizontal edge detection is performed.
This is done entirely using Analogue instructions, as shown in the following kernel for vertical edge detection.

```
//COMPUTE VERTICAL EDGE IMAGE
scamp7_kernel_begin();
    //COPY IMAGE TO F AND SHIFT ONE "PIXEL" RIGHT
    bus(NEWS,AREG_image);
    bus(F,XW);

    //SAVE THE ABSOLUTE DIFFERENCE BETWEEN THE SHIFTED IMAGE
    //AND ORIGINAL AS THE VERTICAL EDGE RESULT
    sub(F,F,AREG_image);
    abs(AREG_vertical_edges,F);
scamp7_kernel_end();
```

This kernel first uses two bus() instructions to create a shifted copy of the captured image in F. This copy was shifted one PE to the right by using NEW & XW as described in the previous chapter.

Differences between the pixels of this captured image and its horizontally shifted copy indicate the presence of vertical edges. We can find the PEs which have such pixel data by simply subtracting the captured image from this shifted copy, and then taking the absolute value of this result. This is performed using the “sub()”, & “abs()” instructions, generating a vertical edge detection map stored in AREG_vertical_edges (i.e. AREG B). A similar process is used to generate the horizontal edge detection map. Examples of these two are displayed below.



Digitizing & Combining Edge Maps

We now have both vertical and horizontal edge maps stored across two different AREG planes. In such a map, the value stored at any particular PE indicates the strength of detected edges at its location (higher values indicating stronger detection). These analogue edge detection maps are then thresholded to produce Digital versions, in which only PEs where sufficiently strong edges were detected contain 1s and all other PEs 0s.

To perform this first a threshold value is loaded into AREG F of all PEs.

```
scamp7_in(F,edge_threshold);//LOAD THRESHOLD INTO F
```

This threshold now stored in F is then subtracted from the AREG storing the vertical edge detection map, placing the result into E. The content of E will now only be positive in PEs where the edge detection map was above the threshold. Performing “where(E)” will set FLAG=1 in these positive PEs and 0 in all others. This has then thresholded the analogue edge map to a digitised edge map stored in FLAG. Finally we store this result by coping FLAG into another DREG, and then perform “all()” to reset the content of FLAG to 1 in all PEs.

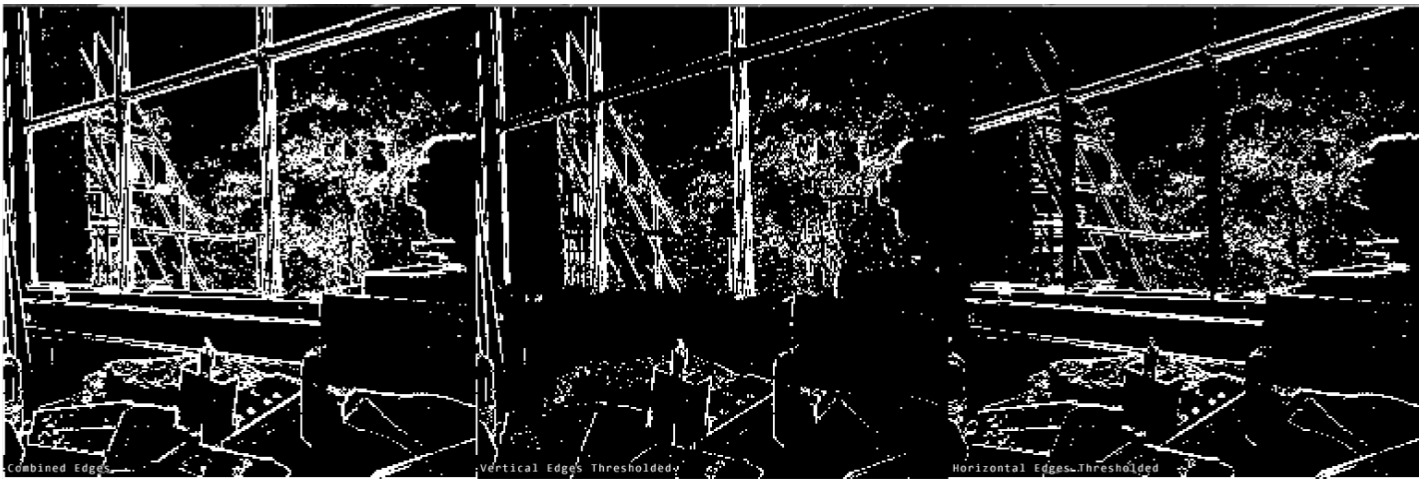
```
//THRESHOLD AREG_vertical_edges TO BINARY RESULT
scamp7_kernel_begin();
    sub(E,AREG_vertical_edges,F);
    where(E);
       //IN PES WHERE AREG_vertical_edges > edge_threshold, FLAG == 1
       MOV(DREG_vertical_edges,FLAG);
    all();
scamp7_kernel_end();
```

A similar process is used to threshold the analogue horizontal edge detection map to a digitised version.

Finally we combine both the digitized vertical edge map with the horizontal one using a few digital instructions.

```
//COMBINE THRESHOLDED HORIZONTAL & VERTICAL EDGE RESULTS
scamp7_kernel_begin();
    MOV(DREG_combined_edges,DREG_vertical_edges);//COPY VERTICAL EDGE RESULT
    OR(DREG_combined_edges,DREG_horizontal_edges);//OR WITH HORIZONTAL EDGE RESULT
scamp7_kernel_end();
```

This produces an edge map with both vertical and horizontal edges such as shown in the left display below.



6.Digital NEWS

This chapter describes the process of transferring digital data between PEs. Each PE is connected to its four immediate neighbours (“North”, "South", “East”, and “West”) for digital data transfer, similar to the connections used for analog data transfer discussed in previous chapters. However, there are key differences between the transfer mechanisms for digital and analog data:

- **Transfer Into Any DREG**: Digital data can be transferred from any DREG in one PE to any other DREG in a neighbouring PE. In contrast, analog data transfer can only occur through the NEWS AREG.

- **Control Of Transfer Direction**: The direction of digital data transfer can vary for each PE. This is different from analog data transfer, where the transfer direction must be uniform across all PEs. As we will see, digital data can even be transferred into a PE from multiple directions simultaneously.

DNEWS & RN, RS, RE, RW

The instruction “DNEWS(X,Y)”, where X≠Y, is used to transfer digital data between PEs. When executed each PE will copy data from DREG Y of neighbouring PEs, and move this data into its own DREG X.

But which of the four neighbours does a PE copy data from when DNEWS is executed? This is controlled by four DREG within the PE (RN, RS, RE & RW), one for each neighbouring PE. Data will only be copied from the “North” neighbour if RN=1, from the “South” neighbour if RS=1 and so on.

TODO DIAGRAM OF DREG TRANSFER CONNECTIONS INTO PE

If a PE copies data from multiple neighbours when DNEWS is executed, this data is automatically combined into a single value via an OR operation. For example if a PE has RN=1, RS=0, RE=1 & RW=0, then when executing DNEWS(X,Y) this PE will end with X = Y(North neighbour) OR Y(East neighbour).

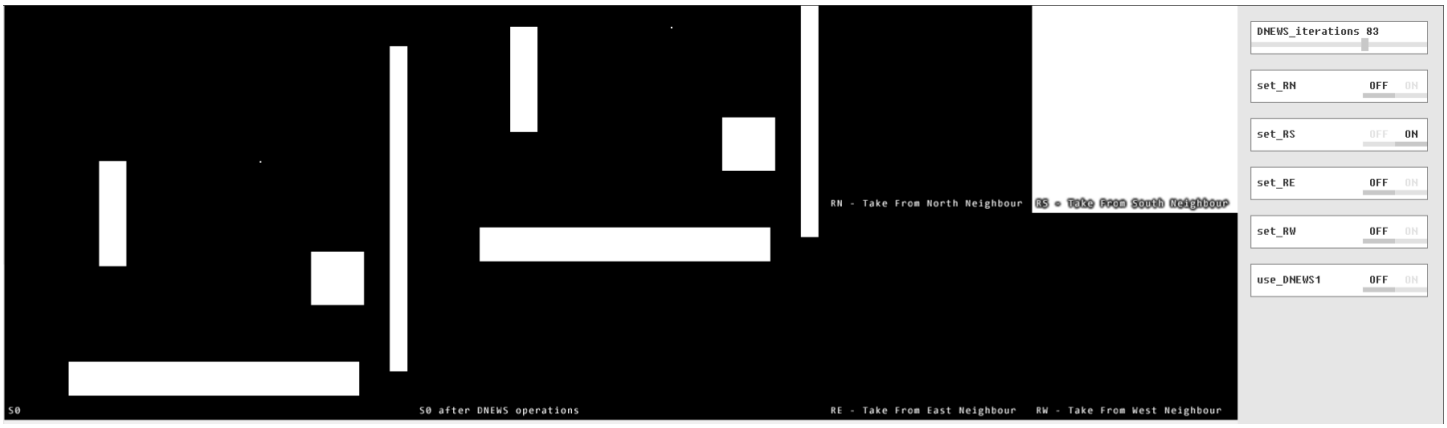
This scheme for controlling the direction of data transfer at the PE level enables many clever algorithms and optimizations to be implemented in various algorithms. However this comes at the cost of four DREG (RN, RS, RE & RW) having to be often reversed for their use in DNEWS.

Let us now see an example of DNEWS in action.

Example Code : DNEWS

EXAMPLE CODE SCAMP5 EXAMPLE CODE SCAMP 7

This example demonstrates the operation of DNEWS. A number of rectangles and points are loaded into the DREG S0, and then DNEWS is repeatedly performed upon this content. The user can select the number of times DNEWS is performed, and set the values of the RN, RS, RE & RW DREGs (0 or 1 across all PEs). This determines distance and directions in which the data is transferred.



Looking into the code, four GUI switches with linked variables are set up, one for each of RN, RS, RE & RW.

```
int set_RN = 0;
vs_gui_add_switch("set_RN",set_RN == 1,&set_RN);
```

DNEWS operations are then performed inside the frame loop.
First we clear the RN, RS, RE & RW DREG, setting each to 0 across all PEs.

```
//Clear all DREG controlling DNEWS behaviour
scamp7_kernel_begin();
    CLR(RN,RS,RE,RW);//sets RN,RS,RW and RW to 0 in all PEs
scamp7_kernel_end();
```

Then separately, each of the RN, RS, RE & RW DREG are set if their associated variable is set to 1, as controlled by the GUI switches.

```
if(set_RN)
{
    scamp7_kernel_begin();
       SET(RN);//sets DREG RN to 1 in all PES
    scamp7_kernel_end();
}
```

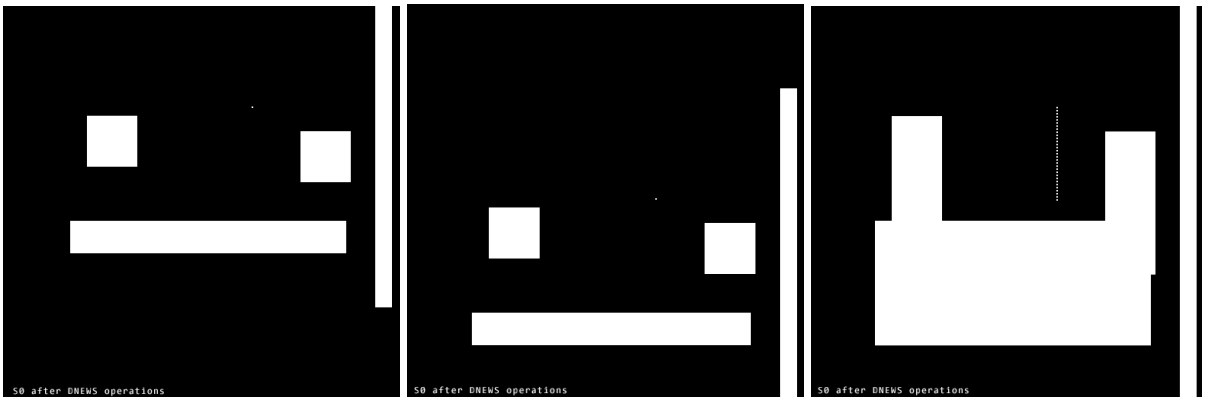
Finally, the DNEWS instruction is repeatedly executed upon the data of S0 transferring it the selected number of times. This actually requires two instructions as “DNEWS(S0,S0)” is not valid. We thus must perform “DNEWS0(S6,S0)”, and then “MOV(S0,S6)” to copy the transfer result in S6 back into S0.

```
//Repeatedly perform DNEWS operations on S0 for the selected number of iterations
for(int n = 0 ; n < DNEWS_iterations ; n++)
{
    scamp7_kernel_begin();
       DNEWS0(S6,S0);//S6 = DNEWS(S0)
       MOV(S0,S6);//Copy the results in S6 back into S0
    scamp7_kernel_end();
}
```

Multi-Direction Digital Transfer

The example allows any combination of the RN, RS, RE & RW DREG planes to be set(all 1s) or cleared(all 0s) when DNEWS is executed upon S0.

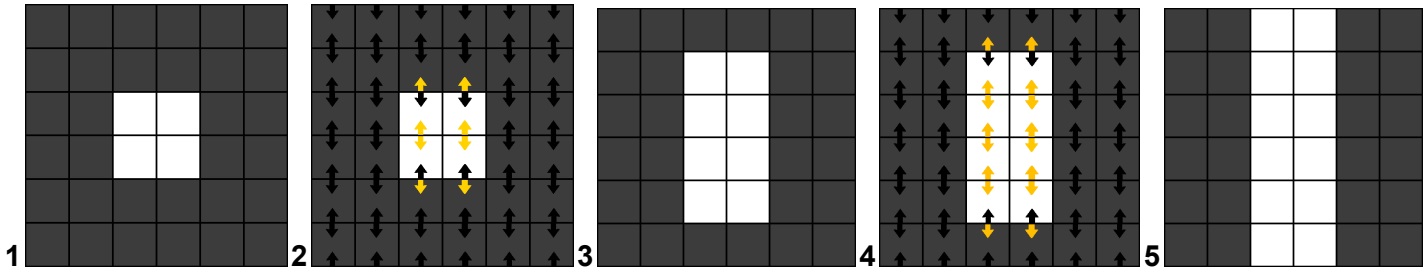
When only one of these registers is set the result is easy to understand. For example if only RS is set, then each PE will copy data from S0 of its south neighbour, effectively shifting the content of S0 upwards. Similarly, if only RN is set then S0 is shifted downwards. Such cases are shown below in the left and middle images.



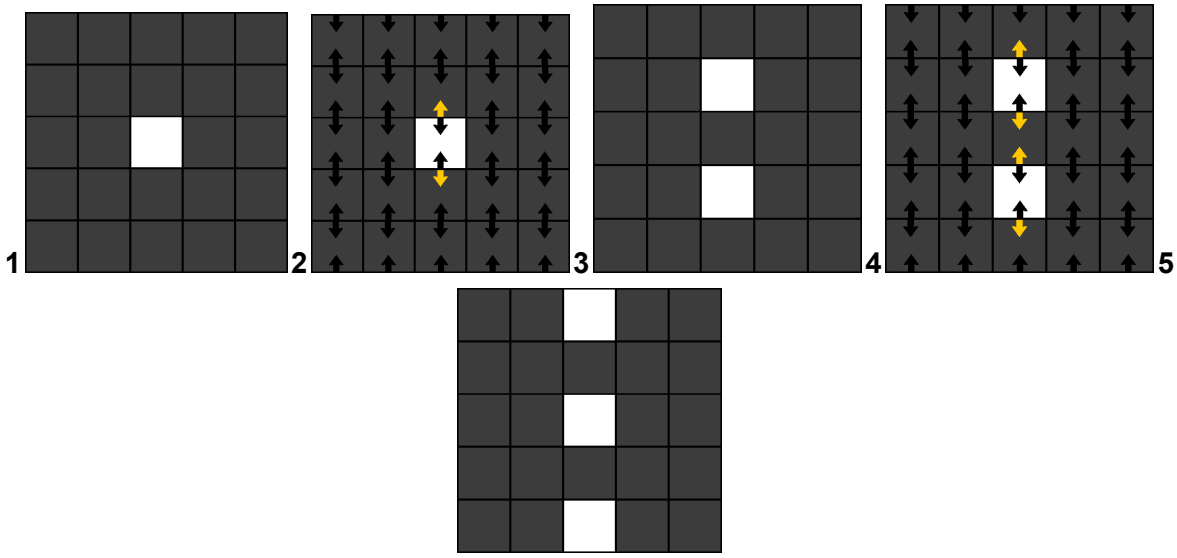
But what if RN & RS are set when DNEWS is executed? This produces a result such as the right hand image above. Patches of 1s are expanded both vertically and horizontally, but isolated 1s instead result in dotted vertical lines. Let us examine how both these behaviours occur.

First note that with RN & RS set to 1 in all PEs, each PE will copy data from both its “North” and “South” neighbours when DNEWS is executed. These copied values are combined into a single value within each PE by an OR operation. Thus if any copied value is a 1, the final value will be a 1.

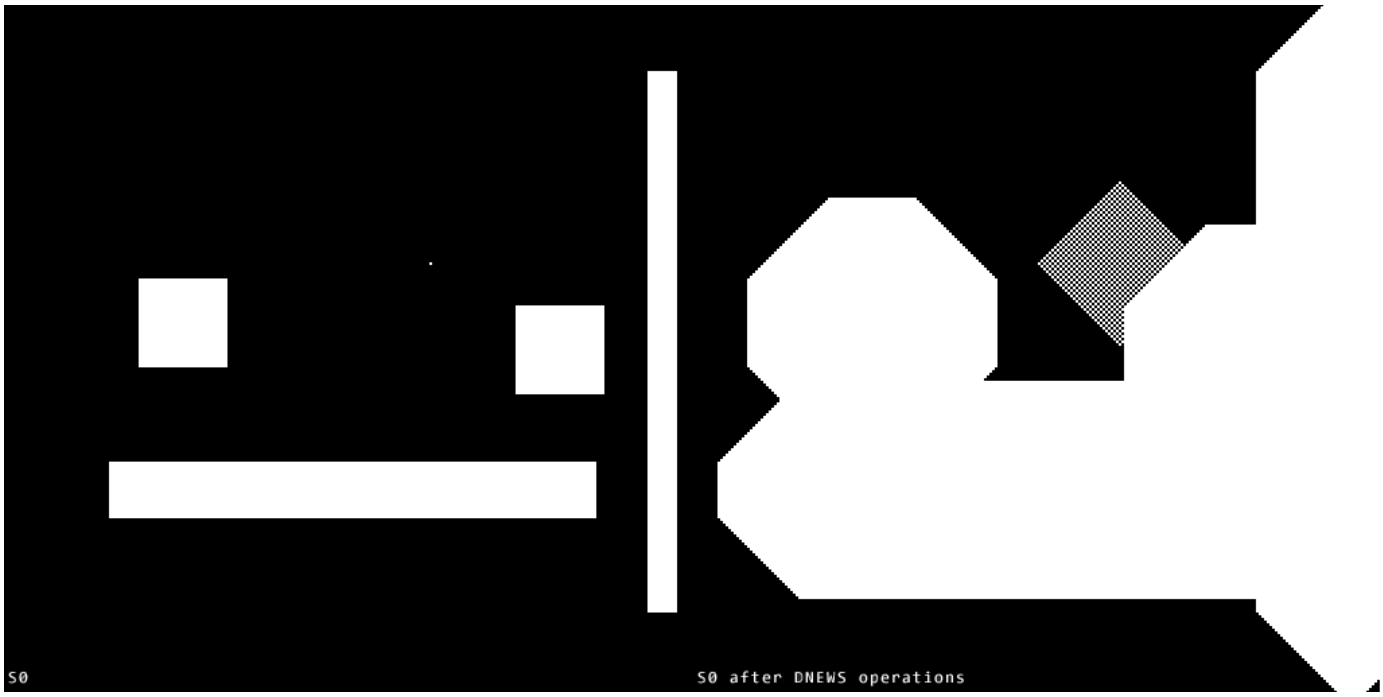
Consider a small region of the PE array, with a 2x2 patch of 1s stored across a DREG plane shown in **1** below. Data copied into each PE when DNEWS is executed is indicated by the arrows in **2**, black for 0s, yellow for 1s. Executing DNEWS then produces **3**, with the steps for executing DNEWS a second time shown in **4** & **5**.



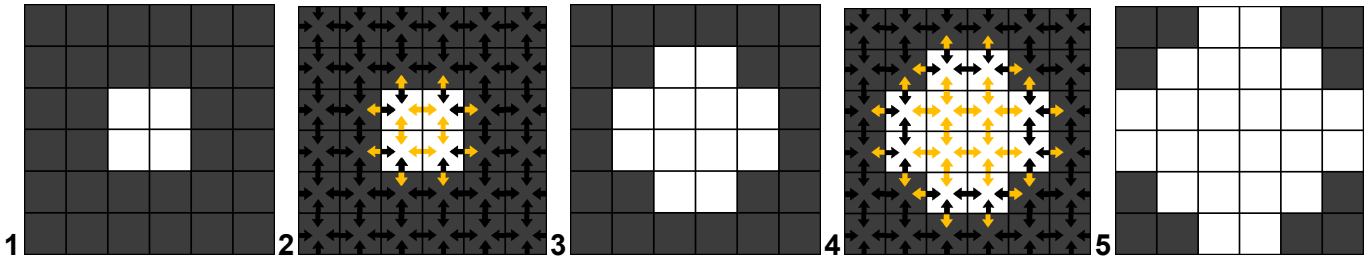
Now consider a single isolated 1 stored in a DREG plane. The images **1**,**2**,**3**,**4**,**5** illustrate the steps of executing two iterations of DNEWS upon the content of this DREG. Following the arrows indicating which data copied at each DNEWS execution, we can follow how a dotted line is produced which is not intuitive until examined.



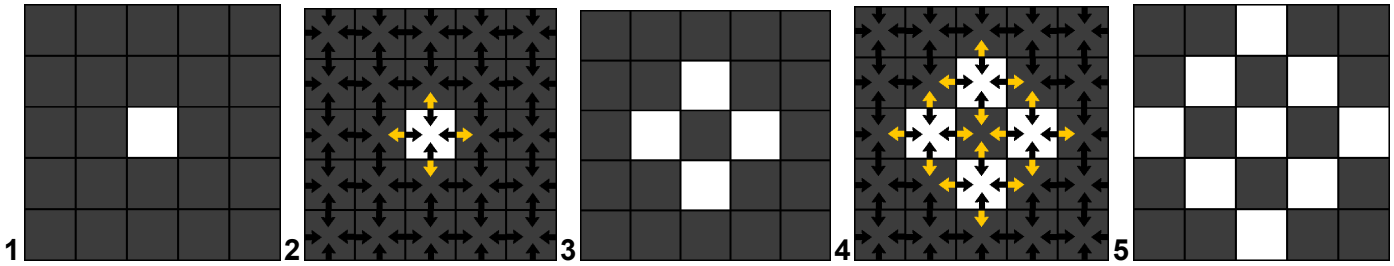
Let us examine one last case where RN, RS, RE & RW are all set, so each PE will copy data from all four neighbours when DNEWS is executed. Repeatedly executing DNEWS upon the same DREG plane then produces a result such as shown below.



Once again there are two main behaviours that occur, one for patches of 1s and one for isolated 1s. Images **1**,**2**,**3**,**4**,**5** below illustrate how when RN, RS, RE & RW are all set, DNEWS will now expand areas of 1s, with corners expanding to form diagonal “edges”.



Similarly **1**,**2**,**3**,**4**,**5** below illustrate how isolated 1s will expand in a similar way, but containing a checkerboard of 1s and 0s.

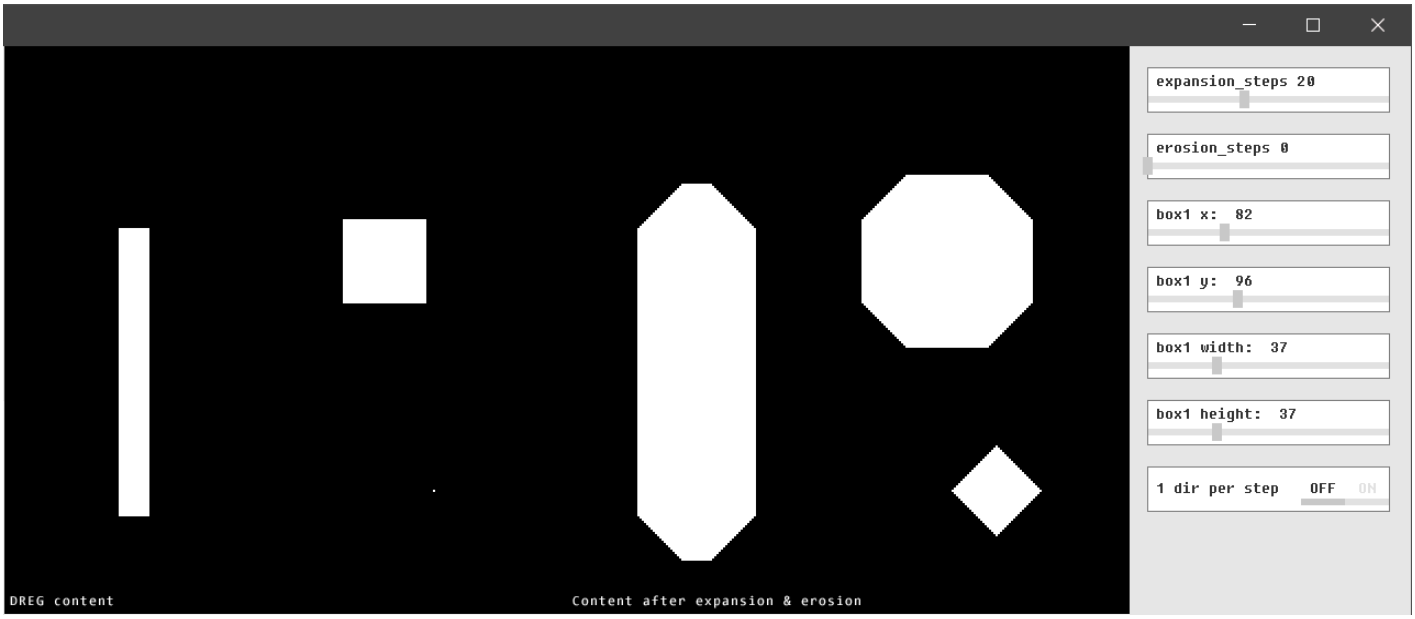


Example Code : Expansion/Erosion with DNEWS

Many algorithms involve expanding or eroding the content of a DREG plane. Expanding involves spreading any 1s into neighbouring PEs, and eroding spreading any 0s into neighbouring PEs.

DNEWS can be used to perform these tasks as demonstrated in this example. The user can control the direction and number of expansion/erosion steps. The original DREG content is shown in the left display, and the content after expansion/erosion on the right. Now let's explain how expansion and erosion are performed.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



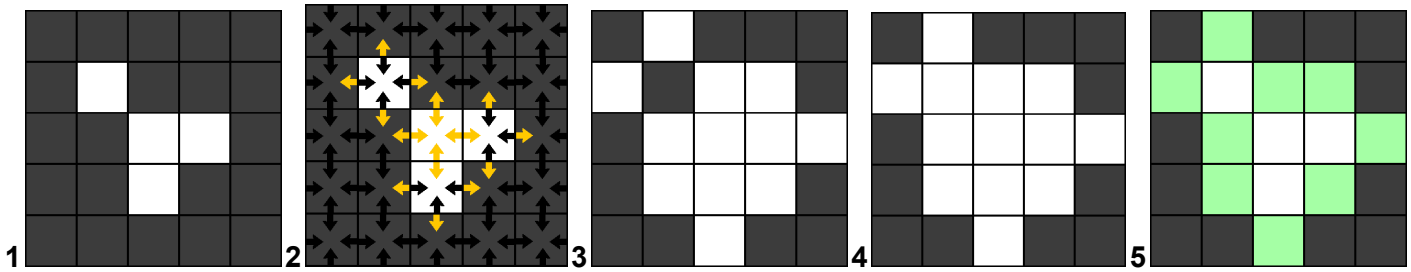
Expansion

The simplest method to expand the content of a DREG plane X involves expanding in all directions at once. This requires we set RN, RS, RE & RW to 1 in all PEs, and then execute DNEWS upon X.

This is illustrated in images **1**,**2**,**3** below. Showing how 1s are spread into neighbouring PEs from executing DNEWS. However, any isolated 1s end up with 0s, as seen in images **1**&**2**.

Isolated 1s within X still need to contain 1s after expansion. To address this an OR operation is performed between the result of DNEWS and the original content of X. This is shown in image **4**, performing an OR between **1**&**3** to fill the gap left by the isolated 1.

Image **5** highlights what PEs now contain 1s compared to the original content of X, and it should be clear that the content of X has been expanded by one PE (or one "pixel").

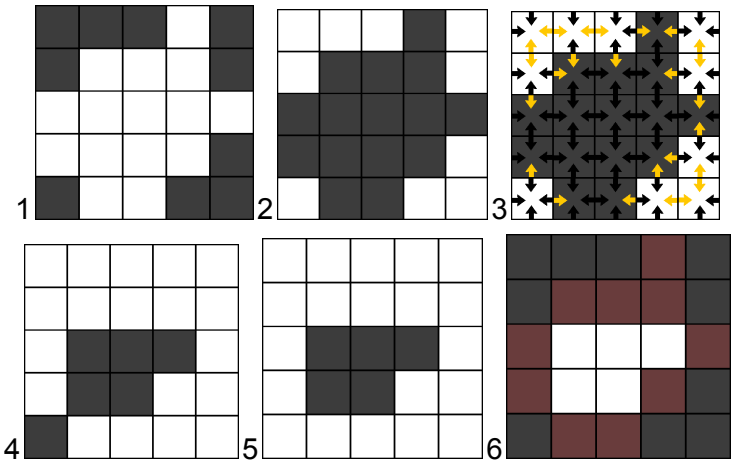


The code taken from the example below expands the content of S0 using the process just described. First RN, RS, RE & RW are all set, and then a kernel is repeatedly executed for the desired number of expansion steps. This kernel simply executes DNEWS(S6,S0), followed by OR(S0,S6), to first spread the 1s of S6 into neighbouring PEs and then combine this with the original content.

```
scamp7_kernel_begin();
      SET(RN,RS,RE,RW);
scamp7_kernel_end();
for(int n = 0 ; n < expansion_steps; n++)
{
      scamp7_kernel_begin();
            DNEWS0(S6,S0);//S6 will contain 1s at locations that S0 will expand into
            OR(S0,S6);//Combine expanded locations with S0 itself
      scamp7_kernel_end();
}
```

Erosion

Performing erosion upon the content of a DREG plane first involves inverting the content, with 0s becoming 1s and vice versa as shown in image **1 & 2** below. Next we expand this inverted content, following the steps of the previous section as shown in images **3**,**4**,**5**. This expansion can be viewed as expanding the 0s (and thus eroding the 1s) of the original content. Finally this result is inverted again as shown in **6**, producing an eroded version of the original content.



The code below performs erosion upon S0. First the instruction NOT(S1,S0) is used to produce an inverted copy of S0's content in S1. Next this inverted content is expanded by DNEWS(S6,S1) and OR(S1,S6). Finally, the instruction NOT(S0,S1) is used to invert the expanded content and copy it back into S0.

```
scamp7_kernel_begin();
    NOT(S1,S0);//invert DREG content
    DNEWS0(S6,S1); //expand inverted content
    OR(S1,S6);
    NOT(S0,S1);//invert back again
scamp7_kernel_end();
```

However, if we placed this code inside a loop to perform multiple erosion steps, an unnecessary number of NOT instructions will be executed in-between each step.

Instead we can achieve the same result performing only two NOT instructions. The first to invert the original content, and the second to invert again after performing multiple expansion steps as shown in the code from the example below. When performing a large number of erosion steps this approach gives better performance as significantly less instructions are executed (~½) .

```
scamp7_kernel_begin();
      NOT(S1,S0);//invert DREG content
scamp7_kernel_end();
for(int n = 0 ; n < erosion_steps ; n++) //expand inverted content
{
      scamp7_kernel_begin();
           DNEWS0(S6,S1);
           OR(S1,S6);
      scamp7_kernel_end();
}
scamp7_kernel_begin();
      NOT(S0,S1);//invert back again
scamp7_kernel_end();
```

Example Code : Shifting Digital Data 1

EXAMPLE CODE SCAMP5 EXAMPLE CODE SCAMP 7

This example demonstrates how to shift the data of a DREG plane using the DNEWS instructions. First a thresholded image is generated into a DREG plane following the process from the Thresholding Example. This binary image is then shifted based upon the slider values “shift_x” & “shift_y” as set by the user.



To correctly shift the image horizontal and vertical shifting must be performed separately. To shift horizontally we must first set up the registers which control the directions of data transfer for DNEWS, RN,RS,RE and RW. This involves clearing the content of these registers to 0 in all PEs with “CLR()”, and then depending on the direction to shift, setting either RW or RE to 1 in all PEs with “SET()”.

```
scamp7_kernel_begin();
      CLR(RN,RS,RE,RW);//Clear all DREG controlling DNEWS behaviour
scamp7_kernel_end();

//set the appropriate DREG to shift horizontally left or right
if(shift_x > 0)
{
      scamp7_kernel_begin();
          SET(RW);
      scamp7_kernel_end();
}
else
{
      scamp7_kernel_begin();
          SET(RE);
      scamp7_kernel_end();
}
```

Once this is done, we execute ”DNEWS()” repeatedly upon the content of the DREG plane containing the thresholded image, shifting it the desired number of times in the selected direction.

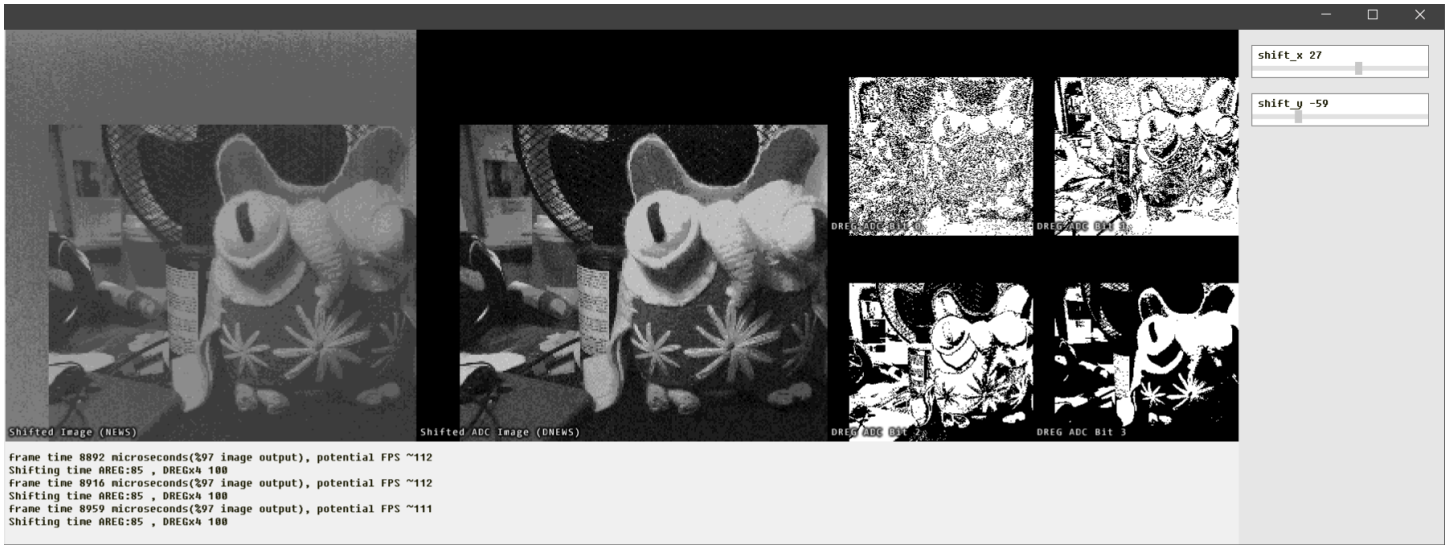
```
int iterations = abs(shift_x);
for(int n = 0 ; n < iterations ; n++)
{
      scamp7_kernel_begin();
          DNEWS0(S6,S1);//S6 = DNEWS(S0)
          MOV(S1,S6);//Copy the results in S6 back into S0
      scamp7_kernel_end();
}
```

This horizontally shifted data can then be vertically shifted, following the exact same steps but now setting the RN,RS,RE,RW registers to shift in the desired vertical direction.

Example Code : Shifting Digital Data 2

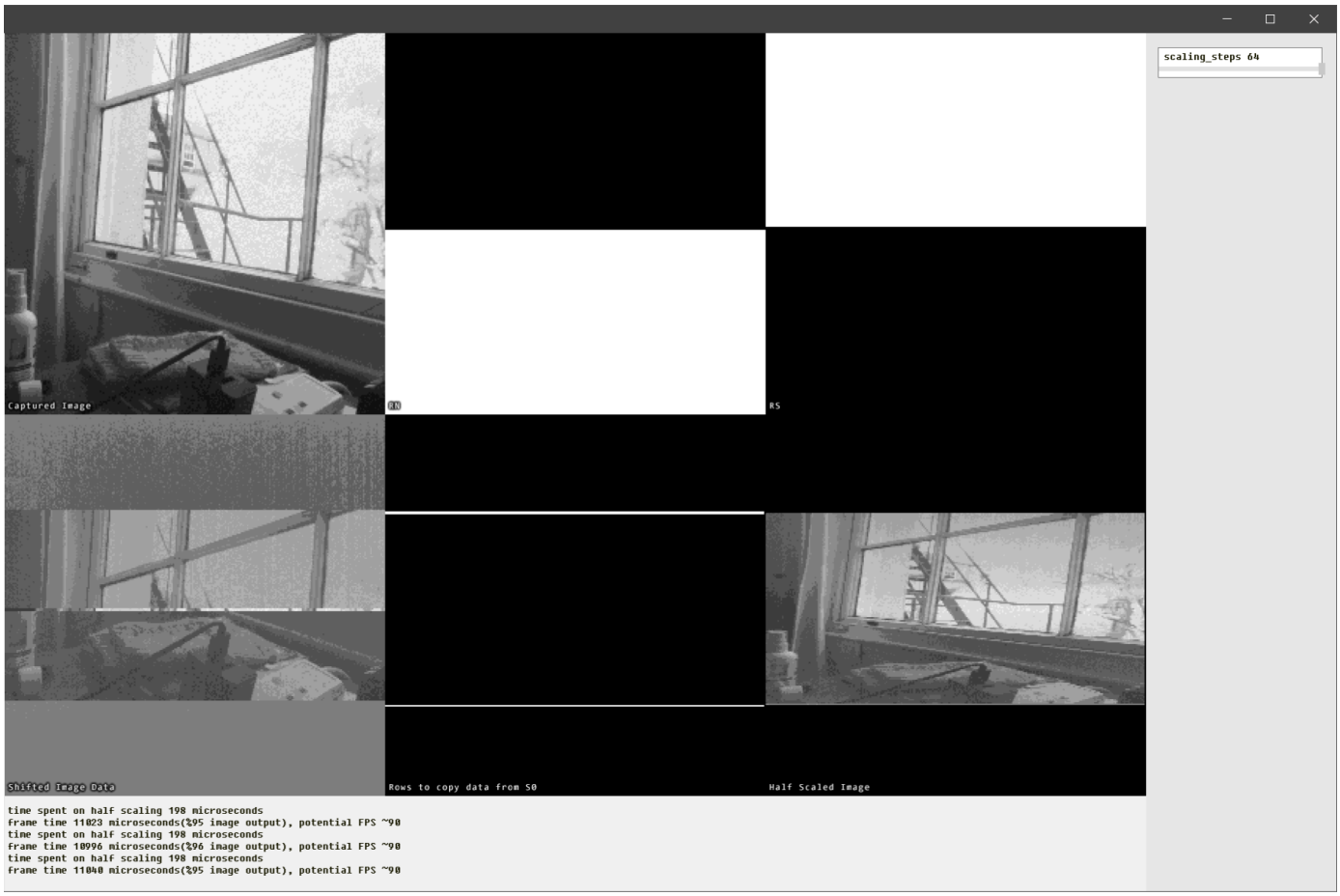
EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**

This example performs ADC as shown in a previous example, generating 4 binary images in separate DREG planes, each representing 1 Bit of a 4 Bit image. The user can then shift these DREG planes, effectively shifting the stored 4 Bit image. This image does not degrade from shifting (middle), compared to its analogue counterpart (left), demonstrating how ADC can be used in certain scenarios to preserve data when it is necessary to move data significant distances across the processor array.



Example : Simple Image Scaling

[EXAMPLE CODE SCAMP5](#) [EXAMPLE CODE SCAMP 7](#)



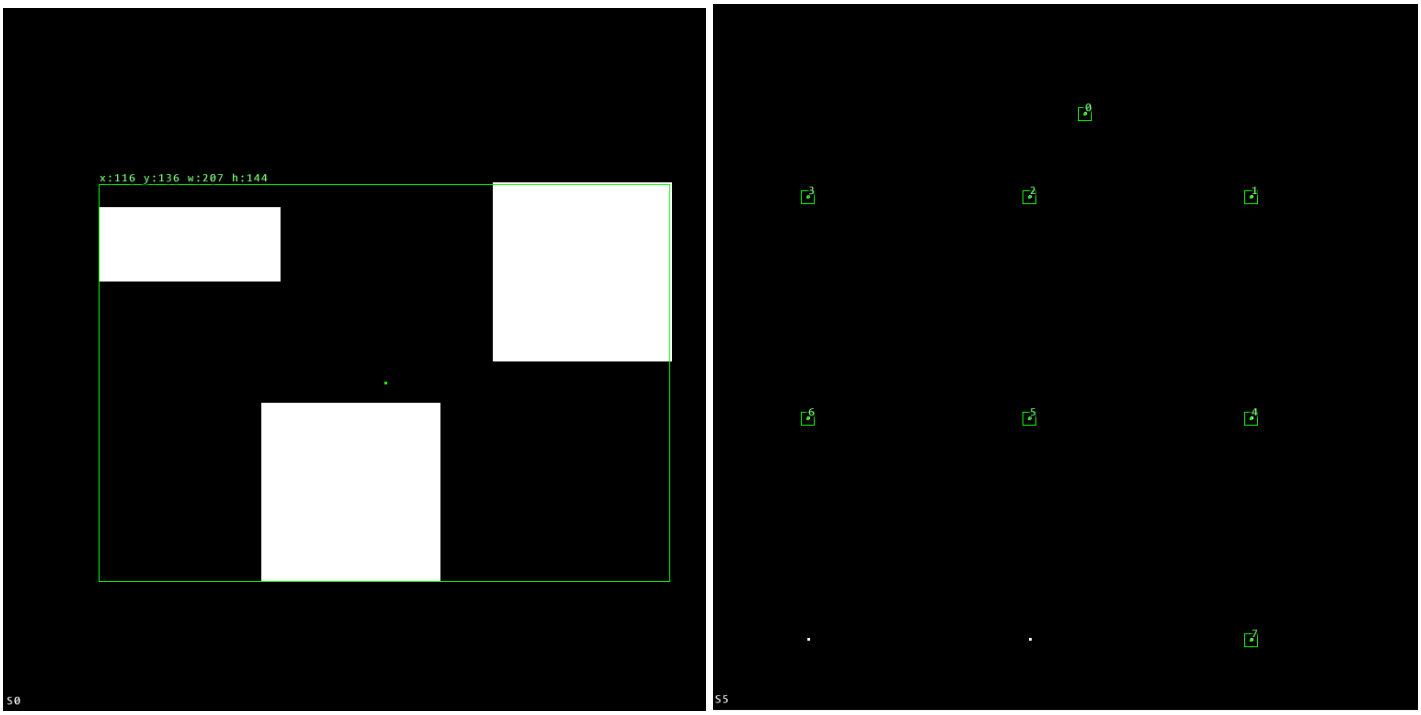
Extra : Image Transformation

General scaling and rotation of stored images is possible upon SCAMP, however how these are achieved is quite in depth. As such we will not explain them here in this tutorial, as this is focussed on covering the functionality of SCAMP rather than a specific method program. However we include the links below to code with the functions for each of these transformations, and the relevant paper giving more explanation.



[VIDEO](#) [CODE SCAMP5](#) [CODE SCAMP7](#) [PAPER](#)

7.Sparse Data Readout



We have previously described how in traditional camera sensors, where the whole image is output from the sensor each frame, data transfer is a major bottleneck consuming significant time and power. PPAs such as SCAMP can avoid this by performing computation on-sensor, extracting some form of desired information, and then only outputting that information rather than entire captured images.

For this purpose SCAMP provides several methods of sparse readout from digital register planes, the most common of which are in the table listed below, and bounding box and event readout shown above.

Digital Readout	**Description**	**Bytes**	**Function**
Global OR	Determine if there are any 1s/white pixels	1 (value of zero indicates no white pixels)	scamp_global_or()
Bounding Box	Get the bounding box which encloses all 1s/white pixels	4 (Coordinates of bounding box top left & bottom right corners)	scamp_scan_boundingbox()
Events	Get coordinates of 1s/white pixels, referred to as “Events”. Can select the number of events to readout.	2 Per Event readout (x,y coordinate pairs)	scamp_scan_events()

SCAMP programs often perform computation upon captured visual data, computing the presence & locations of features/objects etc. The results of this computation can be stored within the DREG of the PEs, and then efficient sparse digital readout methods used to transfer them to the microcontroller (and onwards to PC).

Global OR

A Global OR examines all PEs, checking if there is any PE where a certain DREG is storing a value of 1. This operation can be viewed as examining the binary image of a digital register plane, checking if it contains any white pixels (i.e. 1s).

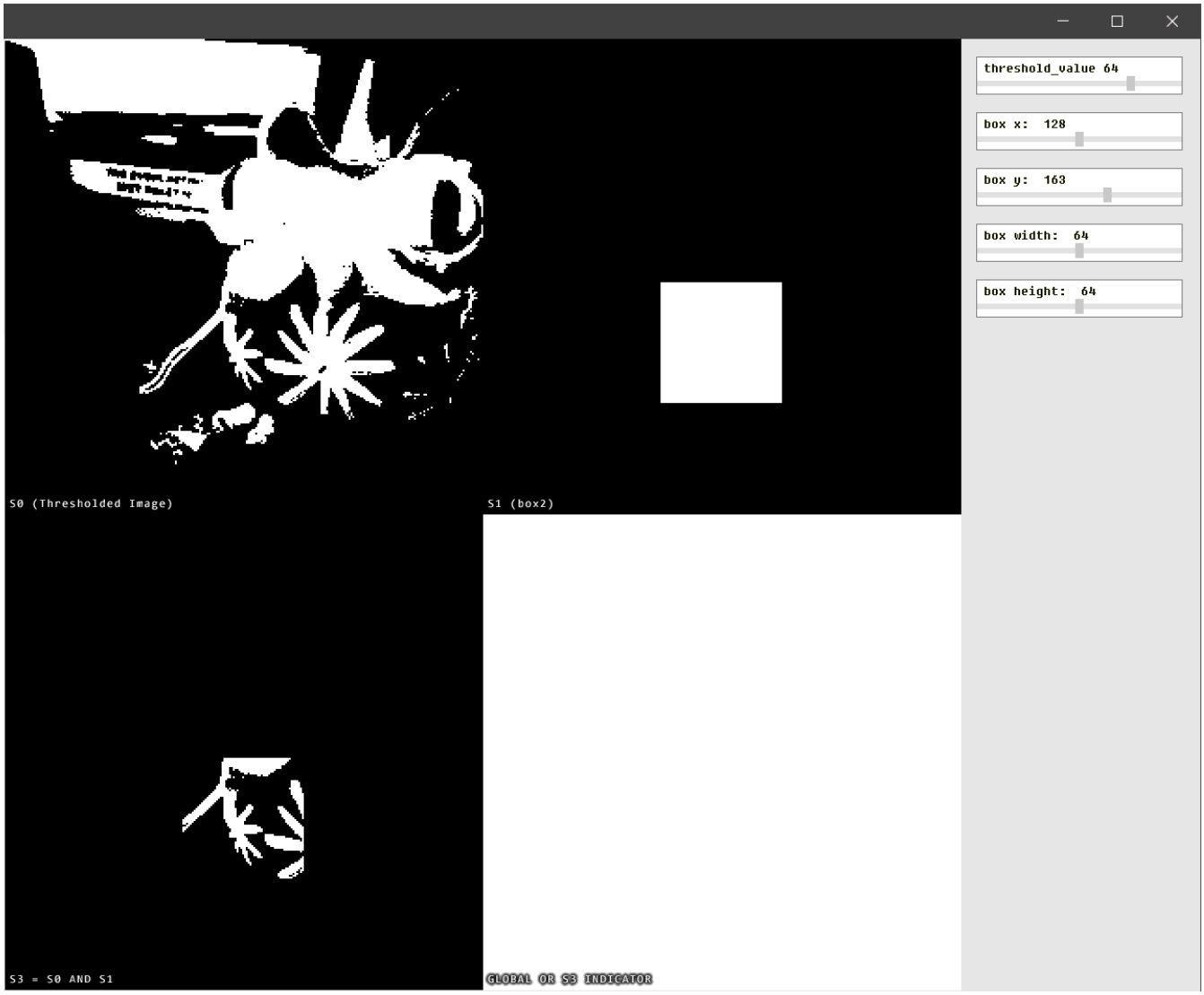
“scamp_global_or()”

The function “scamp7_global_or(DREG X)” is used to perform this operation, returning a single byte value (0-255), which is non-zero if there are any 1s present and zero otherwise.

Example Code : Global OR

The example here demonstrates using “scamp7_global_or” to check if a thresholded image contains any white pixels inside of a rectangular region. Generation of the thresholded image follows the same steps as a previous example. The bottom right display indicates the Global OR result.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



Inside the frame loop, the following code performs the check for white pixels inside the rectangular region.

```
scamp7_kernel_begin();
      AND(S3,S0,S1); //S3 = the thresholded image contained inside the loaded rectangle
scamp7_kernel_end();
int global_or_value = scamp7_global_or(S3);//check if there are any PEs in which S3 = 1
bool white_pixels_inside_rect = global_or_value > 0 ? true : false;
```

First “AND(S3,S0,S1)” is performed, essentially copying the thresholded image into S3, but eliminating any white pixels outside the rectangular region. Then “scamp7_global_or(S3)” is performed, with the value returned by this function then used to set the boolean variable “white_pixels_inside_rect”.

This variable is then used to either SET or CLR the S2 DREG, which is then displayed on the host as a visual indication of the Global OR result, alongside the text output.

```
if(white_pixels_inside_rect )
{
   vs_post_text("global_or %d TRUE\n",global_or_value);
   scamp7_kernel_begin();
      SET(S2);
   scamp7_kernel_end();
}
else
{
    vs_post_text("global_or %d FALSE\n",global_or_value);
    scamp7_kernel_begin();
        CLR(S2);
    scamp7_kernel_end();
}
```

Bounding Box

SCAMP provides the functionality to retrieve the bounding box of a digital register plane's content, specifically bounding all 1s. When viewing a digital register plane's content as a binary image, this is the bounding box of all white pixels of the image.

“scamp_scan_boundingbox()”

The function “scamp_scan_boundingbox()” retrieves the bounding box of a digital register plane’s content.

```
uint8_t bb_data_array [4];
scamp7_scan_boundingbox(S0,bb_data_array);
```

This function takes three parameters.

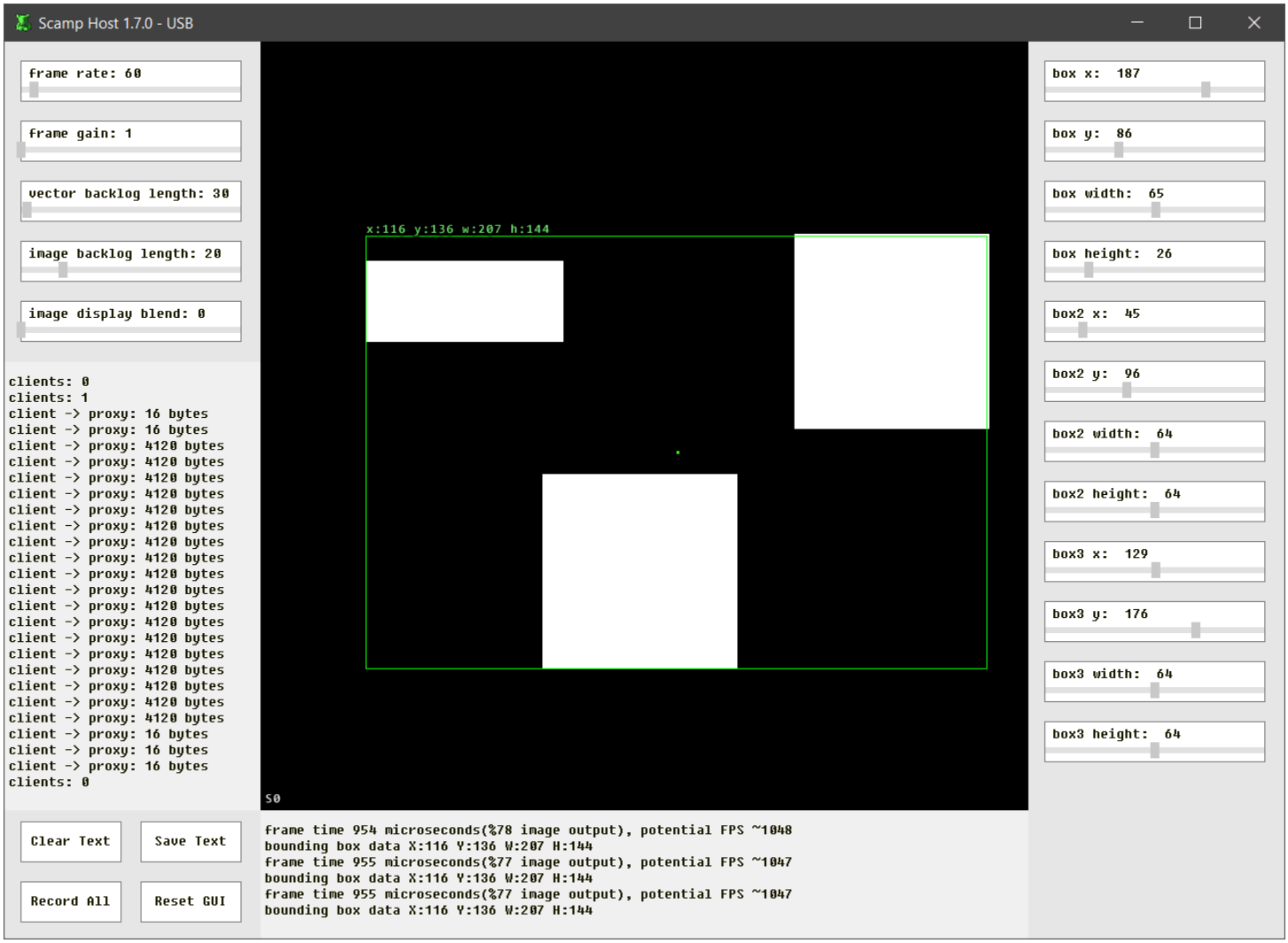
1. Which digital register plane’s content to get the bounding box of (“**S0**”)
2. An array of data into which to write the coordinates of the retrieved bounding box (“**bb_data_array**”)

The coordinates written to the array are in the order of (Top y, Left x, Bottom y, Right x).

Example Code : Scan Bounding Box

This example allows the user to position three different boxes in a digital register plane, and then uses “scamp_scan_boundingbox()” to get the bounding box of all three. The bounding box itself is then drawn to the display along with its size and location.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



This example generates content in the S0 digital register plane, combining three box shaped regions.

```
//Load boxes into DREG planes
DREG_load_centered_rect(S4,box_x,box_y,box_width,box_height);
DREG_load_centered_rect(S5,box2_x,box2_y,box2_width,box2_height);
DREG_load_centered_rect(S6,box3_x,box3_y,box3_width,box3_height);

//Combine boxes into single DREG plane
scamp7_kernel_begin();
      MOV(S0,S4);
      OR(S0,S5);
      OR(S0,S6);
scamp7_kernel_end();
```

“scamp _scan_boundingbox” is then used to retrieve the bounding box of S0’s content, placing the boxes’ top-right and bottom-left coordinates into the passed array “bb_data”.

```
uint8_t bb_data [4];
scamp7_scan_boundingbox(S0,bb_data);
```

This bounding box is then drawn to the display, using “scamp_display_boundingbox”, passing it the array of bounding box coordinates just retrieved previously by “scamp _scan_boundingbox”.

```
scamp7_display_boundingbox(display_00,bb_data);
```

The centre position, and dimensions of the bounding box are then calculated from the coordinate in the array “bb_data” as shown below.

```
int bb_top = bb_data[0];
int bb_bottom = bb_data[2];
int bb_right = bb_data[1];
int bb_left = bb_data[3];

//Calculate the center and dimensions of the bounding box
int bb_width = bb_left-bb_right;
int bb_height = bb_bottom-bb_top;
int bb_center_x = (bb_right+bb_left)/2;
int bb_center_y = (bb_top+bb_bottom)/2;
```

Finally the centre position and dimensions of the bounding box are drawn to the display next to the bounding box itself.

```
const std::array<uint8_t, 4> text_color = {127, 255, 127, 255};

std::string tmp_str = "X:" + std::to_string(bb_center_x) + " Y:" +
std::to_string(bb_center_y) + " Width:" +  std::to_string(bb_width) + " Height:" +
std::to_string(bb_height);

vs_gui_display_text(display_00, bb_left, bb_top, tmp_str.c_str(), text_color);
```

Events

Event readout is used to find the positions of any 1's stored in a digital register plane. Specifically, Event readout will scan through all PEs, returning the locations of those whose DREG (for that digital plane) contains a value of 1. This can also be viewed as scanning the binary image of a digital register plane, returning the locations of any white pixels present. These 1s/white pixels are referred to as "Events".

Many SCAMP algorithms detect features/objects within the observed image. It is common to indicate the presence of a detected feature/object at a PE's location by setting a certain DREG to 1 (and to 0 where there was no detection). Event readout can then be used to retrieve these locations into the microcontroller.

"Scamp_scan_events"

The function "scamp7_scan_events" shown below is used to scan events from a digital register plane.

```
const int event_buffer_size = 100;
uint8_t event_data_array [event_buffer_size*2];
scamp7_scan_events(S0, event_data_array, max_events_to_scan);
```

This function takes three parameters.

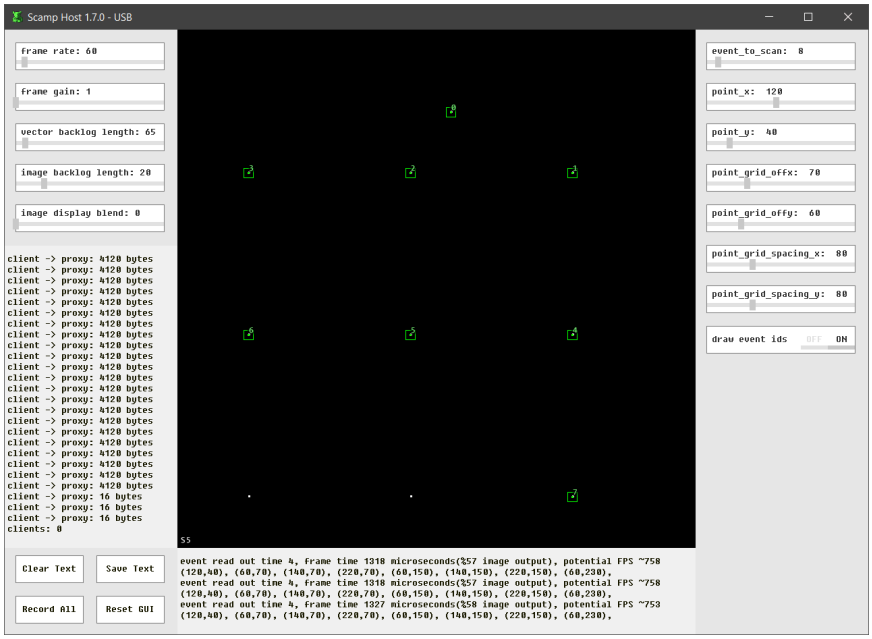
1. The digital register plane to scan ("**S0**")
2. An array into which it should write the coordinates of any events it finds ("**event_data_array**")
3. The number of events to stop scanning at ("**max_events_to_scan**")

The array will then be sequentially filled with pairs of (x,y) Event coordinates, in the order they were scanned. Once there are no events left to scan, the function will begin writing the coordinates (0,0) into the array.

Example Code : Scan Events

This example generates a binary image with a set of white pixels. It then demonstrates Event Readout, using "scamp7_scan_events" to retrieve the locations of these white pixels (i.e. Events). These events are then displayed along with the order in which each was scanned. The user can use sliders to alter the image from which events are being readout from.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



The binary image to scan events from is generated in S5. First we generate a grid of white pixels using two nested loops, iterating across x and y grid locations, and adding a new white pixel into S5 at each location. This involves using “scamp_load_point()” to generate single point images for each location, which are combined with S5’s content via OR operations.

```
//GENERATE A GRID OF POINTS
{
      int start_x = point_grid_offx%point_grid_spacing_x;
      int start_y = point_grid_offy%point_grid_spacing_y;
      for(int x= start_x ; x < 256; x+=point_grid_spacing_x)
      {
          for(int y = start_y ; y < 256; y+=point_grid_spacing_y)
          {
               scamp7_load_point(RN,x,y);//Load white pixel/point at current location
               scamp7_kernel_begin();
                    OR(S5,RN); //Add point to S5
               scamp7_kernel_end();
          }
      }
}
```

A final point is also added to S5.

```
scamp7_load_point(RN,point_y,point_x);
scamp7_kernel_begin();
      OR(S5,RN); //Add point to S5
scamp7_kernel_end();
```

Events are then scanned from S5, and their coordinates written into an array.

```
scamp7_scan_events(S5, event_data_array, number_of_events_to_scan);//Scan Events from
S5
```

The locations of these scanned events are shown on a GUI display using “scamp_display_events()”, passing to it the array of coordinates from “scamp7_scan_events()” previously.

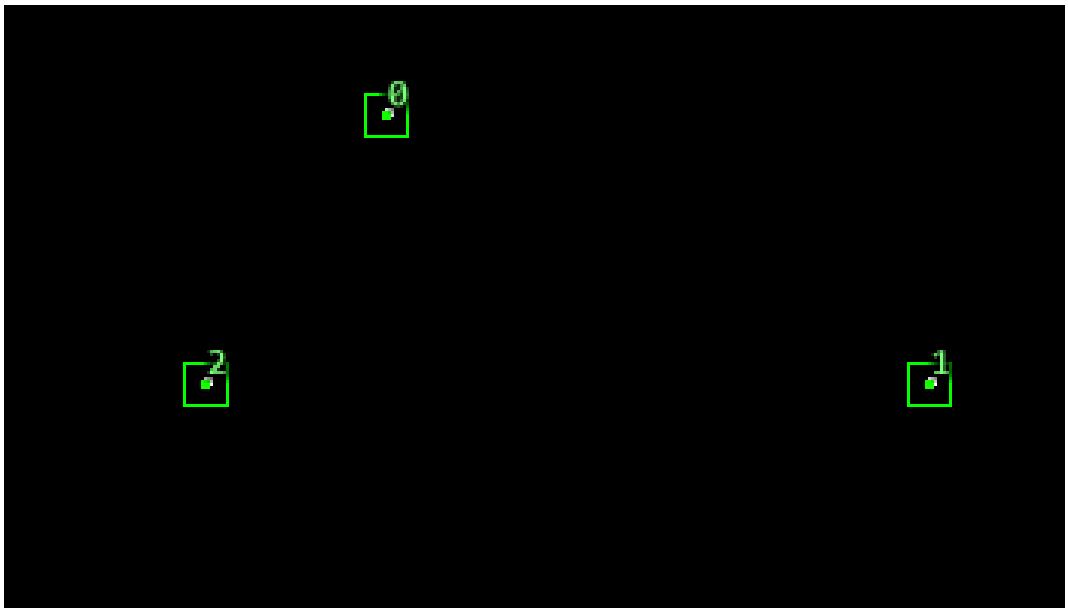
```
scamp7_display_events(display_00,event_data_array,number_of_events_to_scan);
```

In addition to the location of Events we also display the order in which they were scanned from S5 by looping through the array of coordinates, using “vs_gui_display_text()” to draw each event’s index at its location.

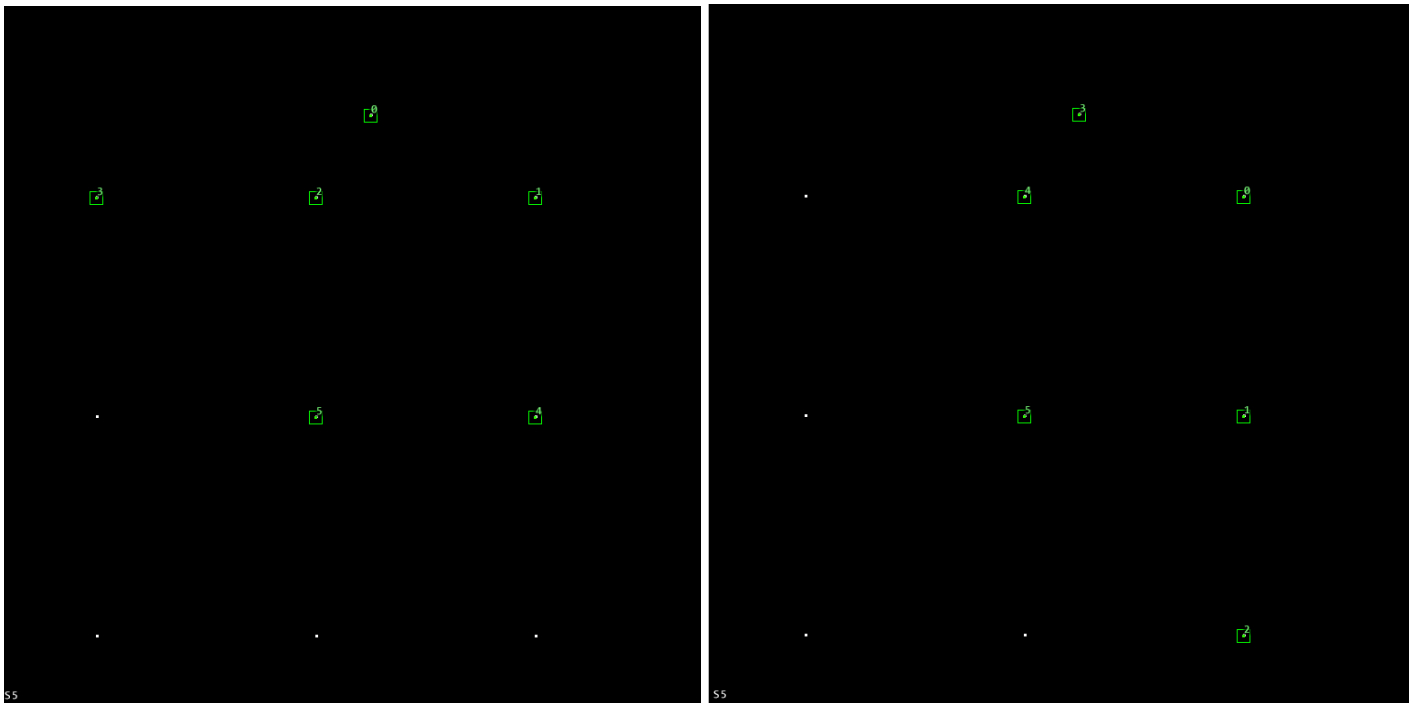
```
for(int n = 0 ; n < number_of_events_to_scan ; n++)
{
      int event_x = event_data_array[n*2];
      int event_y = event_data_array[n*2 + 1];

      const std::array<uint8_t, 4> text_color = {127, 255, 127, 255};
      std::string tmp_str = std::to_string(n);
      vs_gui_display_text(display_00, event_x, event_y, tmp_str.c_str(), text_color);
}
```

This produces an output for each scanned event as shown below, allowing the user to see how the order of events change as they move the grid / point around with the GUI sliders.



Scanning / Event Readout Order



The default order in which PEs are scanned (and hence events are readout) varies between the SCAMP5 & SCAMP7 because of reasons. In SCAMP7 event readout is performed row first, examining each row of PEs one at a time, top to bottom, within each row examining PEs left to right. In SCAMP5 readout is now column first, examining each column of PEs one at a time, left to right, within each row examining PEs going from top to bottom. This is shown above, with SCAMP7 on the left and SCAMP5 on the right.

GLOBAL SUMMATION

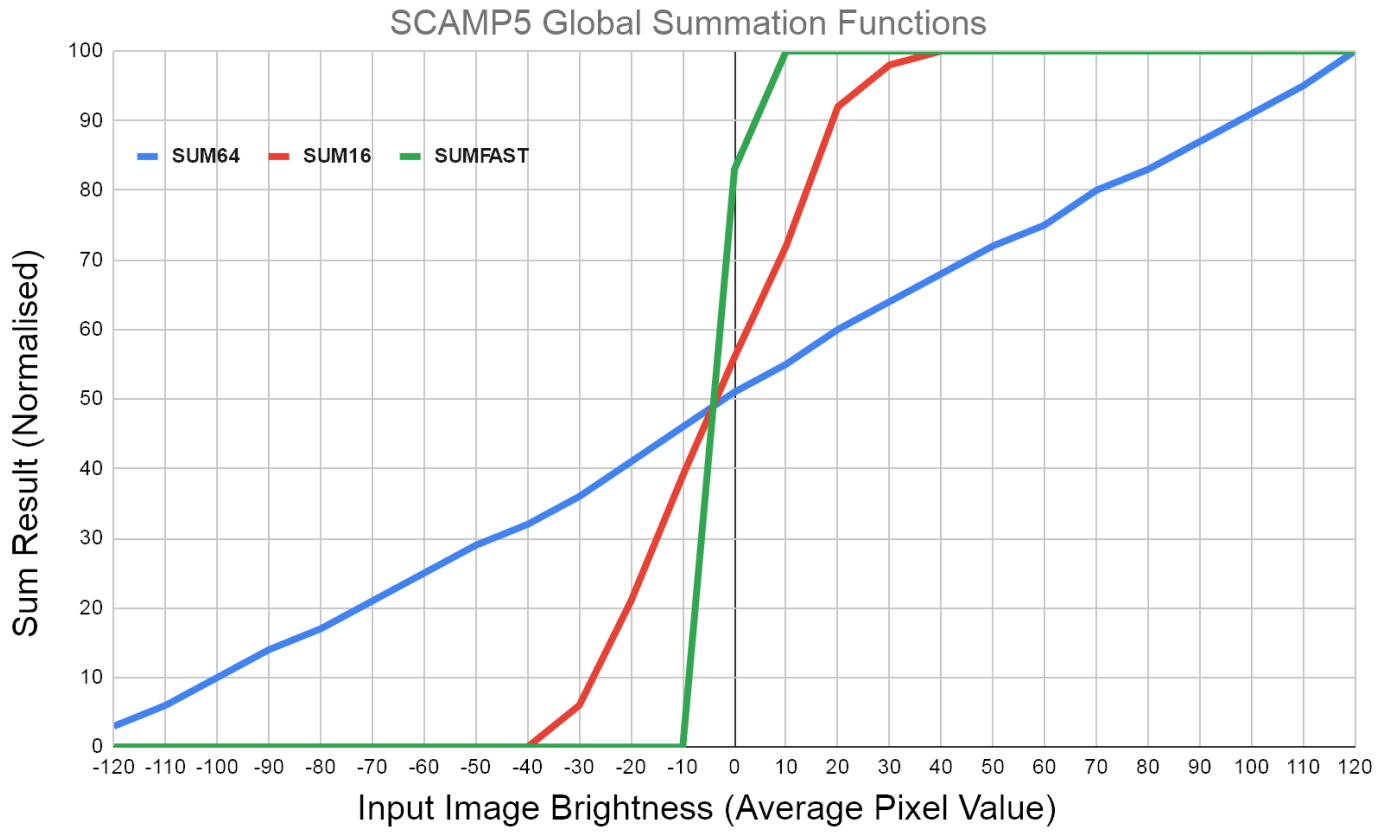
Another readout mechanism SCAMP is armed with is “Global Summation”. This allows us to choose a particular AREG plane, and then readout the sum of all analog registers in that plane. When performed upon a captured image stored in an AREG plane, this summation is that of all pixels of the image. But of course this summation can be performed over any form of data resulting from analog computation, making it a valuable tool.

Without such a global summation readout mechanism we would have to readout the entire image/AREG plane, which is extremely slow, and then perform summation of this data external from the sensor. Instead by using the sensor’s own global summation, only a single value is readout.

The most basic global summation functions are listed in the table below. Importantly each of these operates with a different degree of accuracy, and range of possible summations.

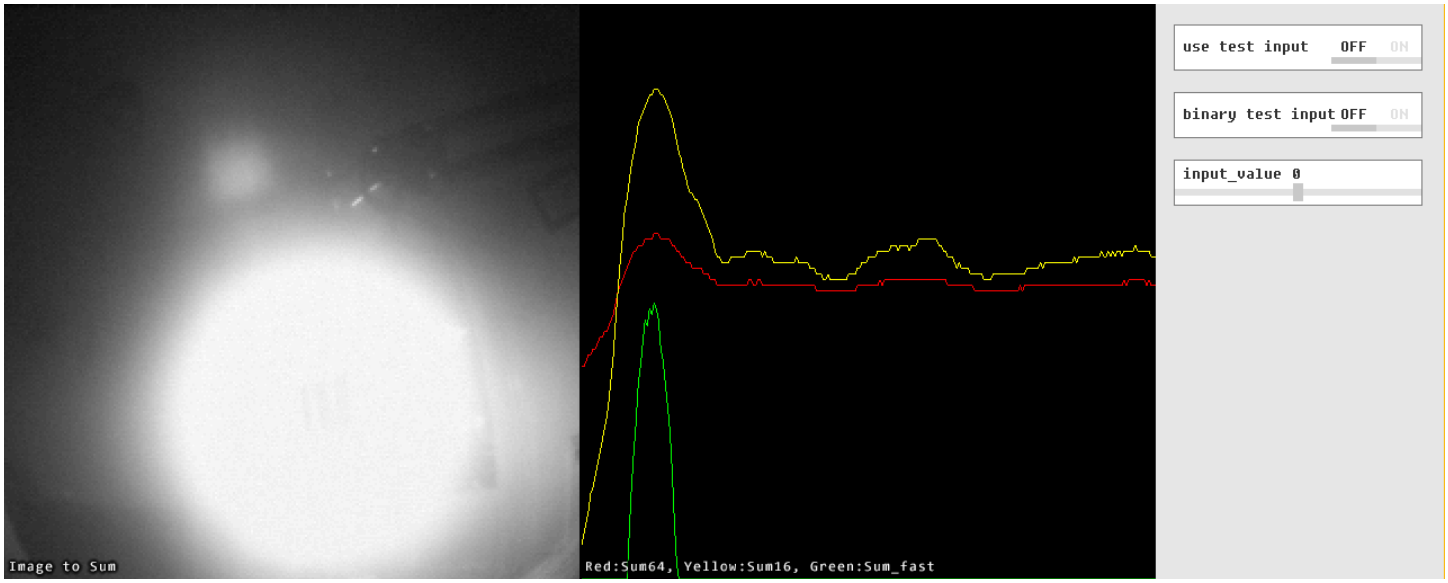
	SCAMP5	
Function	**Time(µs)**	**Operational Range**
scamp_global_sum_64	20	~ [-127,127]
scamp_global_sum_16	7	~ [-40,40]
scamp5_global_sum_fast	1	~ [-10,10]

Each global summation function has a maximum & minimum value it can produce. Whenever the true summation value lies beyond this maximum or minima, the function will not return the correct value, and will instead simply be capped at its maximum (or minima).



This effect can clearly be seen in the diagram above, in which each global sum function is tested upon input images of various brightnesses. Both “sum_16” & “sum_fast” only report changes in value across a small range of input image brightnesses, while “sum_64” operates correctly across the entire range.The data for this was obtained from the example below, comparing the result of each of these summation on the same input.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP7**



Example : Image Stabilisation

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP7**



This example demonstrates using global summation to align two images stored upon different register planes, specifically two binary edge images as seen in a previous chapter. Each captured frame is converted into an edge image, and aligned with a previously stored edge image referred to as the “key-frame”. This alignment involves determining the “alignment transformation” to apply to the edge image, so that there is maximum overlap between its edges and those of the key-frame.

In brief the steps of performing this alignment are…

1. Apply the alignment transformation of the previous frame to the current edge image (to roughly align it with the key-frame).
2. Test various small transformations, applying each to the edge image and determining the alignment strength between this transformed image and the key-frame.
3. Take the transformations which produced the strongest alignment and use them to update the "alignment transformation"

How well two edge images align is referred to as "alignment strength", and it is this which is measured by using global summation. This is done by first performing an AND operation between two binary edge images. This eliminates all edge pixels (ie. 1s) that do not overlap between the images. The number of remaining edge pixels then gives us an indication of the strength of alignment between the two images. We can roughly "count" the number of these overlapping edge pixels by converting the image to analog, and using global summation. The result of summation will then be proportional to the number of overlapping edge pixels, and hence alignment strength.

Note this is only a basic example, in which transformations considered for performing image alignment are only vertical and horizontal translations. A more complex approach would also make use of rotation and scaling method as mentioned previously, and which is similar to the approach used in the the paper Visual Odometry for Pixel Processor Arrays for simple 4DOF odometry.

8.Digital Flood

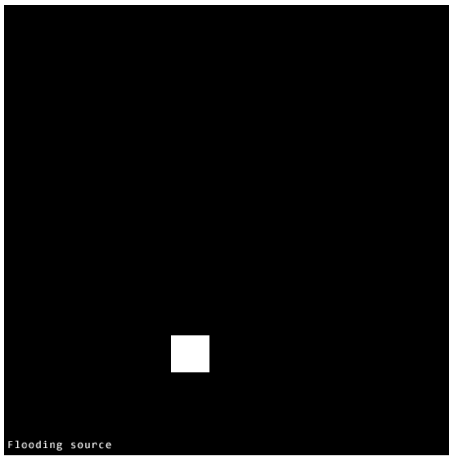
SCAMP has the ability to perform Flood-Fill spreading 1s across DREG planes, similar to the function of flood-fill tools found in most image editing applications such as microsoft paint. This flooding makes use of two DREG planes, named “Source” and “Mask”.

Source & Mask

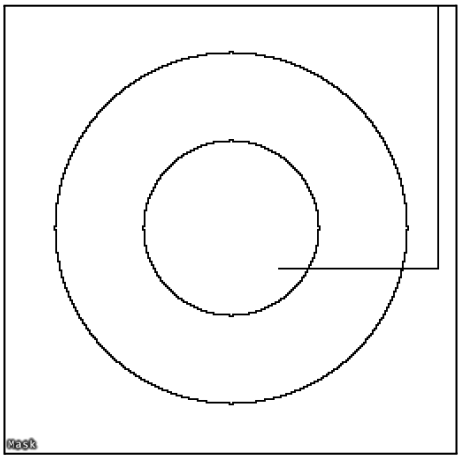
- The **Source** register plane both contains the 1s from which flooding will originate from, and is the plane which will be flooded. When flooding is performed, PEs in which **Source** = 1 will spread 1s into the Source registers of their neighbouring PEs, which in turn will spread 1s to their neighbours and so on.
- The **Mask** register plane is used to restrict/block the spread of 1s across the **Source** during flooding, 1s will only spread into PEs in which **Mask** = 1.

The **Source** register RZ/RP (SCAMP7/SCAMP5). The **Mask** register is RF.

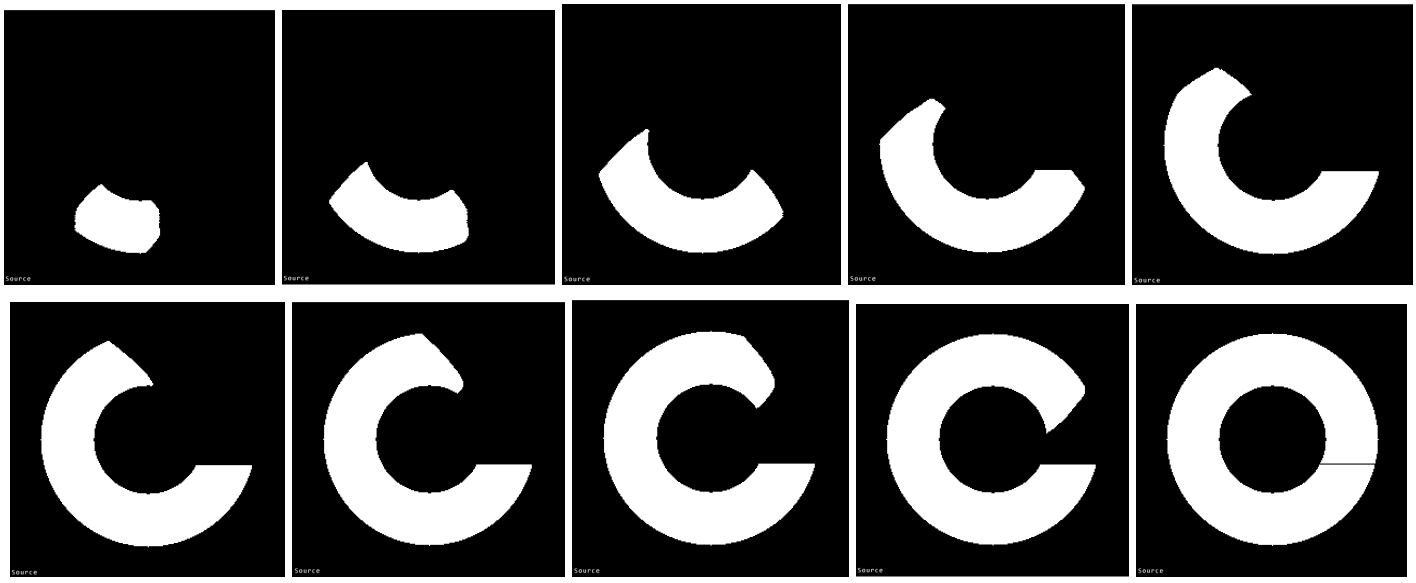
Below is an example of flooding, where a small square of 1s is loaded into the **Source**, and various black line shapes are drawn into **Mask**. Each time flooding is performed, the 1s of **Source** are spread a little further, but are restricted by the 0s of **Mask**. This results in a (broken) circular loop being flooded with 1s over many steps.



Source RZ



Mask RF



RZ after Flooding x1, x2 ,x3 …

“PROP_0” & “PROP_1”

The SCAMP kernel instructions used to perform flooding are named “PROP”, for propagation. The instruction syntax varies slightly between SCAMP7 & SCAMP5 as shown in the table below.

SCAMP7 instruction	**SCAMP5 instruction**	**Description**
PROP_0(RZ)	PROP0()	Flood from Source.
PROP_1(RZ)	PROP1()	Flood from Source & borders of PE array.
PROP_R()	*Not Required*	Required before performing PROP_0(RZ) or PROP_1(RZ).

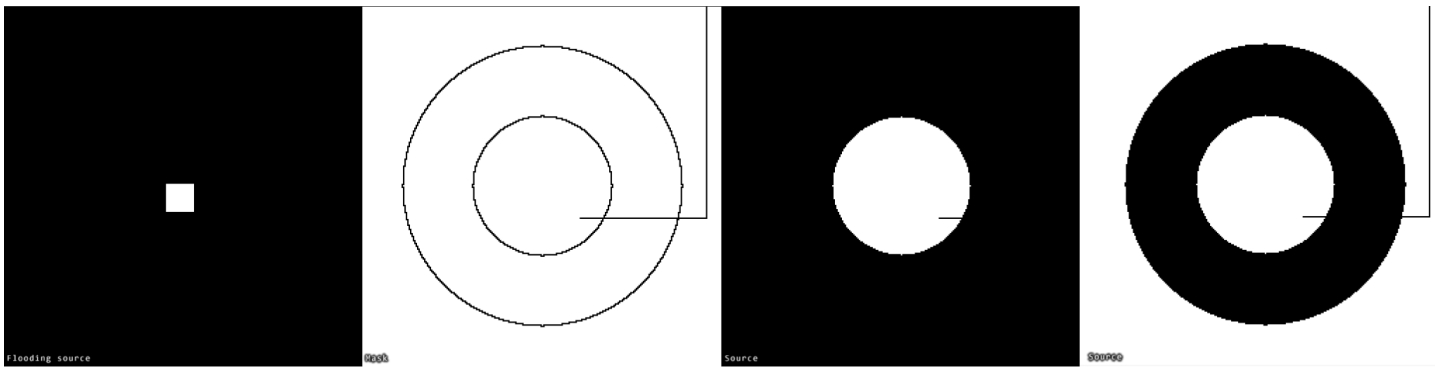
Performing flooding using these instructions will then typically look like the code shown below. First some content is copied into the **Source** and **Mask** DREG (RZ & RF), then the flooding is performed by executing the “PROP_R”, “PROP_0/1” instructions a given number of times. Finally the result after flooding is copied from RZ and RF is reset so as not to interfere with future computation.

```
scamp7_kernel_begin();
      MOV(RZ,S0);//Copy desired content into RZ to act as the Flooding Source
      MOV(RF,S1);//Copy desired content into RF to act as the Flooding Mask

      SET(RN,RS,RE,RW);//Set to allow flooding from all directions in all PEs
      PROP_R(RZ);//Setup Flooding
      for(int n = 0 ; n < 10; n++)
      {
          PROP_0(RZ);//Perform Flooding
      }

      MOV(S6,RZ);//Copy the result of Flooding
      SET(RF);//Reset RF = 1 in all PEs
scamp7_kernel_end();
```

The “PROP_1” instruction can be used to flood from the borders of the PE array, in addition to those PEs in which **Source** = 1. This is shown below in a comparison between “PROP_0” & “PROP_1”.

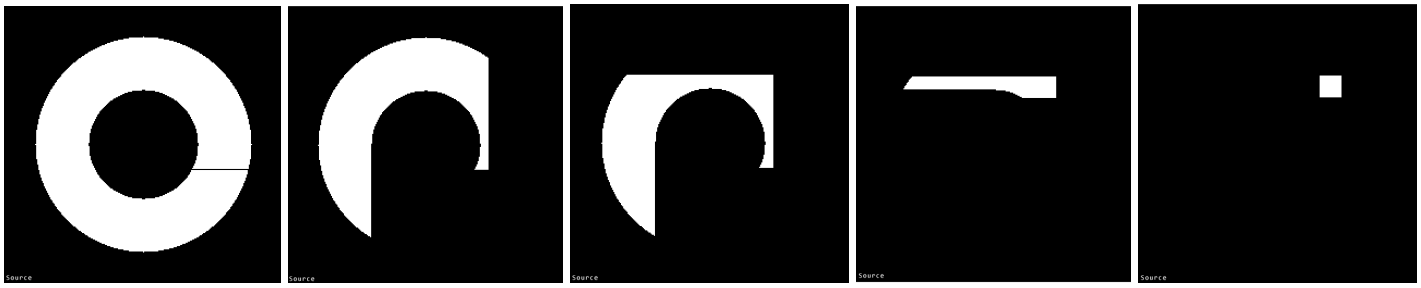


Source **Mask** **Flooding with PROP_0** **Flooding with PROP_1**

Direction Control RN,RS,RE,RW

Similar to DNEWS described in a previous chapter, flooding is affected by the content of the RN,RS,RE,RW registers, each of which controls from what directions a PE can be flooded. For example a PE can only receive 1s from its “north” neighbour if its register RN = 1. This allows flooding to be restricted to only occur horizontally, vertically, or in any number of more complicated schemes. Note in the previous code “SET(RN,RS,RE,RW)” is executed, enabling flooding to occur in all directions in all PEs.

Examples of flooding with various (global) RN,RS,RE,RW contents are shown below.



RN,RS,RE,RW = 1 **RW = 0** **RS,RW = 0** **RN,RS,RW = 0** **RN,RS,RE,RW = 0**

“Scamp_flood”

The SCAMP library also contains the following function which can be used to perform flooding.

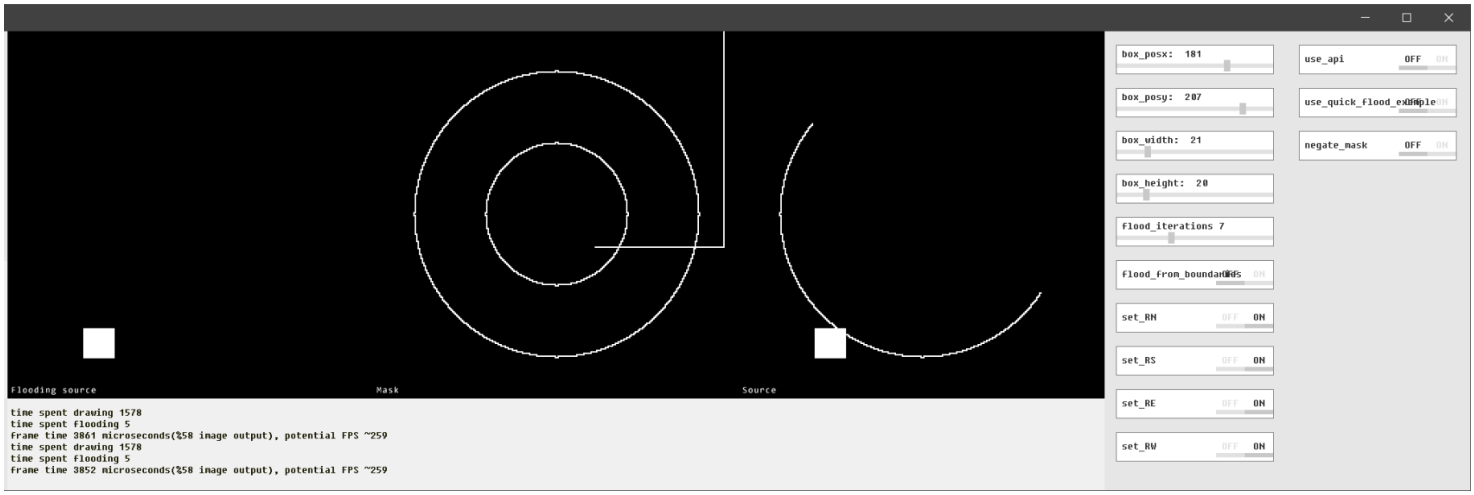
```
void scamp7_flood(DREG Source, DREG Mask, int Flood_from_borders, int
Flooding_iterations, bool Set_RN_RS_RE_RW)
```

Internally this function uses the same “PROP” instructions described previously. While convenient at the time of writing this function seems to be less efficient than using native SCAMP instructions, and is best suited to only flooding larger areas.

Example Code : Flooding 1

The example here demonstrates flooding. A number of circles and lines are drawn into a DREG plane for use as the flooding **Mask**, and a rectangular region into another DREG plane to act as the flooding **Source**. Flooding is then performed using this content. Controls on the GUI can be used to alter all aspects of flooding to help the user understand how it functions.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



This example demonstrates flooding via three methods.

1. Flooding via a single SCAMP kernel
2. Flooding using multiple SCAMP kernels
3. Flooding using “scamp_flood()”

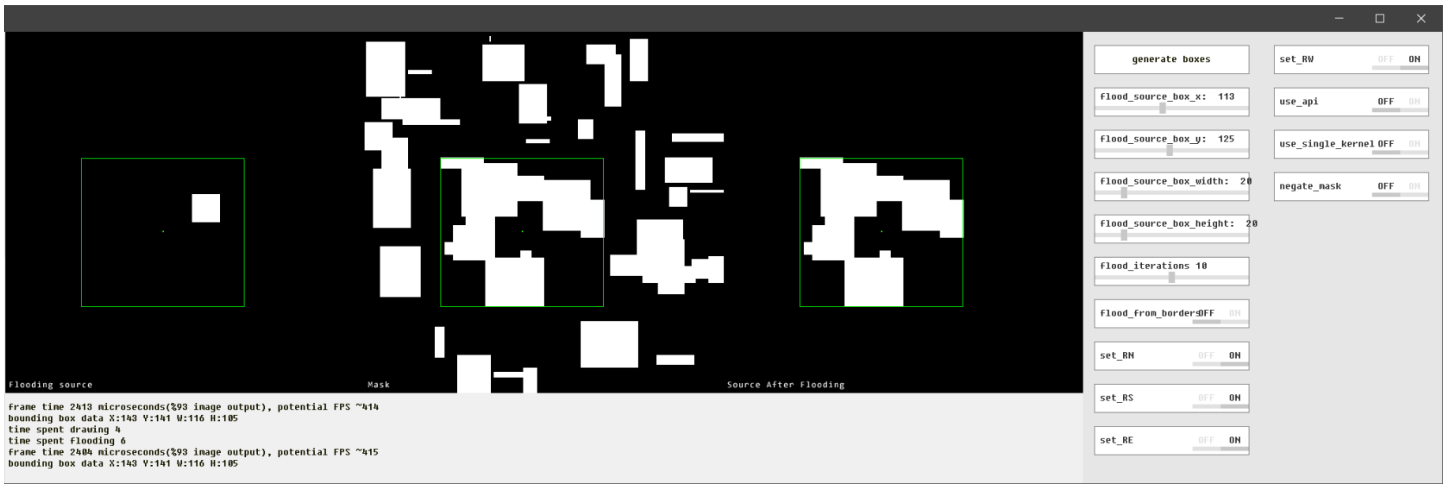
The table below shows the times each method takes to flood a certain sized area, showing how using SCAMP instructions can be significantly faster than the “scamp_flood()” function for flooding small areas. These numbers were obtained on SCAMP7.

Method	**Time to Flood Circle of Radius 30 (µs)**	**Time to Flood Circle of Radius 60 (µs)**	**Time to Flood Circle of Radius 120 (µs)**
Single SCAMP kernel	~1	~2	~4
Multiple SCAMP kernels	~2	~3	~4
library function	~5	~6	~6

Example Code : Flooding 2

This second example provides another demonstration of using flooding. Here the flooding mask is generated from a set of random rectangles, and can be regenerated by pressing the “generate mask” button. The bounding box of the flooded result is then drawn across all displays.

EXAMPLE CODE SCAMP5 EXAMPLE CODE SCAMP 7

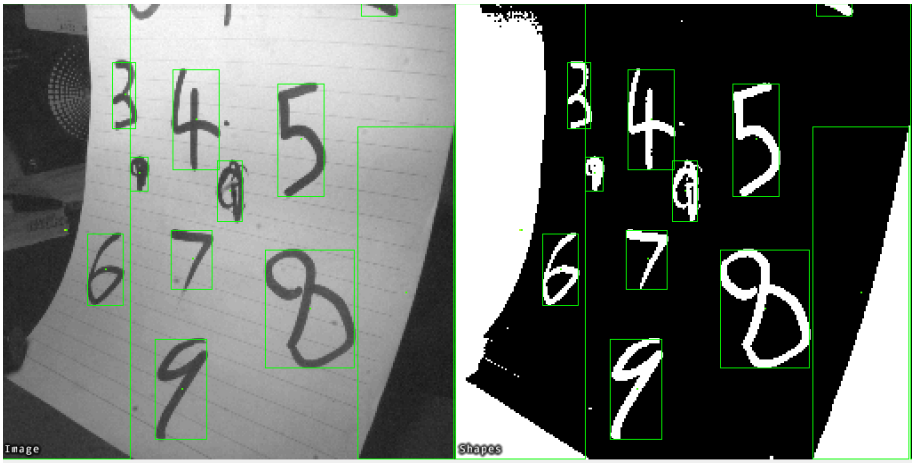



```
//Setup objects for random number generation
//This code looks like nonsense as the std classes here overload the function call operator...
std::random_device rd;//A true random number
std::mt19937 gen(rd()); // Mersenne Twister pseudo random number generator, seeded with a true random number
int random_min_value = 0;
int random_max_value = 255;
//Create distribution object to map a given random value to 0-255
std::uniform_int_distribution<> distr(random_min_value, random_max_value);
```

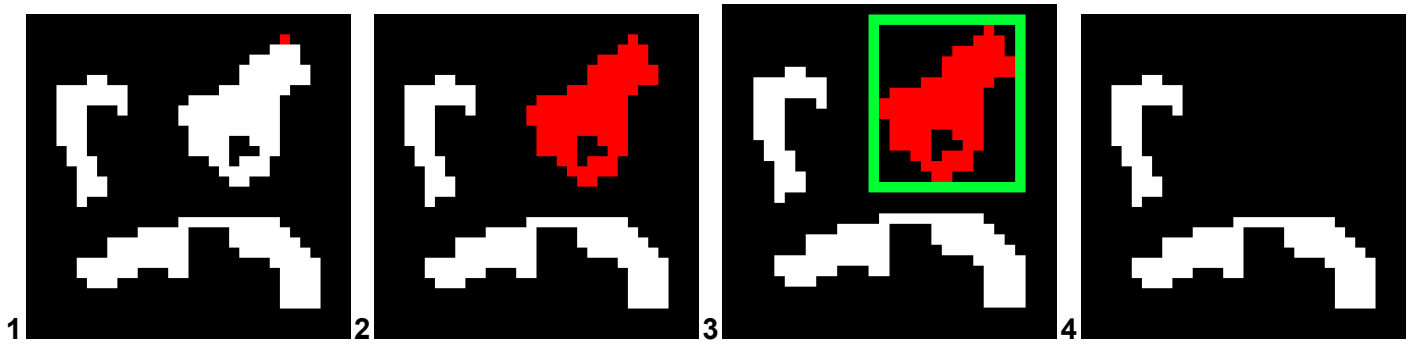
```
scamp7_kernel_begin();
	CLR(S0); //Clear content of S0
scamp7_kernel_end();
for(int n = 0 ; n < boxes_to_add ; n++)
{
	//Load box of random location and dimensions into S5
	int pos_x = distr(gen);
	int pos_y = distr(gen);
	int width = 1+distr(gen)/6;
	int height = 1+distr(gen)/6;
	DREG_load_centered_rect(S5,pos_x,pos_y,width,height);
	scamp7_kernel_begin();
	OR(S0,S5);//Add box in S5 to content of S0
	scamp7_kernel_end();
}
```

9. DREG Blob/Shape Extraction

There are many scenarios in which an algorithm processes an image, producing a number of shapes/blobs in a DREG plane, each representing some sort of detection or region of interest. A very simple example of this would be thresholding a captured image to a binary one, which may then contain a number of shapes such as those shown by the bounding boxes of the image below.



It is then common to examine each shape individually (such as classifying each shape as a digit for the above image). Such per-shape processing requires us to extract each shape, separating it from the rest of the image. This section describes the process of performing this shape extraction.



In brief there are four steps involved in extracting a shape… **1** get a pixel within a shape, **2** flood that shape, **3** extract the shape's bounding box, **4** remove the shape from the binary image allowing others to then be extracted. These are illustrated above, with red pixels indicating the shape being extracted from the image. Now let us go over the code which can be used to perform each of these steps.

Step 1 : Locate a Pixel within one of the Shapes

First we find the location of a pixel that is inside one of the shapes. This is done using the function "Scamp_scan_events()" to get the x,y coordinates of a white pixel (i.e. a "1") inside the binary image containing the shapes. The code below will locate such a pixel in a given DREG plane "DREG_shapes".

```
uint8_t event_data_array [2];
scamp7_scan_events(DREG_shapes, event_data_array,1);
int event_xpos = event_data_array[0];
int event_ypos = event_data_array[1];
```

Step 2 : Flood Fill the Shape

Now with the coordinates of a pixel within one of the shapes, we can perform SCAMP's digital flooding originating from this pixel's location, to flood-fill the whole shape. This involves first loading a single "1" into a DREG plane at that pixel's coordinates using "scamp_load_point()". Now we perform digital flooding, using this DREG plane as the Source register, and the DREG containing shapes as the Mask register as shown below.

```
scamp7_load_point(DREG_flooding_source, event_ypos, event_xpos);
scamp7_flood(DREG_flooding_source, DREG_shapes, 0, 10)
```

Step 3 : Extract bounding Box (Optional)

The flooded DREG plane ("DREG_flooding_source") will now contain a single shape, that of the pixel we located previously in Step 2. If needed we can now use scamp7_scan_boundingbox() to extract the bounding box coordinates of this shape.

```
uint8_t bb_data [4];
scamp7_scan_boundingbox(DREG_flooding_source,bb_data);
```

Step 4 : Remove Extracted Shape

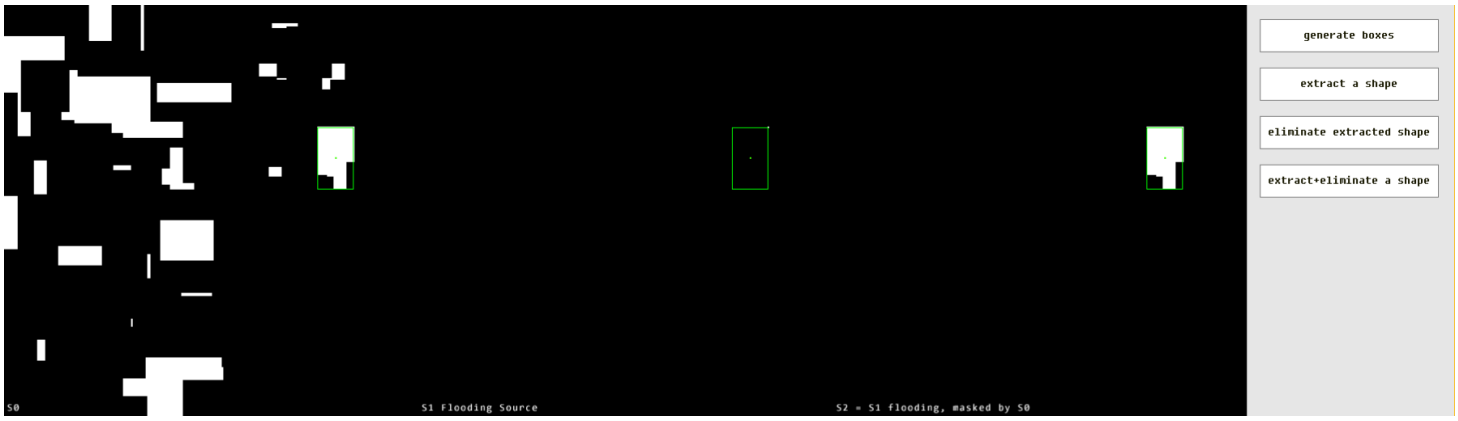
Finally we remove the extracted shape from the binary image of shapes. This allows us to then extract another different shape from the image, repeating steps 1-5. This is performed using some simple boolean logic between DREG planes shown below. If we did not perform this removal, the previous steps 1-4 would simply extract the same shape again.

```
scamp7_kernel_begin();
    //Invert DREG plane holding extracted shape and store in S6
    NOT(S6, DREG_flooding_source);
    //Eliminate extracted shape from "DREG_shapes" by performing AND operation with S6
    AND(DREG_shapes, DREG_shapes, S6);
scamp7_kernel_end();
```

These steps are demonstrated in the following example.

Example Code : Shape Extraction Step by Step

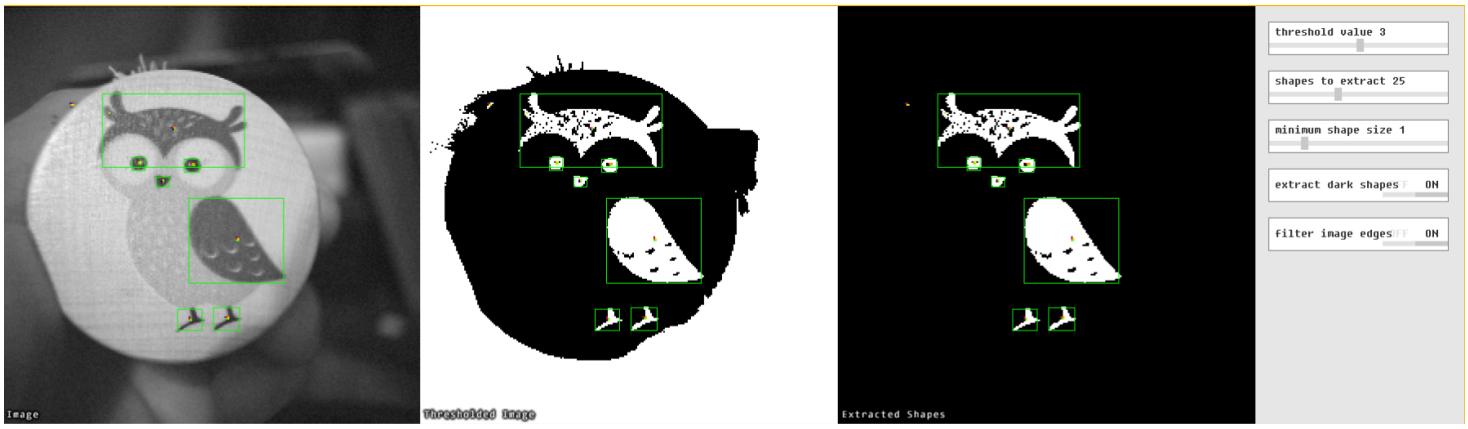
EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**

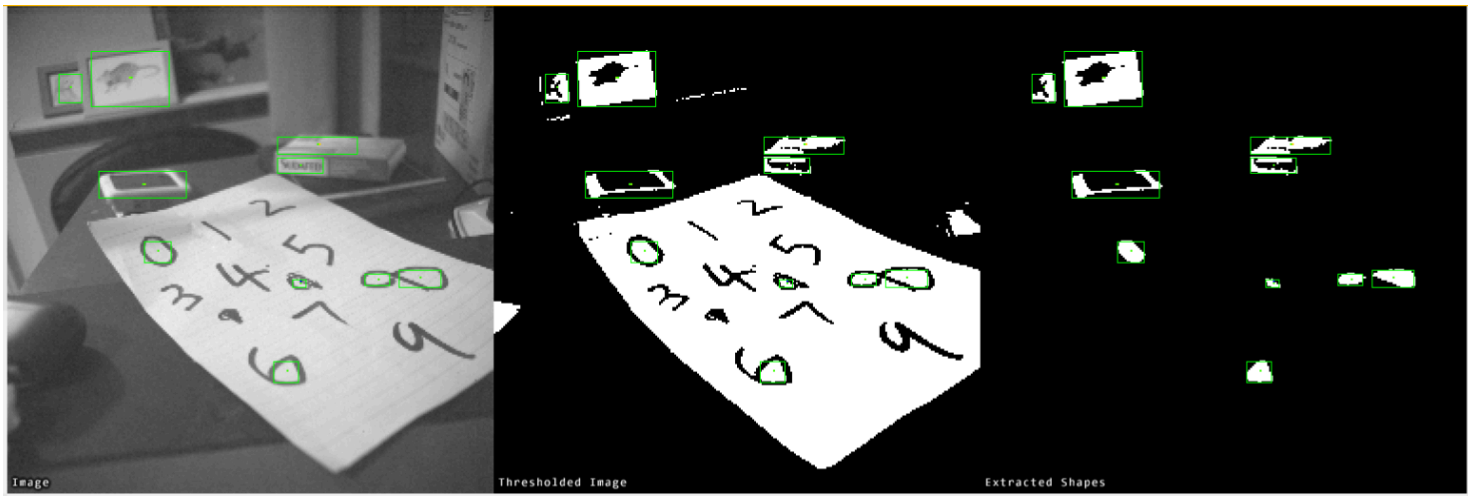


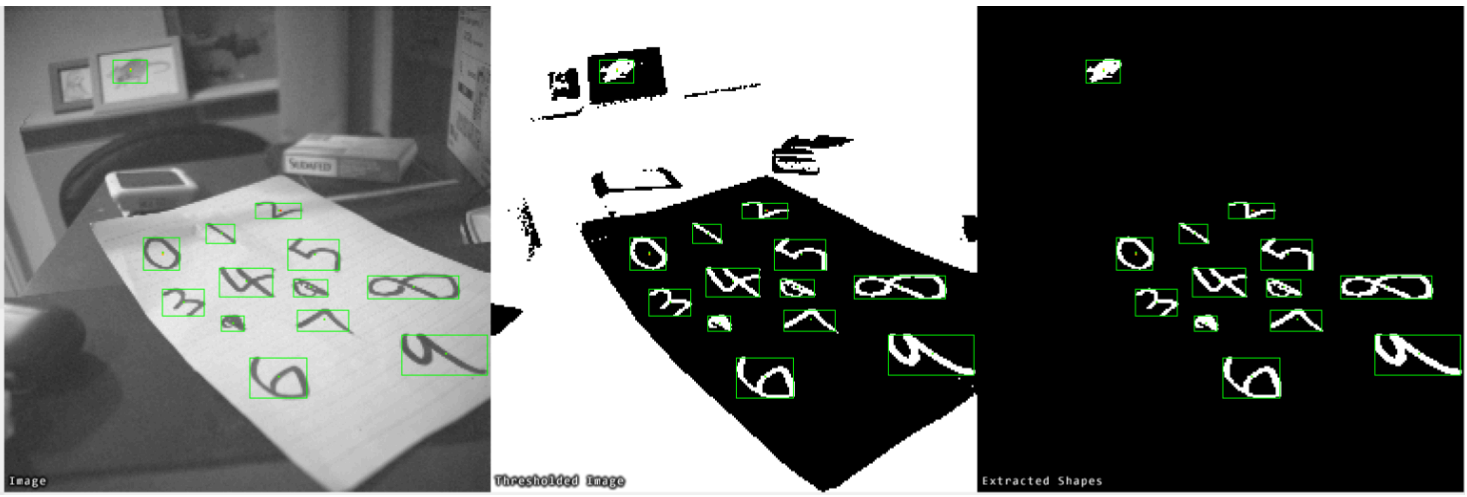
Example Code : Shape Extraction

The following example demonstrates shape extraction on a live camera image. In addition to using the steps described previously in this section, the code uses a few tricks to provide cleaner & more reliable results. First digital erosion (described in a previous section) is used to eliminate very small shapes from the threshold image before finding shapes to extract. These small shapes are things such as single pixel “speckle” noise, and are unwanted in the final results. Secondly shapes that are in contact with the borders of the image are eliminated to ensure only shapes entirely within the sensor's view are extracted. This is done by utilising flooding from the image borders as described in a previous section.

EXAMPLE CODE SCAMP5 EXAMPLE CODE SCAMP 7

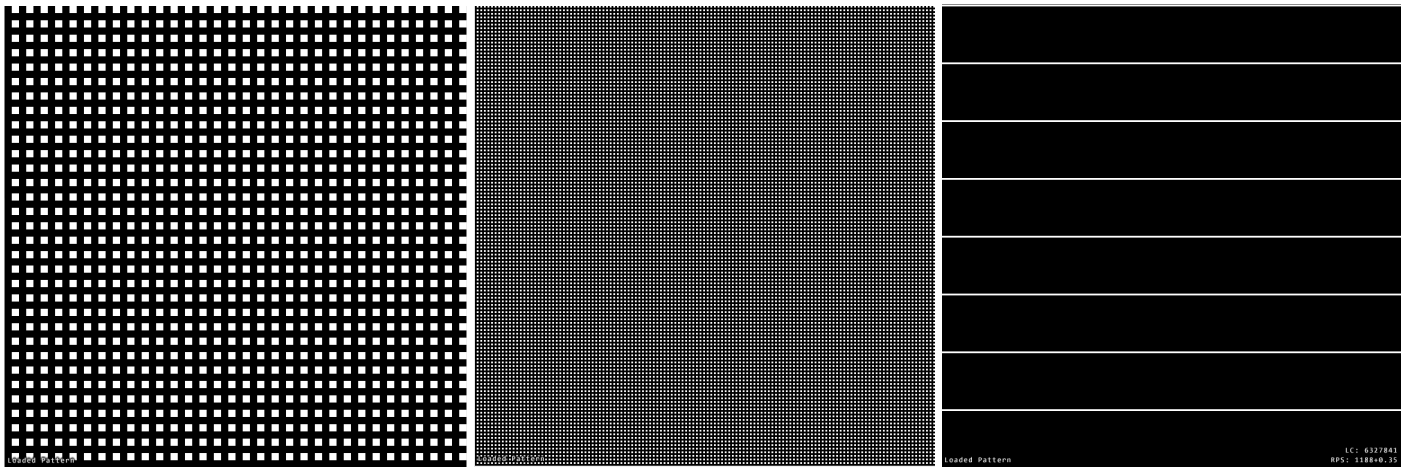






10.Loading DREG Patterns

In previous chapters we saw the relatively simple "scamp_load_point()" and "scamp_load_rect()" functions, which are used to load content (points & rectangles of 1s) into a digital register plane. We will now introduce a more complex function which can be used to efficiently load various patterns to a digital register plane, such as those shown below.



Although it is possible to create such patterns by making numerous "scamp_load_rect()" calls, doing so would have a huge computational cost.

“scamp_load_pattern()”

The function to generate such patterns is "scamp_load_pattern()" as shown below. This takes five parameters, starting with the DREG plane into which to load the pattern. The four remaining parameters are 8-bit binary numbers determining the loaded pattern by controlling which PEs will receive 1s.

```
void scamp7_load_pattern(DREG register,int row, int col, int rowx, int colx);
```

The processor array contains 256 rows and 256 columns of PEs. The **col** & **colx** parameters together “select” a set of columns, while **row** & **rowx** select a set of rows. The pattern is then loaded by placing 1s into the DREG of PEs whose row and column have both been selected (all other PEs receiving 0s).

Column & Row selection

To explain how these parameters function let us first only consider **col** & **colx**. These parameters, and the indexes of each PE column, are treated as 8-bit patterns (ie. 67 = 01000011). A comparison between the bits of these parameters and index of each column is then performed, determining which columns are selected.

The parameters perform the following roles in this comparison

- **col** - Specifies the bit pattern to compare against each column’s index.
- **colx** - Specifies “ignore” locations, where the bit pattern and the column index need not match

Selected columns are then those whose indexes match the **col** bit pattern, excluding any mismatches in “ignore” locations defined by **colx**. Note these are terrible parameter names… something like “**col_selection_match**” and “**col_selection_mask**” would be a far better choice than **col** & **colx**.

Let us now look at some examples of column selection based upon the values of **col** and **colx**. For simplicity we will only consider the first nine columns of PEs (columns 0-8). The example tables show the comparisons between **col** and each column index, with matching bits in green and mismatches in red.

EXAMPLE 1

Col = 5 (00000101), **Colx** = 0 (00000000)

Column Index	0	1	2	3	4	5	6	7	8
Index Bits	00000000	00000001	00000010	00000011	00000100	00000101	00000110	00000111	00001000
Comparison with "Col"	00000000	00000001	00000010	00000011	00000100	00000101	00000110	00000111	00001000
Apply Colx Ignore Bits	00000000	00000001	00000010	00000011	00000100	00000101	00000110	00000111	00001000
Selected	NO	NO	NO	NO	NO	YES	NO	NO	NO

Here we have **colx** = 00000000 meaning there are no "ignore" bit locations. For a column to be selected, its index must now exactly match the bit pattern of **col**. With **col** = 5, we see only the 5th column is selected.

EXAMPLE 2

Col = 0 (00000000), **Colx** = 3(00000011)

Column Index	0	1	2	3	4	5	6	7	8
Column Bits	00000000	00000001	00000010	00000011	00000100	00000101	00000110	00000111	00001000
Comparison with "Col"	00000000	00000001	00000010	00000011	00000100	00000101	00000110	00000111	00001000
Apply Colx Ignore Bits	000000xx	000000xx	000000xx	000000xx	000001xx	000001xx	000001xx	000001xx	000010xx
Selected	YES	YES	YES	YES	NO	NO	NO	NO	NO

In this example **colx** = 00000011, meaning the first two bits are ignored when comparing **col** to a column's index. With **col** = 0, the indexes of the first four columns (indexes 0-3) pass this comparison and are selected.

EXAMPLE 3

Col = 1 (00000001), **Colx** = 254 (11111110)

Column Index	0	1	2	3	4	5	6	7	8
Column Bits	00000000	00000001	00000010	00000011	00000100	00000101	00000110	00000111	00001000
Comparison with "Col"	00000000	00000001	00000010	00000011	00000100	00000101	00000110	00000111	00001000
Apply Colx Ignore Bits	xxxxxxx0	xxxxxxx1	xxxxxxx0	xxxxxxx1	xxxxxxx0	xxxxxxx1	xxxxxxx0	xxxxxxx1	xxxxxxx0
Selected	NO	YES	NO	YES	NO	YES	NO	YES	NO

Now with **colx** = 11111110, all bits except the first are ignored during the comparison between **col** and a column's index. This results in an alternating pattern of every other column being selected.

We can see from these examples that a great range of column selections can be made from these two parameters **col** & **colx**. The parameters **row** & **rowx** determine the selection of rows, functioning in an identical manner to **col** & **colx**.

Let us now look at an example considering all four parameters **col, colx, row, rowx** and which PEs will be selected from the first 8 rows and columns of the processor array.

EXAMPLE 4

Col = 1 (00000001), **Colx** = 254 (11111110), **Row** = 5 (00000101), **Rowx** = 0(00000000)

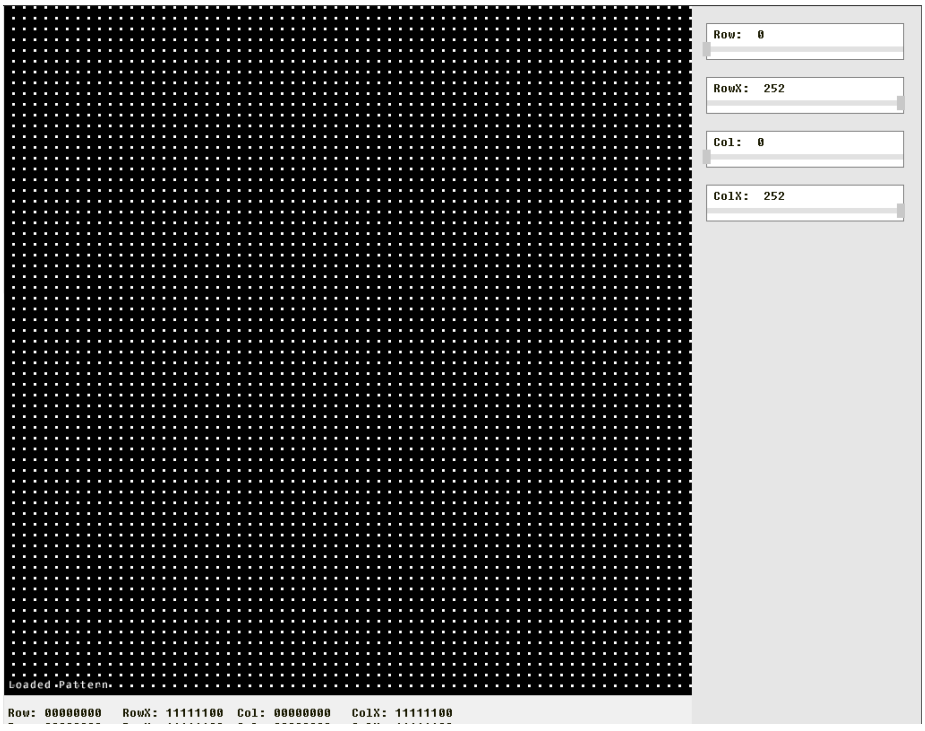
	Column Index	00000000	00000001	00000010	00000011	00000100	00000101	00000110	00000111	00001000
Row Index		xxxxxxx0	xxxxxxx1	xxxxxxx0	xxxxxxx1	xxxxxxx0	xxxxxxx1	xxxxxxx0	xxxxxxx1	xxxxxxx0
00000000	00000000	NO	NO	NO	NO	NO	NO	NO	NO	NO
00000001	00000001	NO	NO	NO	NO	NO	NO	NO	NO	NO
00000010	00000010	NO	NO	NO	NO	NO	NO	NO	NO	NO
00000011	00000011	NO	NO	NO	NO	NO	NO	NO	NO	NO
00000100	00000100	NO	NO	NO	NO	NO	NO	NO	NO	NO
00000101	00000101	NO	YES	NO	YES	NO	YES	NO	YES	NO
00000110	00000110	NO	NO	NO	NO	NO	NO	NO	NO	NO
00000111	00000111	NO	NO	NO	NO	NO	NO	NO	NO	NO
00001000	00001000	NO	NO	NO	NO	NO	NO	NO	NO	NO

This example takes some parameter values from previous examples, but now considers both row and column selection. The values of **col** & **colx** result in every other column being selected, while **row** & **rowx** result in only the fifth row being selected. This produces an alternating selection of PEs along the fifth row of the array.

Example Code : Load Pattern

This example demonstrates the use of “scamp_load_pattern()”. The user can change the values of the parameters **col, colx, row & rowx** and observe the resulting pattern.

[EXAMPLE CODE SCAMP5] [EXAMPLE CODE SCAMP 7]



PLOTTING

The Host GUI has the ability to perform basic plotting of live data sent from the Scamp system, which can be extremely useful for visualisation and debugging. This involves adding displays to the Host GUI, configuring them to act as plots, and sending them data to plot from within the Scamp's program using "vs_post" instructions. Some basic examples of such plotting are given below.

Bar Charts

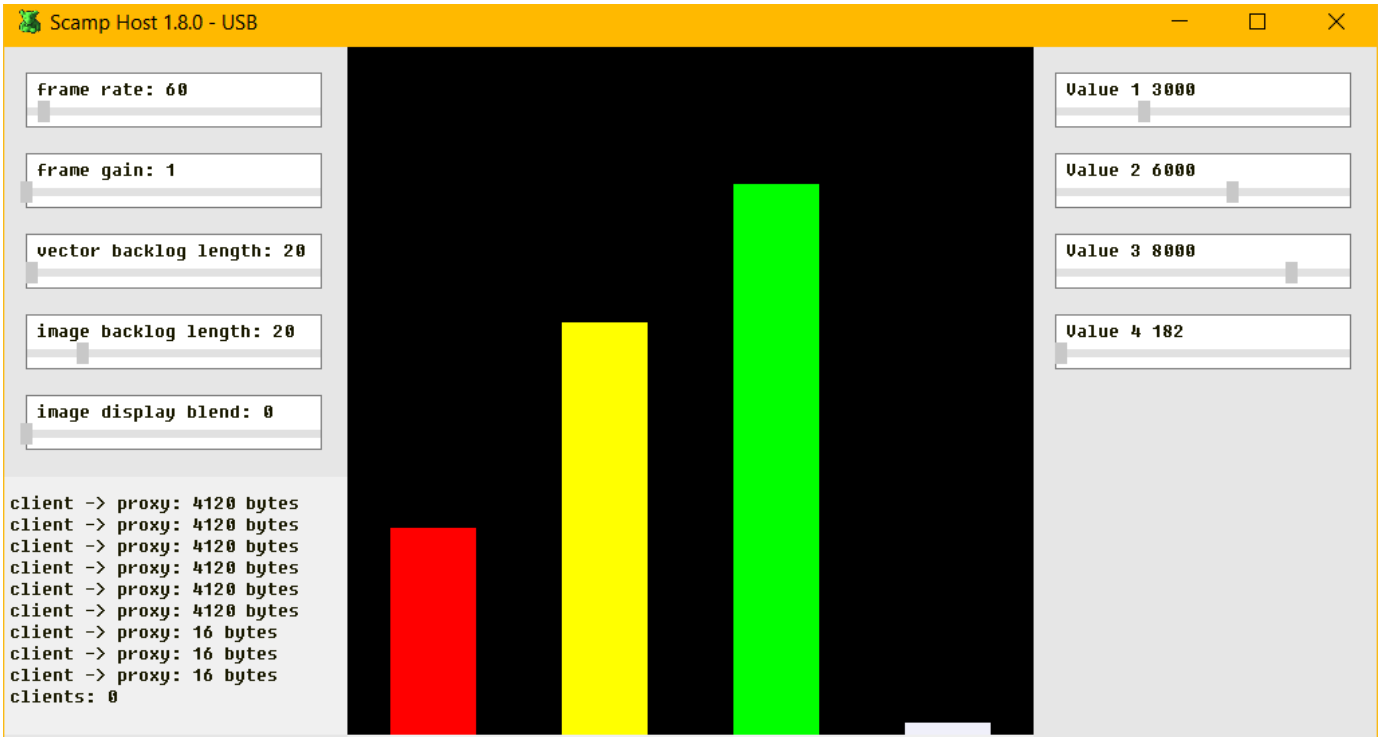
This example shows how to set up a bar chart on the Host GUI, and display some values which can be altered by sliders. A display is added, and then "vs_gui_set_barplot" is called using the handle of that display to configure it to act as a bar chart when it receives an array of data.

```
//Add a display that will contain the bar chart
vs_handle plot_display = vs_gui_add_display("",0,0,display_size,style_plot);
vs_gui_set_barplot(plot_display, plot_min_value, plot_max_value, plot_bar_width);
```

We update this plot by sending an array of data to the display we set up earlier. First "vs_post_set_channel" is used to select our display as the data's destination, then "vs_post_int32" is used to send the array of data. Each value in this array is then plotted as a separate bar.

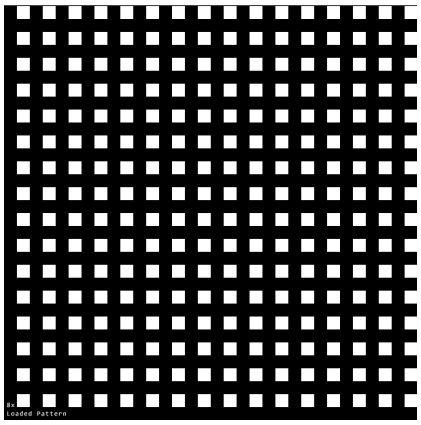
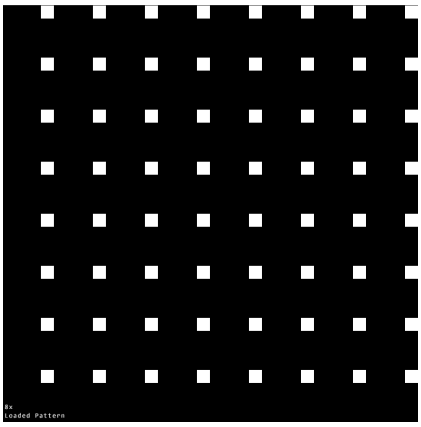
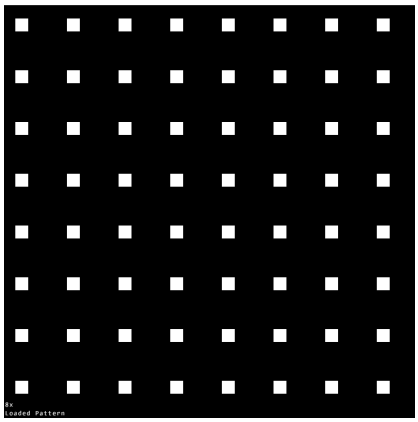
```
int32_t plot_data[4]; //Create an array of data
plot_data[0] = value_1;
plot_data[1] = value_2;
plot_data[2] = value_3;
plot_data[3] = value_4;
vs_post_set_channel(plot_display);//Select the target of "vs_post"
vs_post_int32(plot_data, 1, 4);//Post the array of data
```

EXAMPLE CODE SCAMP5 EXAMPLE CODE SCAMP 7

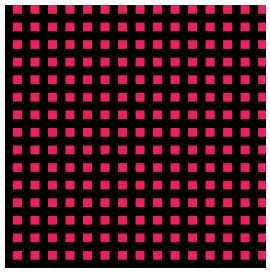
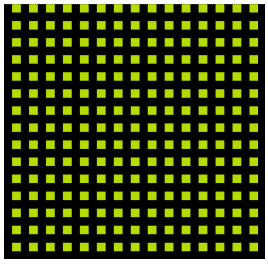
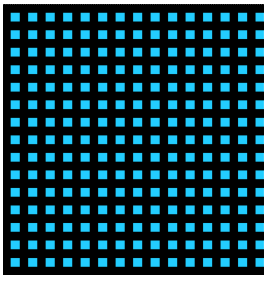
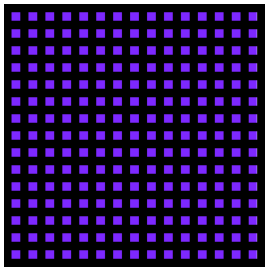
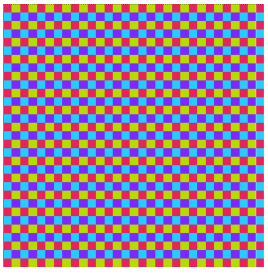


Interleaved Data & Processing

One of the most common uses of “scamp_load_pattern()” from the previous chapter is generating “dot grids”, placing 1s into every Nth PE of a DREG plane. Examples of such grid patterns are shown below, zoomed in x8 for visibility, each white square being a single “1” inside a PE.

Every 2nd PE	**Every 4th PE**	**Every 4th PE, shifted (2,1)**
Col = 0, **Colx** = 254 **Row** =0, **Rowx** = 254	**Col** = 0, **Colx** = 252 **Row** =0, **Rowx** = 252	**Col** = 2, **Colx** = 254 **Row** =1, **Rowx** = 254
		

Remember that the processor array is a 256x256 grid of PEs. These dot grid patterns can be used to universally partition the processor array into smaller sub grids of PEs. For example, using the parameters values of **Colx** = 254, **Rowx** = 254, four separate sparse grid patterns can be loaded, each with different values for **Col**, **Row**. This is shown below, with each subgrid being shown in a different colour, and on the right side showing the PEs of each subgrid overlaid together.

Colx = 254, **Rowx** = 254				
Col = 0, **Row** = 0	**Col** = 1, **Row** = 0	**Col** = 0, **Row** = 1	**Col** = 1 **Row** = 1	Overlaid Grids
				

Being able to partition the array into such sub grids of PEs proves to be extremely useful for many types of algorithms. Using conditional instruction execution (such as with FLAG), each sub grid can be used to process and/or store separate data.

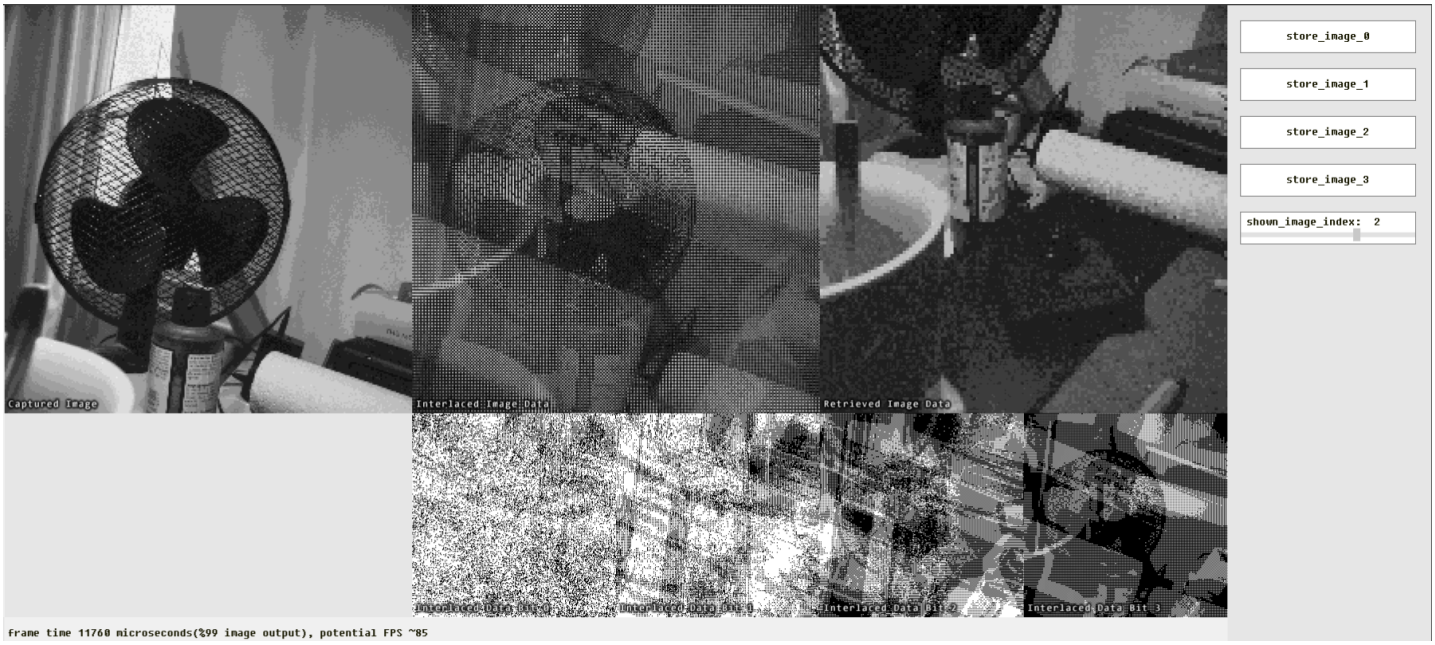
Alternatively, such sub grids can be used to group together blocks of neighbouring PEs into “super pixels”, essentially sacrificing “resolution” for increased data storage per “pixel”. In such a scenario each sub grid is considered as a separate “channel” of the image. For the example above with 4 subgrids, 2x2 “super pixels”

Example Code : Interleaved Image Storage

This demonstration of interleaved data builds upon the previous example of converting and storing captured analogue images to 4-bit representations in DREG using analog to digital conversion. The earlier example used 4 DREG (1 per bit) to store a single 256x256 image. This example again uses 4 DREG, but instead stores four 128x128 images, sacrificing resolution to store more images. This is done by splitting the processor array into 4 sub grids for the process of storage, placing a separate image within the PEs of each grid.

An example of this interleaved data can be seen in the image of the program running below. The four DREG planes storing the interleaved 4-bit data of the stored images are shown along the bottom row of displays. The top middle display shows the result of converting this digital data back to an analogue image, in which four interleaved images can be seen. The top right display then shows one of the four stored images extracted out from the interleaved data, its pixel data replicated to fill all PEs of the array giving it a blocky appearance.

EXAMPLE CODE SCAMP5 EXAMPLE CODE SCAMP 7



\

When any of the “store_image” GUI buttons are pressed the variable “triggered_image_store” is set to the index of a sub grid (0-3) into which to store an image. The pattern of this subgrid is then loaded into S6.

```
int col = triggered_image_store % 2 == 0 ? 0 : 1;
int row = triggered_image_store / 2 == 0 ? 0 : 1;
scamp7_load_pattern(S6, row, col, 254, 254);
```

The code for then performing ADC and storing the 4-bit image into DREGs closely follows that of the previous ADC example. However data is only altered in those PEs belonging to the sub grid pattern loaded to S6. This is performed with use of the RF & RM registers, exploiting RF’s masking of operations upon RM to leave data in other subgrids untouched. An example of the instructions used to perform this are shown below.

```
SET(RF);//RF = 1 in all PEs
MOV(RM,S0);//Will copy S0 to RM in PEs where RF = 1, i.e. ALL PEs!
MOV(RF,S6);//Copy sub grid pattern into RF
MOV(RM,FLAG);//Copy FLAG into RM in PEs of sub grid (where RF == 1)
```

```
MOV(S0,RM);//Copy result of masked operation into DREG of 1st bit
```

Example Code : Super Pixels, 4bit image in 1 DREG

11.Analogue Register Issues

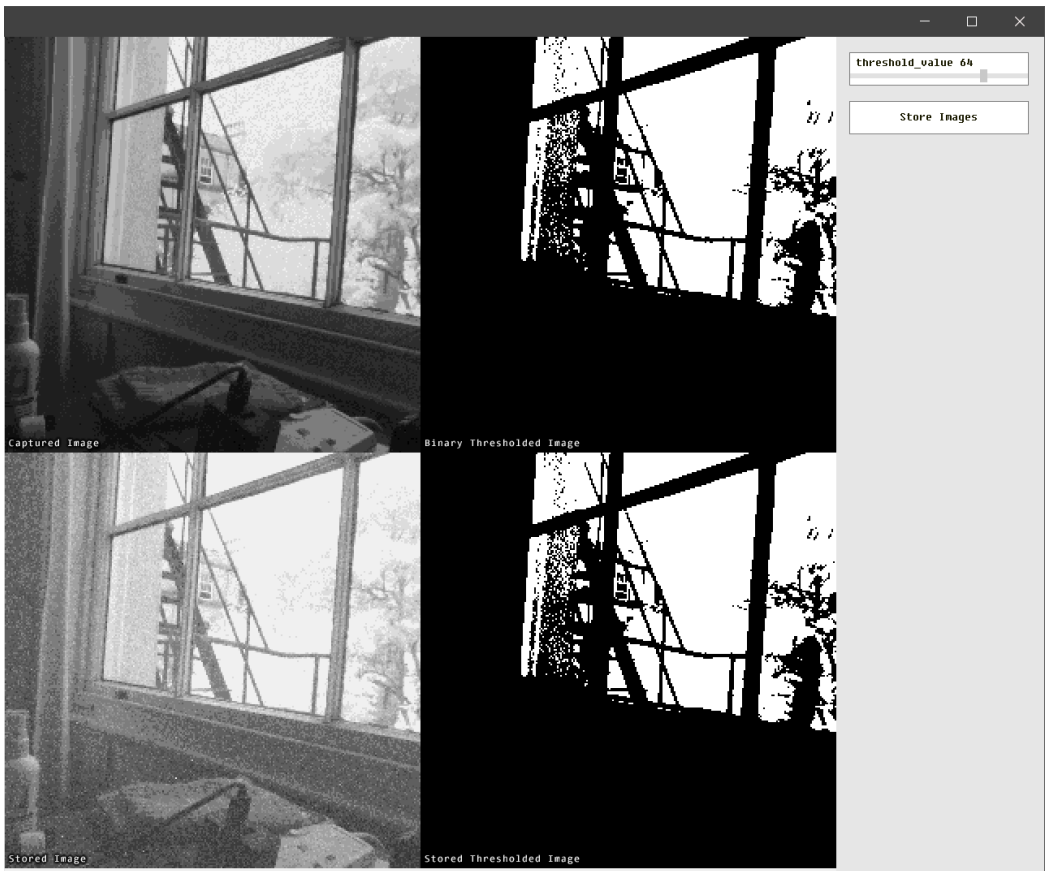
Decay

Unlike digital memory registers which store discrete binary states (0/1), analogue memory registers store continuous values/signals. However this continuous nature makes such registers prone to decay, that is a gradual corruption of the stored data over time. In regards to SCAMP, the analogue registers (AREG) within each pixel-processor demonstrate such decay. The decay of SCAMP AREG occurs as a general drift upwards of the stored values, resulting in a complete loss of data within ~5-10 seconds. This makes AREG unfit for use as long term data storage without the use of mitigation strategies such as quantization of analogue values. Instead AREG should primarily be used for performing various analogue computation, which can be used to perform various tasks far more efficiently than performing an equivalent computation using DREG and digital operations.

Example : AREG Decay

This example showcases analogue decay. The user can store the captured image to an AREG plane, and then observe how this data decays with time. A thresholded binary image is also stored to a DREG plane to show by comparison this digital data will not decay (provided the DREG registers storing the data are periodically refreshed).

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



Analogue Saturation

The analogue memory registers (AREG) inside each pixel processor are capable of storing continuous values. However, there is a both minimum and maximum limit to the values that can be stored, and an “accurate working range” of values inside these limits. Operations involving or producing values outside this range will produce inaccurate and distorted results. These effects that essentially lead to a loss of information are referred to as “Saturation”.

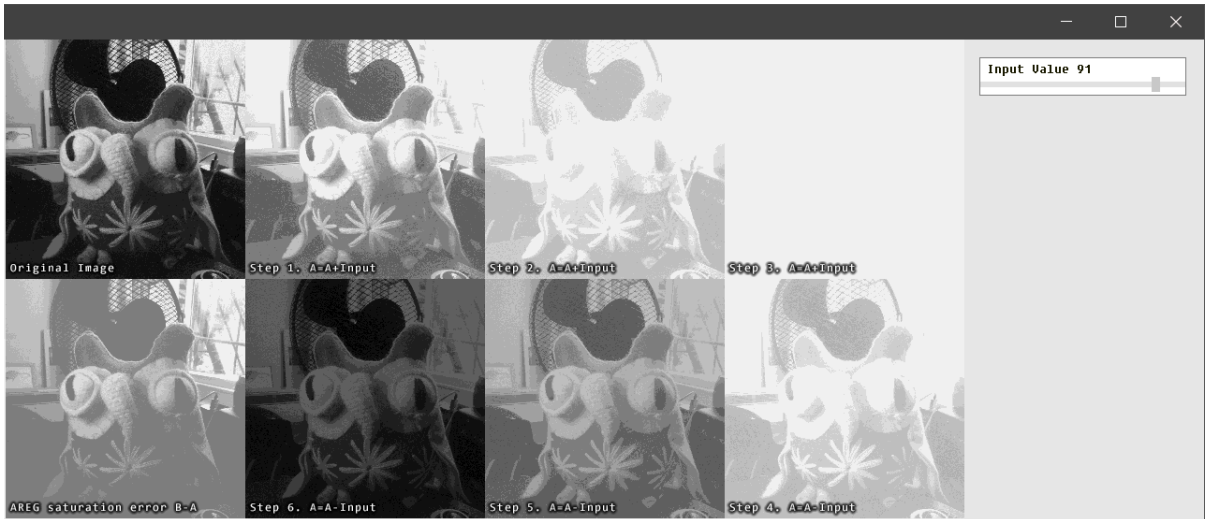
A simple example of saturation can be observed when repeatedly adding positive values to an analogue register (AREG). As these additions accumulate, the register eventually reaches its maximum limit. Beyond this point, further additions will not increase the register’s value, resulting in no observable change, effectively losing all information of the addition.

Example Code : AREG Saturation

The example here demonstrates saturation by adding a fixed value to each pixel of a captured image three times, then subtracting the same value three times. Ideally, these addition and subtraction operations should cancel each other out, with the final results being identical to the original image. However, when the fixed value being used is too big (or too small), the repeated additions can cause the AREG to saturate, leading to discrepancies between the final image and the original, as shown below.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**





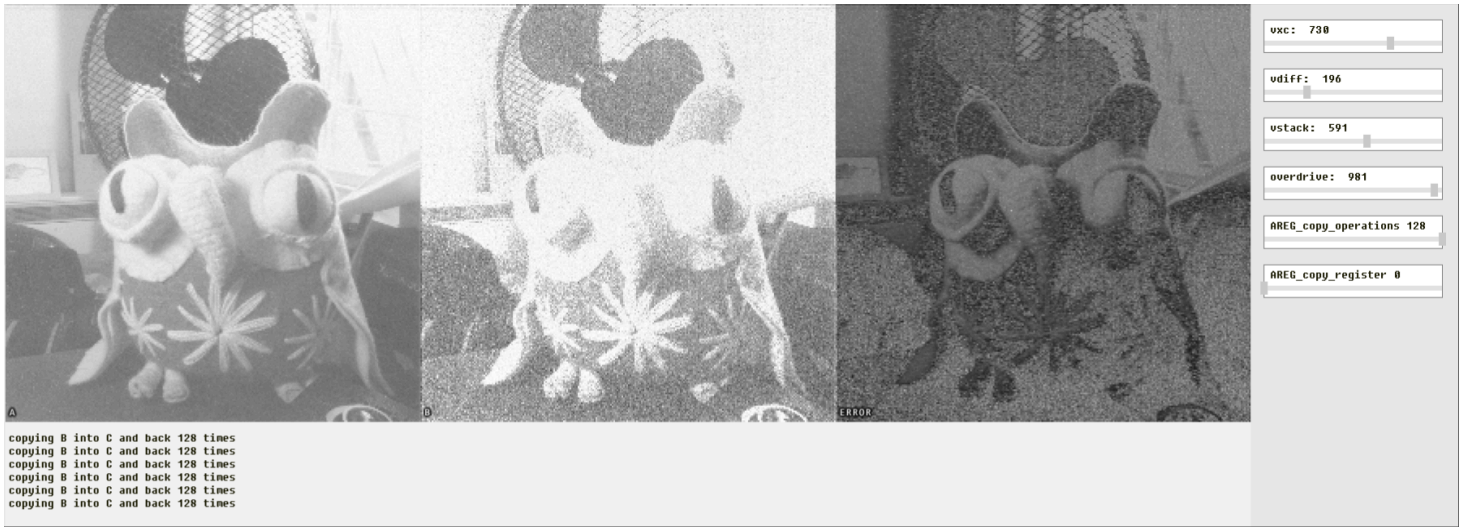
Degradation

Every analogue operation introduces a small amount of noise into the result, making it impossible to achieve perfect accuracy. When a large number of analogue operations are performed sequentially, with each operation building upon the previous one, this noise can gradually accumulate. In such a scenario each analogue operation will decrease the data's signal to noise ratio, eventually rendering the results unusable. This should be kept in mind when performing such long chains of analogue computation, and noise mitigation strategies such as periodic quantization might be required.

Example Code : AREG Degradation

The example here demonstrates such Degradation, by capturing an image then repeatedly copying it back and forth between different AREG planes. Noise will accumulate with each copy, leading to the image progressively degrading. We can see below the original image (left) and how the image is degraded after being copied 120 times (middle).

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**





This program also allows the user to select which AREG is being used for copying, each resulting in a different pattern of degradation noise as shown by the two images. There are a wide range of variables determining the noise of analogue operations, and these will even vary between SCAMP boxes, making accurate noise modelling very challenging.

AREG Quantization

As described in a previous section, the continuous values stored within analogue memory will gradually drift away from their original value. In SCAMP AREG, this leads to an eventual loss of data. However we can attempt to mitigate this drift by quantizing the analogue data to a set of equally spaced discrete values (or “quantization levels”). This involves overwriting each piece of analogue data with the closest value from this discrete set.

By performing this quantization periodically, each piece of analogue data will be “snapped” to the same discrete value repeatedly, and will not have time to drift into a different quantization level. This process effectively “refreshes” the data, locking it into a set of levels and preventing it from decaying.

There are two intertwined aspects to consider when using this approach

1. The time between each “refresh” (quantization)
2. The number of quantization levels used

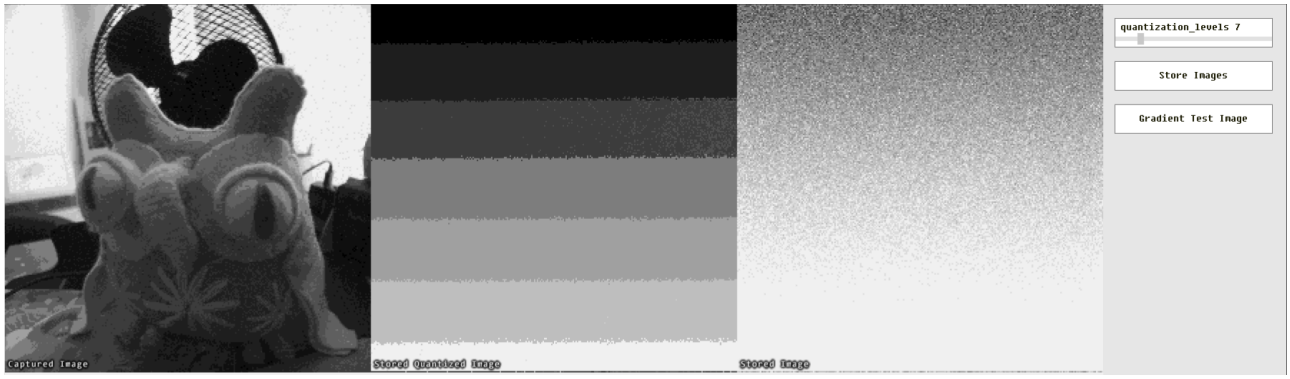
The greater the number of quantization levels, the smaller the spacing between them, and the less time needed for analogue data to drift across them. Thus as the number of levels increases, the frequency of performing “refresh” quantizations must also increase to ensure data remains locked in place. For example, data quantized to 16 values must be refreshed far more often than data quantized to 3 values.

Example Code : Quantized Analogue Storage

This example demonstrates using analogue quantization for long term storage of analogue data. The user can select the number of levels of quantization and store two copies of captured image in two different AREGs. One quantized each frame to prevent it decaying, while the other is not. The content of both these AREG are then displayed side by side as shown below, with the quantized AREG in the middle and decaying non-quantized on the right. The user can also generate and store a gradient image as a test image for this quantization.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**





The section of code performing quantization of AREG B is shown below. This takes each PE's analogue data, and "snaps" it to the closest quantized value. This involves a loop iterating across the quantization values, from lowest to highest, and using a DREG "mask" to keep track of which PEs have not yet had their data quantized.

At each iteration of this loop, each PE's data is compared against a "decision value", to determine if it should be overwritten by the current quantization value. This decision value is placed halfway between the current and next quantization values. However a PE's data is only overwritten if it has not already been quantized. After this loop is complete, any PE's that still have not been quantized are overwritten by the maximum quantization value.

The DREG S6 is used to track which PEs have been quantized. The comparison performed each iteration between PE data and decision value is executed by the lines "sub(F,F,B)" followed by "Where(F)". This is combined with the mask of PEs yet to be quantized by executing "AND(S5,S6,FLAG)", giving the PE's whose data should be overwritten by the current quantization value.

```
scamp7_kernel_begin();
      SET(S6);//Use the S6 DREG to track which PEs have been quantized
scamp7_kernel_end();

int quantization_interval = 256/(quantization_levels-1);
//Value of the current quantization level & value to compare against data
int current_value = -127;
int decision_value = current_value + quantization_interval/2;

//Iterate through each quantization level, from lowest to highest
//Quantize data in PEs closest to each level by comparing to descion_value
for(int n = 0 ; n < quantization_levels-1 ;n++)
{
      scamp7_in(F,decision_value);
      scamp7_in(E,current_value);
      scamp7_kernel_begin();
          sub(F,F,B);//Decision_value - Data
          where(F);//Compare stored data with decision value
                AND(S5,S6,FLAG);//S5 = PEs to quantize to current level
                NOT(S6,FLAG);//Update mask of quantized PEs before FLAG is changed
                WHERE(S5);
                      mov(B,E);//Quantize data in PEs where S5 = 1 to current level
          all();
      scamp7_kernel_end();
      current_value += quantization_interval;//update current quantization value
      decision_value = current_value + quantization_interval/2;//update decision value
}
//snap all remaining PEs that have not been quantized to the max quantization value
scamp7_in(E,127);
scamp7_kernel_begin();
      WHERE(S6);
      mov(B,E);
      all();
scamp7_kernel_end();
```

Example Code : Background Subtraction

This example demonstrates simple background subtraction. Here “background” means those static, unchanging parts of the scene observed by the sensor (assuming it itself is stationary). By then subtracting this background from the current image, we are (ideally) left with an image of only dynamic “foreground” objects. The background image itself is stored using the same long term analogue data storage shown in the previous example, and is dynamic allowing the sensor to be moved or new objects to be placed into the background.

The stored background is updated every frame, but only in those PEs where the observed image is stable and has not changed for some length of time. This is done using frame differencing, as seen in an earlier example, to detect the location of any motion occurring within the image. The background should then not be updated in any PE around which motion has recently occurred. To achieve this another AREG plane is used as an “update mask” for the background. This mask gradually increases in value over time across all PEs, but is reset to a minimum value inside any PE around which motion occurs. The stored background is then only updated in PEs where this analogue mask has reached a threshold value.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



● *Optimization Basics*

Kernel Calling Cost

TLDR: its about 0.3µs on both SCAMP5 & SCAMP7

By now you should be familiar with using SCAMP instruction kernels. The time to execute such kernels will of course depend on the number of instructions they contain. However there is also a time cost associated with simply executing a kernel, referred to as the “calling” cost. Let us consider two pieces of code.

Here a small kernel containing two instructions is called 100 times.

```
//CODE 1
for(int n = 0 ; n < 100 ; n++)
{
      scamp7_kernel_begin();
            DNEWS1(S6,S1);
            MOV(S1,S6);
      scamp7_kernel_end();
}
```

Here a large kernel containing 200 Instructions is called once.
(The loop placed inside the kernel generates 200 instructions, letting us avoid writing out 200 instructions…)

```
//CODE 2
scamp7_kernel_begin();
      for(int n = 0 ; n < 100 ; n++)
      {
            DNEWS1(S6,S1);
            MOV(S1,S6);
      }
scamp7_kernel_end();
```

Both of these pieces of code will execute the exact same SCAMP instructions, however the number of kernel calls involved is vastly different. The first involves 100 kernel calls, while the second calls only a single kernel. Roughly speaking, the execution times for each piece of code will then be as follows.

Code 1 Execution Time = **Cost Of Instructions** + 100 x **Kernel Calling Cost**
Code 2 Execution Time = **Cost Of Instructions** + **Kernel Calling Cost**

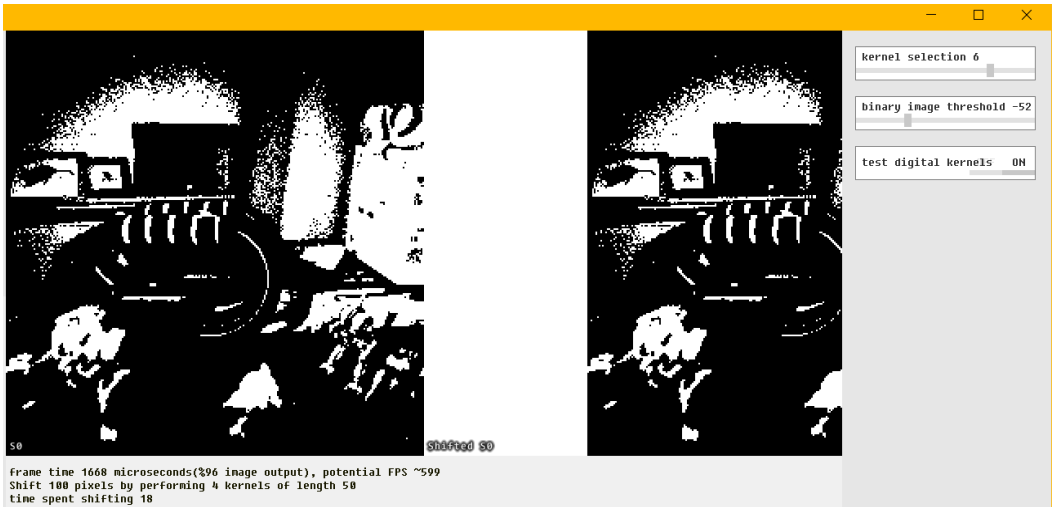
It should be clear from this that the second piece of code will execute faster due to the greatly reduced number of kernel calls. By comparing results this also allows us to experimentally measure the actual calling cost which comes out to approximately 0.3µs (microseconds). This may seem negligible, but when optimizing code for maximum speed this can become a significant factor. Programs which repeatedly call small kernels a large number of times, are particularly bad cases. This often occurs when small kernels are placed inside of loops involving a large number of iterations such as in the first piece of code.

As a general rule it is almost always more efficient to call a small number of large kernels (each containing many instructions), than calling a large number of small kernels (each containing only a handful of

instructions). This idea is demonstrated in the next example, in which the exact same SCAMP instructions are executed but the number of kernel calls involved can be altered.

Example Code : Kernel Calling Cost

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**

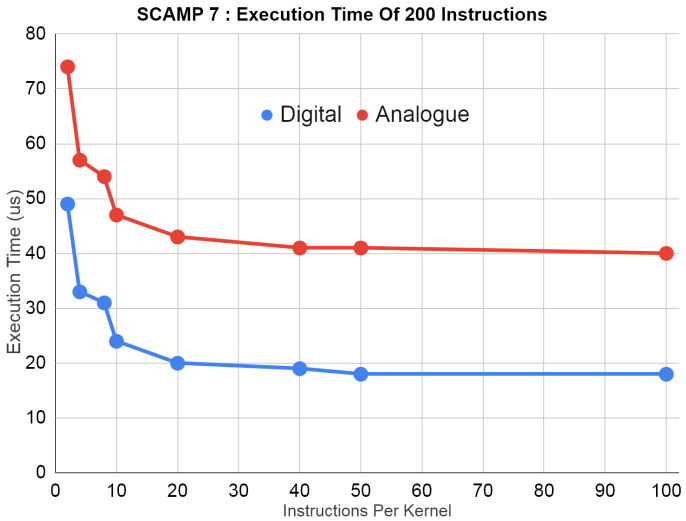


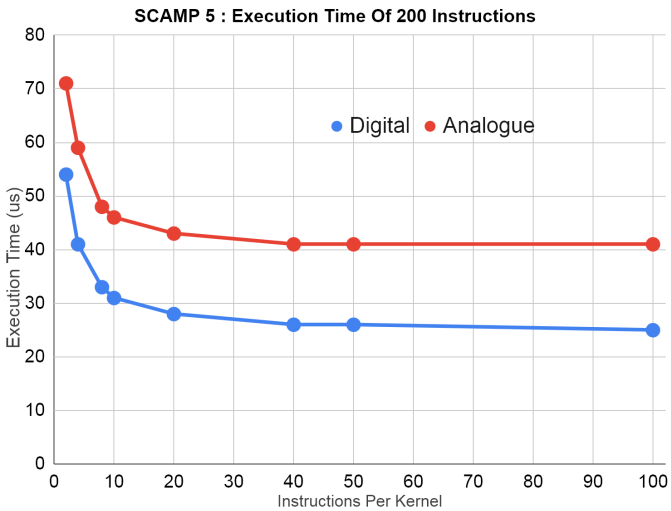
This example simply captures an image, and shifts it 100 PEs/Pixels to the right. The slider “kernel selection” allows the user to select from a range of different sized kernels to perform this shifting. This results in different execution times, printed to the text output.

```
Shift 100 pixels by performing 50 kernels of length 4
time spent shifting 34
```

The user can also switch between shifting analogue data using NEWS AREG and bus, or shifting digital data using DNEWS. This is important to investigate as kernels which contain only digital instructions can be executed faster than kernels containing any analogue instructions.

Plotting the execution time to perform this shifting against the size of the kernel used we obtain the following graphs for SCAMP-5 & SCAMP-7 respectively.





This shows how the execution time involved in shifting decreases with kernel size, at first rapidly, but then tapering off around kernels of size 20-40. Digital instructions particularly benefit from this, with execution time being cut in half going from kernel size 2 to kernel size 20 (2 instructions vs 20 instructions). We also observe how digital instructions are executed faster than analogue, and are faster on SCAMP-7 compared to SCAMP-5.

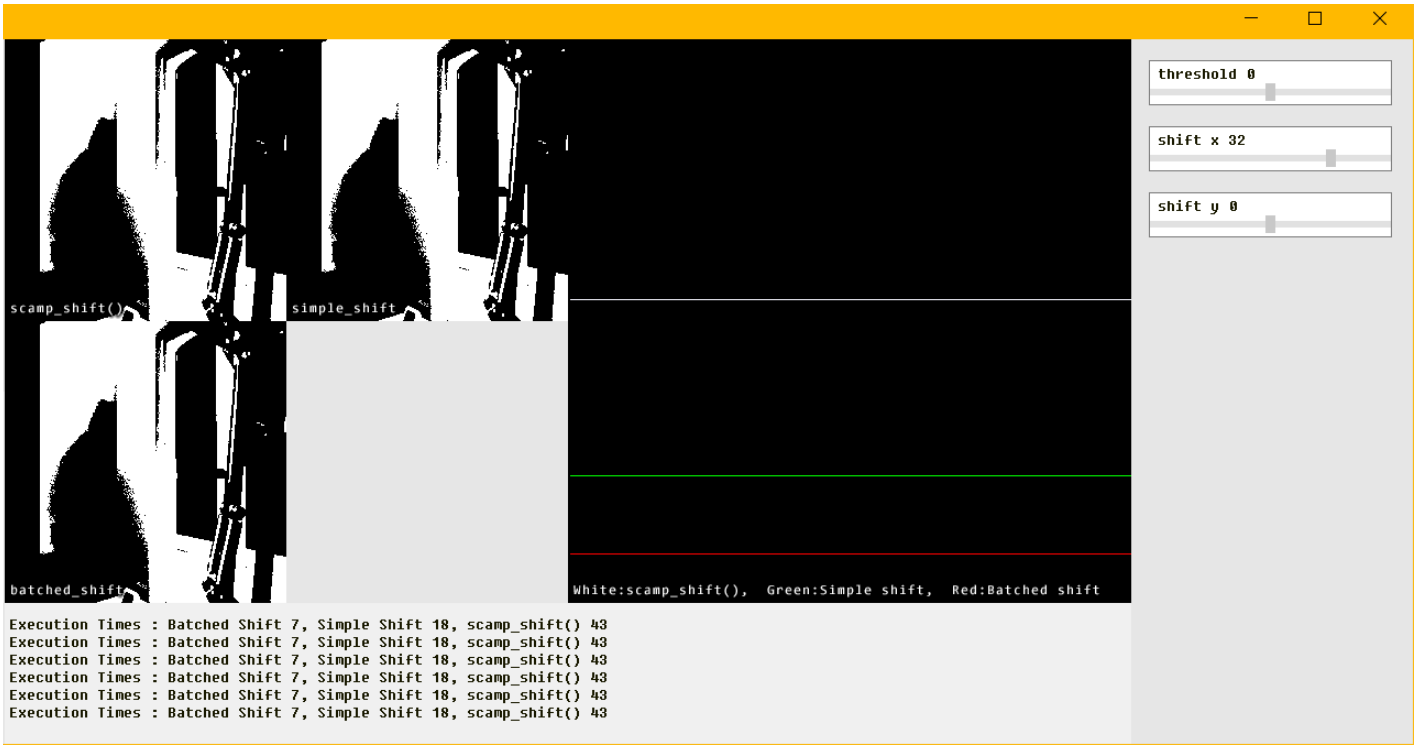
Kernel Compilation Note

Kernels within a program are “compiled” during runtime when each is called for the first time. This compilation essentially decodes each instruction within the kernel into assembly instructions that can be sent to the PE array. Whenever a kernel is called again its compiled list of instructions can be directly sent to the PE array. This means any C++ code within a kernel will only be executed once when the kernel is compiled. Because of this it is almost never a good idea to place conditional or branching C++ code inside of kernels. Further any loops placed inside of kernels should perform a fixed number of iterations.

Example : Optimising Shifting (Digital)

We have seen the performance cost of performing kernel calls, now let us look at an example of optimizing code to minimize the number of these calls to increase performance. This example shifts digital data using DNEWS and three different functions the user can switch between. As each of these functions involves a different number of kernel calls, each will have a different execution time when performing the same shifting operation. Let us describe each briefly.

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



1.Naive Shifting

The function “simple_shift_S5(int shiftx, int shifty)” shifts the content of a DREG plane using a simple for loop to execute DNEWS instructions. First data is shifted horizontally by correctly setting up the DNEWS direction control registers (RN,RS,RE,RW) and then performing the DNEWS loop shown below for appropriate number of iterations. A similar process is then performed to shift data vertically.

```
for(int i = 0 ; i < iterations ; i++)
{
      //Perform a single DNEWS instruction, then copy the result back with MOV
      scamp5_kernel_begin();
          DNEWS0(S6,S5);
          MOV(S5,S6);
      scamp5_kernel_end();
}
```

Inside this loop is a short kernel, called every iteration, which simply shifts the data one PE/pixel. This means the number of kernel calls needed to shift the data is equal magnitude the shift (in terms of Manhattan distance). For example shifting the data 50 pixels horizontally would require 50 kernel calls!

2.Batched Shifting

The previous naive method can be inefficient due to the very large number of kernel calls when shifting data long distances. To remedy this we can incorporate kernels that shift larger distances, performing multiple DNEWS instructions within a single kernel. This involves now using a While loop containing multiple kernels, each shifting a particular distance such as shown in the code snippet below.

```
while(remaining_iterations > 0)
{
      if(remaining_iterations % 8 == 0)//Check if remaining shift is divisible by 8
      {
          scamp7_kernel_begin();//Kernel to shift data 8 times
                  DNEWS0(S6,S5);
                  DNEWS0(S5,S6);
                  DNEWS0(S6,S5);
                  DNEWS0(S5,S6);
                  DNEWS0(S6,S5);
                  DNEWS0(S5,S6);
                  DNEWS0(S6,S5);
                  DNEWS0(S5,S6);
            scamp7_kernel_end();
            remaining_iterations -= 8;//Update remaining shift
      }
      else
      {
            if(remaining_iterations % 4 == 0)//if remaining shift is divisible by 4
            {
                  scamp7_kernel_begin();//Kernel to shift data 4 times
                        DNEWS0(S6,S5);
                        DNEWS0(S5,S6);
                        DNEWS0(S6,S5);
                        DNEWS0(S5,S6);
                  scamp7_kernel_end();
                  remaining_iterations -= 4;//Update remaining shift
            }
            else
            {
                  //Cases for remaining shift is divisible by 2, or == 1
            }
      }
}
```

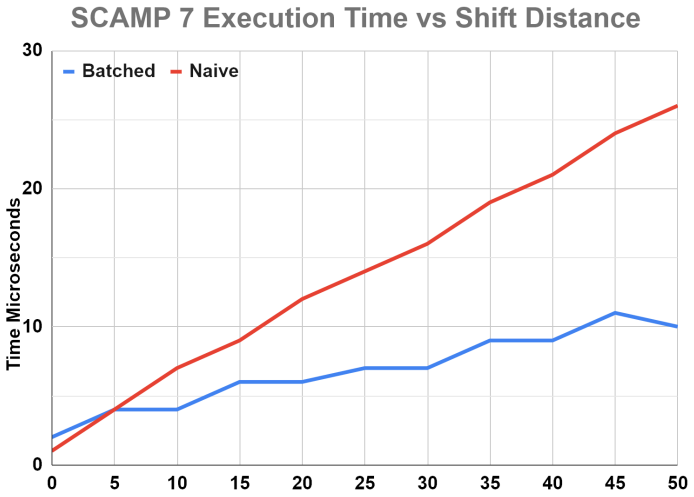
In each iteration of this loop we test if the remaining distance to shift is divisible by 8, and if so call a kernel which shifts the data 8 times. We similarly test if the remaining shift is divisible by 4, and by 2, calling kernels to shift the appropriate distances in each case.

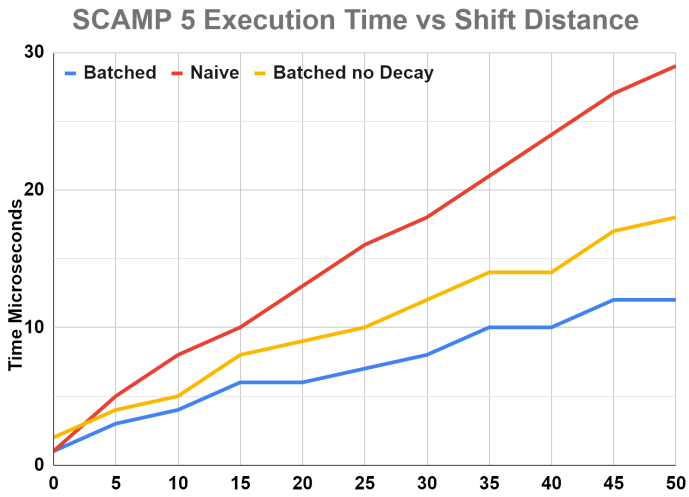
Such an approach significantly reduces the number of kernel calls, thus reducing execution time. As an example, when shifting data 50 times to the right this approach would only call 7 kernels. The loop's first iteration would execute the kernel to shift a distance of 2 (as 50 is divisible by 2), leaving the remaining shift distance at 48. The loop's second iteration would execute the kernel to shift a distance of 8 (as 48 is divisible

by 8), leaving the remaining shift distance at 40. The 3rd, 4th, 5th, 6th & 7th would similarly call the kernel to shift a distance of 8, reducing the remaining shift to 32, 24, 16, and finally 0.

Execution Time Comparison

The graphs below show the execution times of shifting digital data using these two approaches. It is clear that the rate at which execution cost increases with shift distance is smaller for the batched approach, making it the preferred choice in almost all situations. For shifts of distances 20 or more, this batched approach is over twice as fast. Again remember the exact same sequences of SCAMP instructions are being executed in both approaches, the only difference being the number of kernel calls involved.





SCAMP5 unfortunately is more prone to digital register decay when executing sequential DNEWS instructions. Due to this the kernels within the While loop of the batched shifting approach must be modified to avoid this decay. This involves longer kernels which do not have back to back DNEWS instructions, increasing execution time. However, this still is significantly faster than the naive approach. The images below show the errors this decay causes on SCAMP5 when shifting a long distance.





Example : Optimising Shifting (Analogue)

EXAMPLE CODE SCAMP5 EXAMPLE CODE SCAMP 7

Example : Optimising Scaling

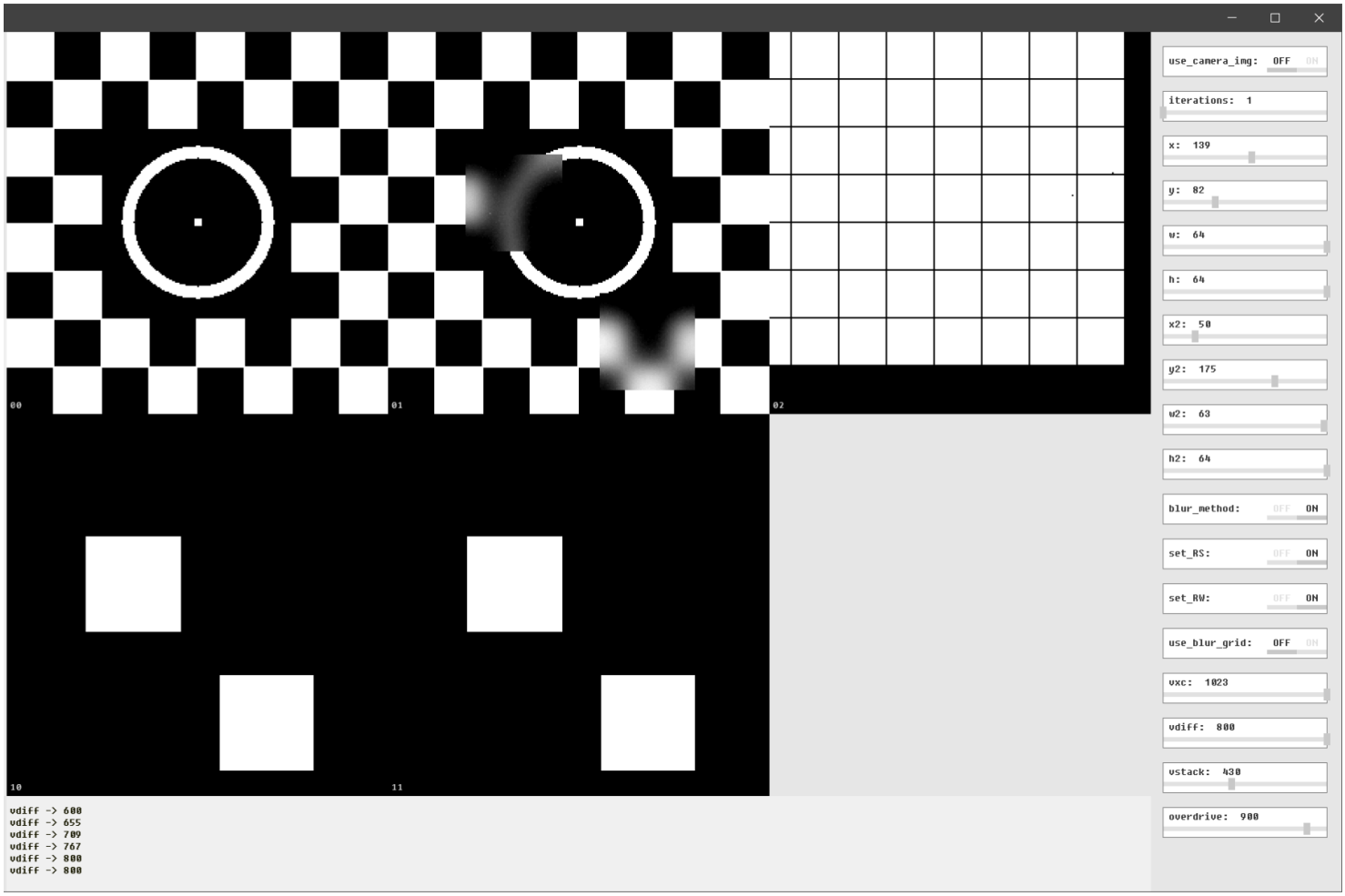
Example : Convolutions

Example : Highest Valued Pixel

EXAMPLE : NMS

Analogue Diffusion

EXAMPLE CODE SCAMP5 **EXAMPLE CODE SCAMP 7**



DREG Drawing

DYNAMIC KERNELS

The VXC Slider…

VERY HIGH SPEED AREG OUT (1BIT AT A TIME)

Register Conventions, WORKING VS "PERMANENT" DATA, IE S0,S1 VS S5,S6

PIX AREG & Light Capture

IMAGE GAIN

COLORED DREG DRAWING!

DREG DISCUSSION

These DREG do not suffer from the degradation issues faced by AREG, however only store 1-bit values. By comparison the range of values an AREG can represent is considered roughly equivalent to 8 bits.

Multiple DREG can be used in conjunction to represent a multi-bit value, and addition and subtraction routines can be built from DREG logic operations (See). However such an approach is extremely slow compared to the AREG equivalent, where addition and subtraction is achieved in a single instruction.